Dossier del Trabajo Integrador Final: caso FitZone Sports

Universidad Nacional de Entre Ríos

Facultad de Ciencias de la Administración

Licenciatura en Sistemas

Programación V, año 2026

Equipo 08

Apellido y nombre	Legajo	Rol	Usuario de Git
Zanandrea, Martín	_____	P1 (Arquitecto / Backend Lead); asume también el rol P2 (Backend Developer)	Martinzanandrea
Ava, Teo	_____	P3 (Frontend Developer)	Teo
Retamoso, Máximo	_____	P4 (QA / DevOps / Mobile)	maxi

Fecha de entrega: 19/11/2026

URL del repositorio: https://github.com/Martinzanandrea/FitZoneSports

Tag: v1.0-entrega

Hash del commit: `b3c4419bc6aef65079ed68eb5a5b36391127bd49`

Tabla de contenido

1. Resumen ejecutivo del sistema **5**

2. Arquitectura: diagramas C4 de los niveles 1, 2 y 3 ..... **6**

2.1 Nivel 1: contexto del sistema 6

2.2 Nivel 2: contenedores 6

2.3 Nivel 3: componentes 7

3. Registros de decisión arquitectónica: ADR-001 a ADR-023 ..... **10**

Índice de decisiones 10

ADR-001 — Monolito modular como estilo arquitectónico 11

ADR-002 — PostgreSQL como motor de base de datos relacional 13

ADR-003 — Estrategia offline del control de acceso 15

ADR-004 — Control de concurrencia en la reserva de canchas: bloqueo pesimista más índice único 17

ADR-005 — NestJS como framework backend, su contenedor de IoC y TypeORM como ORM 19

ADR-006 — React con Vite, componentes con Atomic Design y estado global con Context 21

ADR-007 — Sincronización del terminal de sede con el servidor central 22

ADR-008 — Estrategia de pruebas, pirámide adoptada y umbral de cobertura 25

ADR-009 — Despliegue en plataformas administradas, Docker para desarrollo y corrección, y procedimiento de reversión 27

ADR-010 — Tratamiento de los medios de pago: pasarela simulada, tokenización y sin datos de tarjeta 29

ADR-011 — Aplicación web progresiva (PWA) en lugar de una app móvil nativa 30

ADR-012 — GitHub como repositorio compartido (monorepo) 32

ADR-013 — Modelado de datos: UUID, sede parcial, tipos exactos y exclusive arc 33

ADR-014 — Autenticación con JWT en cookie httpOnly y autorización por rol, dueño y sede 35

ADR-015 — Flujo obligatorio de membresía y pago, con recuperación ante pago rechazado 37

ADR-016 — Cálculo de montos de pago exclusivamente en el servidor 39

ADR-017 — Sistema de auditoría en dos capas: interceptor de aplicación y trigger de base 41

ADR-018 — Clases grupales como plantillas recurrentes, no eventos únicos 42

ADR-019 — Verificación de email con Gmail SMTP, en vez de Resend o Auth0 44

ADR-020 — Jerarquía de roles de gestión: Gerente General y Gerente de Sede 46

ADR-021 — Envío de correo mediante proveedor abstracto (Brevo en producción, Gmail en desarrollo) 48
ADR-022 — Vigencia de membresías por fechas y no superposición 51
ADR-023 — Almacenamiento de archivos intercambiable (Supabase Storage o disco local) 54

4. Modelado: clases, secuencia, estados, actividad, DER y despliegue 57
4.1 Diagrama de clases 57
4.2 Diagramas de secuencia 58
4.3 Diagramas de estados 60
4.4 Diagrama de actividad 62
4.5 Diagrama entidad-relación (DER) 63
4.6 Diagrama de despliegue 68

5. Patrones de diseño aplicados y su justificación 69
Strategy — precio dinámico de canchas 69
Observer — lista de espera de clases 70
Repository — abstracción de la persistencia 70
Patrones creacionales y estructurales 71

6. Estrategia de pruebas y reporte de cobertura 72
6.1 Niveles de prueba y resultados 72
6.2 Cobertura del backend 72
6.3 Pipeline de integración y entrega continuas 73
6.4 Otros informes de pruebas 74

7. Cumplimiento de los requisitos no funcionales y su evidencia 75
7.1 Evidencia disponible en docs/rnf 75
7.2 Evidencia pendiente 77

8. Planificación: plan, cronograma, trazabilidad, seguimiento y riesgos 78
Plan de proyecto del Trabajo Integrador Final: caso FitZone Sports 78

9. Bitácoras individuales de los integrantes 92
Bitácora — Zanandrea, Martín — Rol: Backend lead 92
Bitácora — Ava, Teo — Rol: Frontend 105
Bitácora — Retamoso, Máximo — Rol P4 111

10. Índice de los manuales entregados y su ubicación en el repositorio 120
Manual de usuario 120
Manual de instalación y operación 120
Manual técnico 121

11. Anexo: tablero del proyecto exportado 122
11.1 Nota de transparencia 122
11.2 Resumen por responsable 122

11.3 Sprints 123
11.4 Tarjetas 124
Referencias **129**

1. Resumen ejecutivo del sistema

FitZone Sports es la plataforma de gestión de una cadena de gimnasios con más de 25 sedes, planteada como caso del Trabajo Integrador Final de Programación V (Universidad Nacional de Entre Ríos [UNER], 2026a). Los socios contratan su membresía, ingresan a cualquier sede con un código QR que cambia cada minuto, reservan clases grupales y alquilan canchas de paddle y fútbol 5. El personal de cada sede valida ingresos, controla el aforo y cobra en el mostrador, y la gerencia administra sedes, precios y reportes consolidados.

El sistema es un monolito modular NestJS con TypeORM sobre PostgreSQL, con un cliente React instalable como PWA que funciona como portal del socio y consola de administración. Las reglas críticas se garantizan en la base de datos: no hay sobreventa de canchas (RN-02) ni dos sesiones de acceso abiertas para un mismo socio (RN-01). El control de acceso sigue operando sin conexión mediante QR firmados con ES256, una réplica local y una cola de ingresos que se concilia al reconectar (RNF-01). Los pagos usan una pasarela simulada que conserva solo el token (RNF-02).

La calidad se respalda con 848 pruebas unitarias en el backend (73,65 % de sentencias cubiertas), 153 en el frontend, 18 de integración contra PostgreSQL y un pipeline de integración continua. La documentación reúne 23 ADR, diagramas C4, UML y DER, tres manuales y las bitácoras de los tres integrantes del Equipo 08.

2. Arquitectura: diagramas C4 de los niveles 1, 2 y 3

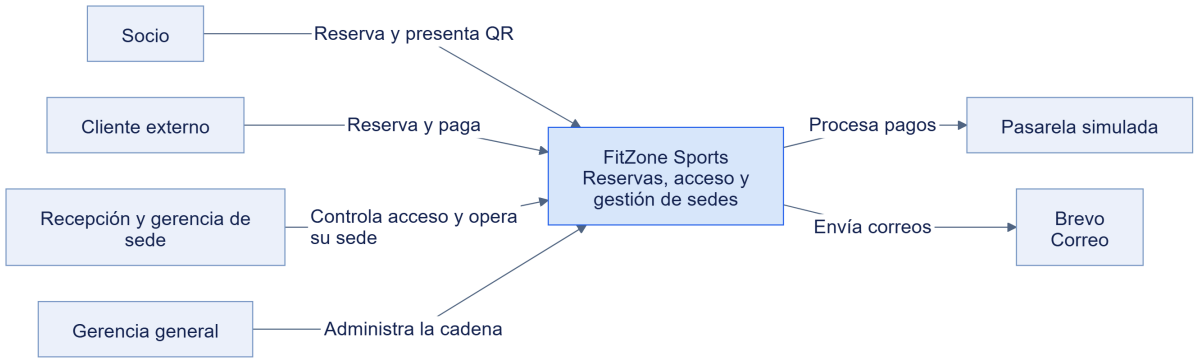
La arquitectura se describe con el modelo C4 (Brown, s. f.): el nivel 1 muestra el sistema y su entorno, el nivel 2 los contenedores que lo componen y el nivel 3 los componentes de dos módulos representativos. La pila tecnológica es NestJS (s. f.) con TypeORM (s. f.) sobre PostgreSQL (PostgreSQL Global Development Group, s. f.) en el servidor, y React (Meta Platforms, Inc., s. f.) con Vite (s. f.) en el cliente. El estilo elegido es un monolito modular (ADR-001), de modo que una transacción local resuelve las reglas RN-01 y RN-02 sin coordinar servicios.

2.1 Nivel 1: contexto del sistema

El sistema interactúa con el socio, el cliente externo, el personal de recepción y gerencia de sede, la gerencia general, una pasarela de pago simulada y el proveedor de correo Brevo.

Figura 1

C4 nivel 1: contexto de FitZone Sports



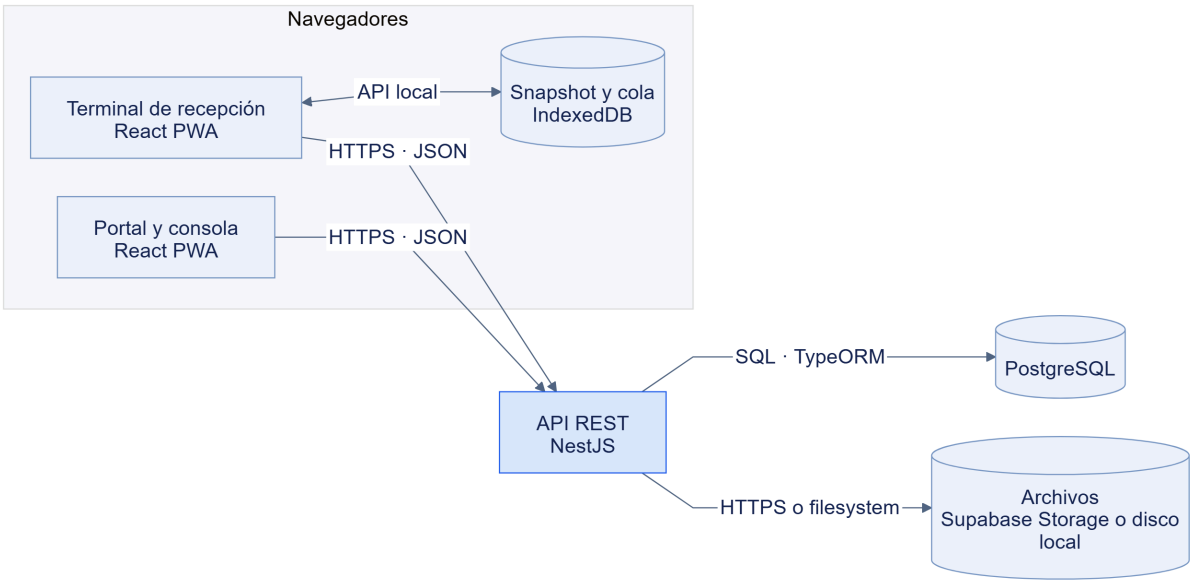
Nota. Elaboración del Equipo 08 (Retamoso, Máximo, 09/11/2026, v3.0) con Mermaid. Fuente editable: `docs/diagramas/c4-1-contexto.mmd`. La imagen se incluye en su resolución original y puede ampliarse en el visor del PDF.

2.2 Nivel 2: contenedores

Dos aplicaciones React PWA (el terminal de recepción, con instantánea y cola locales en IndexedDB, y el portal con la consola) consumen la API REST NestJS por HTTPS. La API accede a PostgreSQL por SQL con TypeORM y a un almacén de archivos (Supabase Storage o disco local).

Figura 2

C4 nivel 2: contenedores



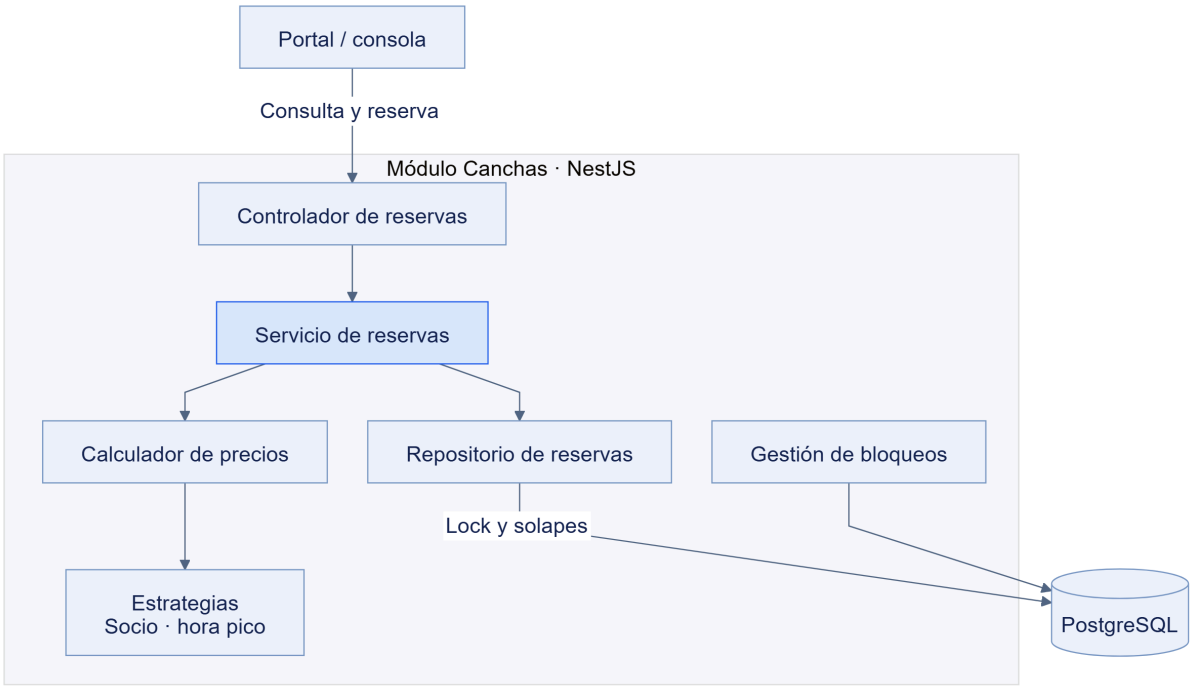
Nota. Elaboración del Equipo 08 (Retamoso, Máximo, 09/11/2026, v3.0) con Mermaid. Fuente editable: `docs/diagramas/c4-2-contenedores.mmd`. La imagen se incluye en su resolución original y puede ampliarse en el visor del PDF.

2.3 Nivel 3: componentes

Se detallan dos módulos: Canchas, donde se concentra la regla de precio y la reserva sin sobreventa, y Acceso, que sostiene el ingreso por QR sin conexión.

Figura 3

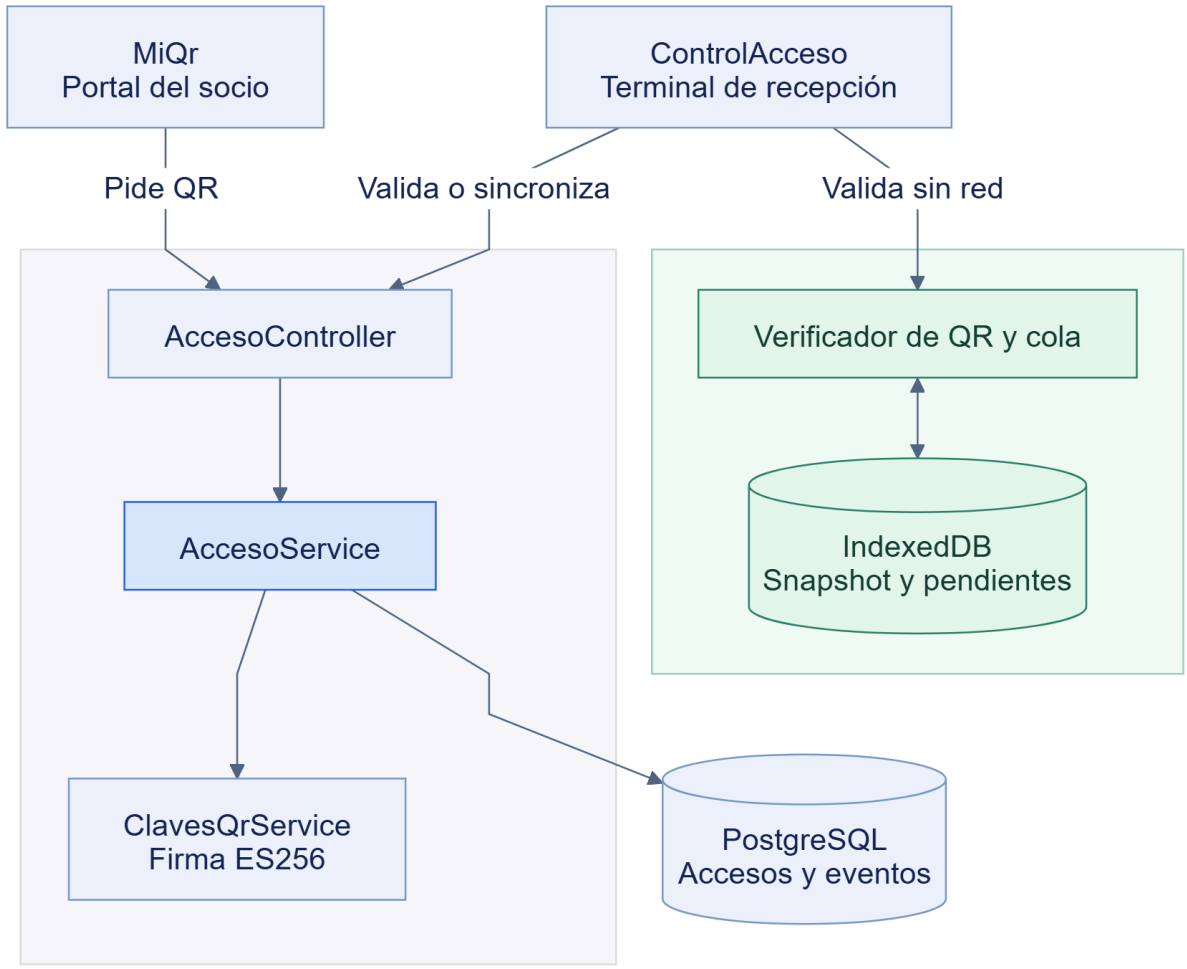
C4 nivel 3: componentes del módulo de Canchas



Nota. Elaboración del Equipo 08 (Retamoso, Máximo, 09/11/2026, v2.0) con Mermaid. Fuente editable: `docs/diagramas/c4-3-componentes-canchas.mmd`. La imagen se incluye en su resolución original y puede ampliarse en el visor del PDF.

Figura 4

C4 nivel 3: componentes del módulo de Acceso



Nota. Elaboración del Equipo 08 (Retamoso, Máximo, 09/11/2026, v2.0) con Mermaid. Fuente editable: `docs/diagramas/c4-3-componentes-acceso.mmd`. La imagen se incluye en su resolución original y puede ampliarse en el visor del PDF.

3. Registros de decisión arquitectónica: ADR-001 a ADR-023

Cada decisión se documentó con la plantilla del Anexo B de los lineamientos (UNER, 2026b), que sigue la propuesta de registros de decisión de Nygard (2011): estado, fecha, autor, sprint, contexto, alternativas consideradas, decisión, consecuencias y requisitos relacionados. Un ADR no se corrige, se supera. Los ADR-001 a ADR-010 son los obligatorios; los ADR-011 a ADR-023 son decisiones propias del desarrollo.

Índice de decisiones

Obligatorios (ADR-001 a ADR-010)

Tabla 1

Obligatorios (ADR-001 a ADR-010) (ADR)

ADR	Decisión	Estado	Requisitos
ADR-001	Monolito modular frente a microservicios, SOA y serverless	Aceptado	RN-02, RNF-04
ADR-002	PostgreSQL por integridad transaccional	Aceptado	RNF-03
ADR-003	Estrategia offline del control de acceso	Aceptado	RNF-01
ADR-004	Bloqueo pesimista e índice único en la reserva de canchas	Aceptado	RN-02
ADR-005	NestJS, contenedor de IoC y TypeORM	Aceptado	Unidad II
ADR-006	React, Atomic Design y estado global con Context	Aceptado	Unidad IV
ADR-007	Sincronización del terminal de sede	Aceptado	RNF-01, RN-01
ADR-008	Pirámide de pruebas y umbral de cobertura	Aceptado	Unidad V
ADR-009	Despliegue, Docker y reversión	Aceptado	Unidad V
ADR-010	Pasarela simulada y tokenización sin datos de tarjeta	Aceptado	RNF-02

Decisiones propias del desarrollo

Tabla 2

Decisiones propias del desarrollo (ADR)

ADR	Decisión	Estado
ADR-011	PWA en lugar de app móvil nativa	Aceptado
ADR-012	GitHub como repositorio compartido (antes ADR 0001)	Aceptado
ADR-013	Modelado de datos: UUID, sede parcial, ENUM, NUMERIC, exclusive arc (antes 0004)	Aceptado
ADR-014	JWT en cookie httpOnly y autorización por rol, dueño y sede (antes 0005)	Aceptado
ADR-015	Flujo obligatorio de membresía y pago (antes 0006)	Aceptado

ADR	Decisión	Estado
ADR-016	Montos calculados en el servidor (antes 0007)	Aceptado
ADR-017	Auditoría en dos capas (antes 0008)	Aceptado
ADR-018	Clases como plantillas recurrentes (antes 0009)	Aceptado
ADR-019	Gmail SMTP para verificación de email (antes 0010)	Reemplazado por ADR-021
ADR-020	Gerente General y Gerente de Sede (antes 0011)	Aceptado
ADR-021	Correo con proveedor abstracto (antes 0012)	Aceptado
ADR-022	Vigencia de membresías por fechas (antes 0013)	Aceptado
ADR-023	Almacenamiento de archivos intercambiable	Aceptado

Renumeración

Los ADR 0001 a 0013 se renumeraron para que los diez obligatorios ocupen los números que pide la cátedra. Cada archivo conserva su historial (`git log --follow`) y una nota con su número original. El ADR 0003 (stack) se desglosó en ADR-002 (base de datos), ADR-005 (backend) y ADR-006 (frontend).

ADR-001 — Monolito modular como estilo arquitectónico

- **Estado:** Aceptado
- **Fecha:** 2026-08-24
- **Autor:** Zanandrea, Martín
- **Sprint:** 2

Contexto

FitZone Sports gestiona reservas de canchas y clases donde la sobreventa (RN-02) es un riesgo crítico de negocio, y el acceso a las sedes exige que un socio no esté en dos lugares a la vez (RN-01). Ambas reglas necesitan transacciones ACID sobre datos que comparten varios módulos (usuarios, membresías, canchas, pagos). El equipo era de cuatro integrantes con experiencia en TypeScript, el plazo era de 16 semanas y el sistema tenía que permitir

sumar sedes sin detenerse (RNF-04). En el debate de la semana 1 (FT1-9) se compararon cuatro estilos.

Alternativas consideradas

Tabla 3

Alternativas consideradas (ADR-001)

Alternativa	A favor	En contra	Por qué se descartó
Microservicios	Escalado y despliegue independiente por dominio	Reservas y pagos quedan en servicios distintos: RN-02 pasa a necesitar sagas o transacciones distribuidas; mucha infraestructura para 4 personas	El costo de coordinar transacciones entre servicios supera el beneficio a esta escala
SOA con bus de servicios	Integración formal entre áreas	Requiere un ESB y contratos pesados; el mismo problema transaccional que microservicios	Sobredimensionado para un único producto
Serverless (funciones)	Sin servidores que mantener, escala a cero	Sin conexiones persistentes al pool de Postgres; los locks de reserva y el cron de ocurrencias encajan mal; arranques en frío	No se adapta a transacciones con `SELECT ... FOR UPDATE` ni a tareas programadas
Monolito modular	Una transacción ACID resuelve RN-01 y RN-02; un solo despliegue; módulos con límites claros	Se escala todo junto; un fallo no controlado afecta a todo el proceso	Elegida

Decisión

Usamos un **monolito modular**: una única aplicación NestJS, desplegada como un solo proceso y organizada en módulos por dominio (usuarios, membresías, acceso, clases, canchas, pagos, sedes, precios, instructores, notificaciones, auditoría, administración, autenticación), que se comunican por servicios exportados y eventos. El criterio determinante fue poder garantizar RN-02 con una transacción local y un bloqueo de fila, sin coordinar servicios.

Consecuencias

- **Positivas:** transacciones reales para RN-01/RN-02 (ver ADR-004), despliegue simple, menor curva de aprendizaje y refactors que cruzan módulos sin versionar contratos.
- **Negativas:** escalar un módulo implica escalar toda la aplicación; un error no controlado puede afectar la disponibilidad general; la disciplina de no acoplar

módulos depende del equipo, no de la red.

- **Deuda asumida:** si un módulo (por ejemplo, acceso en sede) necesitara escalar o desplegarse aparte, habría que extraerlo y reemplazar llamadas internas por una API; la división actual en módulos de Nest lo facilita.

Requisitos relacionados

RN-01, RN-02, RNF-04

Originalmente ADR 0002. Se pasó a la plantilla del Anexo B y se sumaron las alternativas SOA y serverless discutidas en el debate.

ADR-002 — PostgreSQL como motor de base de datos relacional

- **Estado:** Aceptado
- **Fecha:** 2026-09-06
- **Autor:** Zanandrea, Martín
- **Sprint:** 2

Contexto

El sistema maneja pagos, reservas con cupo y membresías con vigencia: datos con relaciones fuertes y reglas que deben valer aunque falle la aplicación (no sobreventa, una sola sesión de acceso abierta por socio, membresías sin superposición). La consulta de disponibilidad de canchas tenía que responder en milisegundos (RNF-03) y el equipo necesitaba una base compartida por los cuatro integrantes desde el primer sprint, sin administrar servidores. Esta decisión formaba parte del ADR 0003 original (stack, hoy ADR-005); se separa para registrar contra qué se eligió.

Alternativas consideradas

Tabla 4

Alternativas consideradas (ADR-002)

Alternativa	A favor	En contra	Por qué se descartó
PostgreSQL (en Supabase)	ACID, índices únicos parciales, restricciones de exclusión, `FOR UPDATE`, tipos ENUM y NUMERIC exactos; plan gratuito administrado	Dependencia de un proveedor externo; migraciones manuales	Elegida

Alternativa	A favor	En contra	Por qué se descartó
MySQL / MariaDB	Conocido, ACID con InnoDB	Sin índices únicos parciales ni restricciones de exclusión: RN-01 y la no superposición de membresías quedarían solo en la aplicación	Menos garantías declarativas en la base
MongoDB	Esquema flexible, rápido para prototipar	Transacciones multi-documento más limitadas; sin claves foráneas; las reglas de integridad pasan al código	La integridad transaccional es el requisito central del caso
SQLite	Cero instalación	Un solo escritor a la vez, sin servidor compartido para el equipo ni para producción	No sirve para un entorno multiusuario desplegado

Decisión

Usamos **PostgreSQL** como única base de datos, alojada en Supabase en producción y en un contenedor `postgres:17` en desarrollo (ver ADR-009). El criterio determinante fue poder declarar en la propia base las reglas críticas: índice único parcial `uq_cancha_horario_confirmado` (RN-02), índice único parcial `uq_acceso_abierto_por_usuario` (RN-01), restricción de exclusión GiST para membresías (ADR-022) e índices de disponibilidad (RNF-03).

Consecuencias

- **Positivas:** las reglas de negocio críticas quedan garantizadas aunque haya un error en el código o un acceso directo a la base; tipos exactos para montos (NUMERIC); un solo motor para datos transaccionales y reportes.
- **Negativas:** producción depende de la disponibilidad de Supabase; el esquema se gestiona con scripts SQL propios en lugar de migraciones generadas por el ORM; agregar un valor a un ENUM requiere una migración.
- **Deuda asumida:** los reportes consolidados de las 25 sedes corren sobre la misma base que las transacciones; con mucho volumen habría que pasarlos a una réplica de lectura.

Requisitos relacionados

RNF-03, RN-01, RN-02, RF-13

ADR-003 — Estrategia offline del control de acceso

- **Estado:** Aceptado
- **Fecha:** 2026-11-09
- **Autor:** Retamoso, Máximo
- **Sprint:** 7

Contexto

RNF-01 exige que el control de acceso funcione aunque se caiga la conexión principal a internet (disponibilidad de 99,5 %). El QR del socio es un JWT que rota cada 60 segundos (RF-04) y que originalmente se firmaba con el secreto del servidor (HS256) y se validaba solo con un `POST /acceso/validar` al servidor central. La tabla `control_acceso` tenía las columnas `validado_offline` y `sincronizado_en`, pero ningún código las usaba: sin conexión, la pantalla de recepción bloqueaba la validación. Para validar en la sede sin el servidor, el terminal necesitaba poder verificar la firma del QR y saber si la membresía está vigente, sin conocer el secreto de firma.

Alternativas consideradas

Tabla 5

Alternativas consideradas (ADR-003)

Alternativa	A favor	En contra	Por qué se descartó
QR firmado con clave asimétrica + copia local de socios vigentes	El terminal verifica la firma con la clave pública, sin secreto; funciona sin red; RN-01 se re-verifica al sincronizar	Hay que distribuir la clave pública y una lista de vigencias a cada terminal; ventana de inconsistencia mientras dura la caída	Alternativa elegida
Mantener el QR con HS256 y copiar el secreto al terminal	Cambio mínimo en el servidor	Un terminal comprometido permitiría generar QR válidos para cualquiera	Riesgo de seguridad inaceptable
Validación manual con lista impresa o DNI	Sin desarrollo	No hay registro automático; RN-01 imposible de controlar	No cumple RNF-01 de forma verificable
Conexión de respaldo (4G) en cada sede	Sin cambios de software	Costo de infraestructura; no cubre la caída del servidor central	No resuelve la caída del backend

Decisión

Se firma el QR con **ES256** (curva P-256): la clave privada vive solo en el servidor (`ClavesQrService`, variable `QR_PRIVATE_KEY`) y la clave pública se publica como JWK en `GET /acceso/clave-publica`. El terminal de sede (la pantalla de recepción, una PWA, ver ADR-011) descarga el snapshot de su sede con `GET /acceso/snapshot/:sedeId` y guarda en IndexedDB la clave pública y la lista de socios con membresía vigente o próxima, con sus fechas de inicio y fin. El snapshot vale 24 horas desde que el servidor lo genera. Sin conexión, el terminal verifica la firma del QR con WebCrypto, su vencimiento (con tolerancia de reloj), la vigencia local de la membresía, la sesión abierta y el aforo, y registra el ingreso en una cola local con un `clientEventId`; al reconectar lo envía con `POST /acceso/sincronizar` (ver ADR-007). Los ingresos sincronizados quedan con `validado_offline = true`. El criterio determinante es que la validación offline no puede requerir ningún secreto en la sede.

Si el servidor no tiene `QR_PRIVATE_KEY` configurada, el QR se sigue firmando con HS256 y el modo offline no está disponible: `GET /acceso/clave-publica` y el snapshot responden 404.

Consecuencias

- **Positivas:** el acceso sigue funcionando con el servidor central caído; la seguridad del QR no depende de los terminales; el terminal no guarda secretos.
- **Negativas:** durante la caída RN-01 no se puede garantizar entre sedes (cada terminal solo conoce el estado del snapshot y sus propios ingresos); se resuelve al sincronizar, con el conflicto registrado. El terminal guarda datos personales mínimos (identificador, nombre, apellido y fechas de membresía) de los socios de la red.
- **Deuda asumida:** hay que rehacer el snapshot si crece la red de sedes (hoy incluye a los socios vigentes de toda la red); no hay documentado un procedimiento de rotación de la clave ES256.

Límites conocidos

- Para recargar la página sin conexión hace falta la PWA con su service worker instalado. Sin él, el navegador no puede abrir la aplicación y el terminal solo funciona

si la pantalla de control de acceso ya estaba abierta cuando se cayó la conexión.

- Los registros de `jti` usados y de códigos cortos están en memoria del servidor: no sobreviven a un reinicio ni se comparten entre varias instancias de la API. El uso único del QR offline se garantiza en la sincronización con el índice único `uq_evento_offline_jti`.
- La ventana de revocación llega a 24 horas: un socio dado de baja o suspendido puede ingresar sin conexión hasta que venza el snapshot del terminal o se renueve con conexión.
- Si una respuesta del servidor se pierde por un timeout después de haber registrado el ingreso, el terminal valida localmente y encola el mismo ingreso; al sincronizar puede terminar en un conflicto `SESION_ABIERTA`.
- La lista de ingresos offline que muestra la pantalla vive solo en el estado de la pantalla: se pierde al recargar. Los ingresos siguen en la cola de IndexedDB y en el contador de pendientes.
- Sin conexión, el aforo muestra "No se pudo cargar el aforo". El terminal no calcula el aforo local que sí usa para validar.
- No hay una prueba de punta a punta del flujo completo (corte de red real, reconexión y sincronización contra el servidor); las pruebas cubren las funciones por separado.

Requisitos relacionados

RNF-01, RF-04, RN-01

ADR-004 — Control de concurrencia en la reserva de canchas: bloqueo pesimista más índice único

- **Estado:** Aceptado
- **Fecha:** 2026-09-06
- **Autor:** Zanandrea, Martín
- **Sprint:** 2

Contexto

RN-02 exige que si dos usuarios reservan la misma cancha en el mismo horario y en el mismo segundo, solo uno tenga éxito. El flujo de reserva lee la disponibilidad, calcula el

precio y recién después inserta: sin control, dos transacciones pueden leer "libre" a la vez e insertar las dos. El mismo problema aparece en el último cupo de una clase (RF-07) y en la membresía de un socio (ADR-022). Esta decisión estaba repartida entre el ADR-001, la nota del ADR-018 y la bitácora; se registra acá de forma explícita.

Alternativas consideradas

Tabla 6

Alternativas consideradas (ADR-004)

Alternativa	A favor	En contra	Por qué se descartó
Bloqueo optimista (columna `version`)	No bloquea lecturas; bueno con poca contención	El perdedor recibe un error de versión y hay que reintentar; la fila que cambia (la reserva) todavía no existe al momento de leer, así que no hay versión que comparar	No protege la inserción de una fila nueva
Solo índice único en la base	Garantía absoluta, sin código	El perdedor recibe un 23505 genérico; no cubre reglas que no son igualdad (bloqueos de mantenimiento, cupo de clases)	Se usa, pero como red de seguridad, no como único mecanismo
Nivel de aislamiento SERIALIZABLE	La base detecta cualquier anomalía	Errores de serialización que hay que reintentar en todo el código; más difícil de razonar	Demasiado amplio para un problema localizado
`SELECT ... FOR UPDATE` **sobre la cancha + índice único parcial**	Serializa solo las reservas de esa cancha; el chequeo posterior es confiable; la base rechaza cualquier duplicado igual	Dos reservas de horarios distintos de la misma cancha esperan una a la otra unos milisegundos	Elegida

Decisión

Usamos **bloqueo pesimista** dentro de una transacción:

`TypeOrmBookingCanchaRepository.crearReservaSegura` toma `SELECT ... FOR UPDATE` sobre la fila de la cancha, verifica mantenimiento, bloqueos y reservas confirmadas en ese horario, e inserta. Como garantía en la base, el índice único parcial `uq_cancha_horario_confirmado (cancha_id, fecha, hora_inicio) WHERE estado = 'CONFIRMADA'` rechaza cualquier duplicado aunque se saltee el servicio. El perdedor recibe **409 Conflict** con el mensaje "Ese horario ya fue reservado para esta cancha". Las clases usan el mismo patrón con el lock sobre la ocurrencia (`TypeOrmBookingClaseRepository`).

Consecuencias

- **Positivas:** RN-02 verificada contra Postgres real: 10 reservas simultáneas confirman exactamente una (`backend/test/integracion/reserva-cancha.int-spec.ts`); el lock es por cancha, así que otras canchas no esperan; una reserva cancelada libera el horario porque el índice es parcial.
- **Negativas:** las reservas de la misma cancha se serializan; el índice cubre igualdad de hora de inicio, por lo que turnos superpuestos con distinto inicio dependen del lock y de que los turnos sean de una hora en punto (`turnos-cancha.ts`).
- **Deuda asumida:** si se permitieran turnos de duración libre, habría que reemplazar el índice por una restricción de exclusión sobre el rango horario (como la de membresías).

Requisitos relacionados

RN-02, RF-10, RF-07

ADR-005 — NestJS como framework backend, su contenedor de IoC y TypeORM como ORM

- **Estado:** Aceptado
- **Fecha:** 2026-09-06
- **Autor:** Zanandrea, Martín
- **Sprint:** 2

Contexto

El equipo tenía experiencia previa en JavaScript y TypeScript y un cronograma de 16 semanas. El monolito modular (ADR-001) necesitaba un framework con módulos e inyección de dependencias de primera clase, porque los patrones del caso (Strategy para precios, Observer para lista de espera, Repository para reservas) se apoyan en poder intercambiar implementaciones. También hacía falta un ORM con soporte de transacciones y locks para ADR-004. Esta decisión formaba parte del ADR 0003 original (stack); se separa en backend (este ADR), base de datos (ADR-002) y frontend (ADR-006).

Alternativas consideradas

Tabla 7

Alternativas consideradas (ADR-005)

Alternativa	A favor	En contra	Por qué se descartó
NestJS + TypeORM	Módulos, IoC con tokens y `useFactory`, decoradores, guards e interceptores; TypeORM con `setLock('pessimistic_write')` y transacciones	Curva inicial de decoradores; TypeORM tiene comportamientos implícitos	Elegida
Express plano + Knex	Mínimo y explícito	Sin IoC: la inyección de estrategias y repositorios se hace a mano; cada integrante estructuraría distinto	Se pierde la estructura modular que pide ADR-001
Spring Boot + JPA	IoC maduro, muy usado en la industria	Java: el equipo tendría dos lenguajes (front y back); más lento para iterar	Costo de aprendizaje en el plazo disponible
NestJS + Prisma	Tipado generado, migraciones integradas	Locks de fila y SQL específico de Postgres (índices parciales, exclusión) requieren SQL crudo	TypeORM cubría mejor `FOR UPDATE` en ese momento

Decisión

Usamos **NestJS 11** con TypeScript. El contenedor de IoC resuelve las variantes por token: `PRICING_STRATEGIES` (estrategias de precio), `BOOKING_CANCHA_REPOSITORY` y `BOOKING_CLASE_REPOSITORY` (implementación TypeORM o en memoria para pruebas), la fábrica de notificadores por canal, el `EnviadorCorreo` (ADR-021) y el `Almacenamiento` de archivos (ADR-023). Como ORM usamos **TypeORM** con `synchronize: false`: el esquema lo definen los scripts SQL versionados (ver ADR-009) y las entidades solo lo mapean.

Consecuencias

- **Positivas:** todo el equipo trabaja en un solo lenguaje; los patrones de diseño se implementan con inyección en lugar de `new`, lo que permite probarlos con dobles; guards e interceptores resuelven autorización (ADR-014) y auditoría (ADR-017) de forma transversal.
- **Negativas:** decoradores y metadatos agregan magia que cuesta depurar; TypeORM no detecta solo las diferencias entre entidades y esquema real (hay que verificarlo, como

se hizo al armar las migraciones).

- **Deuda asumida:** si se cambiara de ORM, los repositorios de reserva ya están detrás de interfaces, pero el resto de los servicios usa `Repository<T>` de TypeORM directamente.

Requisitos relacionados

Unidad II, RF-11, RF-08

Originalmente ADR 0003 (stack tecnológico). La parte de base de datos pasó al ADR-002 y la de frontend al ADR-006.

ADR-006 — React con Vite, componentes con Atomic Design y estado global con Context

- **Estado:** Aceptado
- **Fecha:** 2026-10-11
- **Autor:** Retamoso, Máximo
- **Sprint:** 5

Contexto

El sistema tiene dos caras: el portal responsive del socio y la consola de administración del personal, con pantallas distintas por rol. Hacía falta un framework con ecosistema maduro, un catálogo de componentes reutilizables (Unidad IV) y una forma de compartir el estado de sesión (usuario, rol, sede) entre todas las pantallas. React y Vite se habían elegido en el ADR 0003 original (hoy ADR-005); la organización de componentes se decidió al reorganizar `shared/components` (FT1-32).

Alternativas consideradas

Tabla 8

Alternativas consideradas (ADR-006)

Alternativa	A favor	En contra	Por qué se descartó
React 19 + Vite + Tailwind 4, Atomic Design, Context API	Ecosistema y experiencia del equipo; build rápido; átomos, moléculas y organismos reutilizables entre portal y consola; Context alcanza para el estado que es realmente global (sesión)	Context re-renderiza a todos los consumidores; la consistencia del catálogo depende de la disciplina del equipo	Elegida
Vue 3 + Pinia	Curva suave, store oficial	Menos experiencia previa del equipo	Costo de aprendizaje

Alternativa	A favor	En contra	Por qué se descartó
Redux Toolkit / Zustand para todo el estado	Una sola fuente de verdad explícita, devtools	Boilerplate para datos que son locales de cada pantalla (listados, formularios)	El único estado realmente global es la sesión
Librería de componentes (MUI, Chakra)	Componentes listos y accesibles	Estética genérica y difícil de adaptar al diseño propio; peso del bundle	Se prefirió un catálogo propio con Atomic Design

Decisión

Usamos **React 19 con Vite y TypeScript**, estilos con **Tailwind CSS 4** y un catálogo propio organizado con **Atomic Design** en `frontend/src/shared/components/` (`atoms/`, `molecules/`, `organisms/`, documentado en su README). El **estado global** es la sesión, en `AuthContext` (usuario, rol, sede, sincronizado entre pestañas); el resto del estado vive en cada pantalla. La comunicación con la API está aislada en una **capa de servicio** por módulo (`modules/*/*.api.ts`) sobre una única instancia de axios (`api/axios.ts`) que maneja cookie de sesión, timeouts y sesión vencida.

Consecuencias

- **Positivas:** una sola fuente de verdad para la sesión; los componentes no llaman a axios directamente; portal y consola comparten átomos y moléculas.
- **Negativas:** sin store global para datos de dominio, dos pantallas que muestran lo mismo lo piden dos veces; Context no escala a estado muy dinámico.
- **Deuda asumida:** si apareciera estado compartido entre muchas pantallas (por ejemplo, carrito o datos offline del terminal, ver ADR-003), convendría sumar un store (Zustand) o una caché de servidor (TanStack Query).

Requisitos relacionados

Unidad IV

ADR-007 — Sincronización del terminal de sede con el servidor central

- **Estado:** Aceptado
- **Fecha:** 2026-11-09
- **Autor:** Retamoso, Máximo

- **Sprint:** 7

Contexto

Si el terminal de sede valida ingresos sin conexión (ADR-003), necesita una copia local de los datos para decidir y una forma de devolver al servidor lo que pasó durante la caída. Hay que definir qué se replica, cada cuánto y qué hacer cuando lo registrado offline choca con el estado central (por ejemplo, el mismo socio ingresó en otra sede mientras esta estaba desconectada: RN-01).

Alternativas consideradas

Tabla 9

Alternativas consideradas (ADR-007)

Alternativa	A favor	En contra	Por qué se descartó
Réplica mínima por sede + cola de ingresos con resolución en el servidor	Datos chicos (id, nombre, apellido y fechas de membresía); el servidor decide los conflictos con toda la información	Ventana de datos desactualizados igual al intervalo de refresco	Alternativa elegida
Réplica completa de la base en cada sede	El terminal sabe todo	Volumen innecesario y datos personales replicados en 25 sedes	Desproporcionado y riesgoso
Base local con replicación bidireccional (CouchDB/PouchDB)	Sincronización resuelta por la herramienta	Otra base y otro modelo de datos solo para el terminal	Agrega una pieza de infraestructura para un caso acotado

Decisión

Qué se replica. El snapshot de la sede (`GET /acceso/snapshot/:sedeId`): la clave pública de firma del QR, el aforo, los identificadores de quienes están dentro y, por cada socio con membresía vigente o próxima de toda la red, `id`, nombre, apellido y las fechas `desde` y `hasta`. No incluye DNI, correo, teléfono ni foto. El snapshot vale 24 horas desde que lo genera el servidor.

Frecuencia. El terminal lo descarga al abrir la pantalla si no tiene uno guardado o si el guardado tiene más de 1 hora, y vuelve a revisar cada 5 minutos si hace falta renovarlo. La renovación solo ocurre con conexión.

Cola. Cada ingreso offline se guarda en IndexedDB con un `clientEventId` generado en el terminal (idempotencia) y la hora local de ocurrencia. Con conexión, y mientras haya pendientes, el terminal sincroniza al volver la red y cada 30 segundos. Envía `POST /acceso/sincronizar` en lotes de hasta 100 eventos de una misma sede.

Resolución en el servidor. Cada evento se procesa en su propia transacción y el resultado por evento puede ser:

- `CREADO`: el ingreso se registra con `validado_offline = true`.
- `OBSERVADO`: el ingreso se registra igual porque ya ocurrió, con una observación (`MEMBRESIA_NO_VIGENTE` o `AFORO_EXCEDIDO` al sincronizar).
- `CONFLICTO`: no se registra el ingreso. Ocurre con `SESION_ABIERTA` (el socio ya tenía un ingreso abierto, RN-01) o `QR_REUSADO` (el `jti` del QR ya estaba registrado). El evento queda guardado en `eventos_acceso_offline` con `acceso_id` nulo. No se notifica a la gerencia.
- `RECHAZADO`: el evento no es válido (`TOKEN_INVALIDO`, `QR_VENCIDO`, `FECHA_INVALIDA` por más de 72 horas de antigüedad, `USUARIO_INEXISTENTE`).

Un reintento con el mismo `clientEventId` no duplica: el servidor devuelve el resultado ya guardado (`YA_PROCESADO`). El terminal conserva en pantalla un resumen de la última sincronización con el detalle de los eventos que no salieron `CREADO`.

Consecuencias

- **Positivas:** RN-01 queda registrada en `eventos_acceso_offline` aunque no se pueda impedir durante la caída; el servidor sigue siendo la fuente de verdad; los reintentos son seguros.
- **Negativas:** un socio dado de baja durante la caída puede ingresar hasta que venza o se renueve el snapshot (ventana de hasta 24 horas); hay que guardar datos personales mínimos en el dispositivo; un `CONFLICTO` por `SESION_ABIERTA` no registra el ingreso, de modo que la persona que entró físicamente no figura dentro hasta que alguien lo resuelva a mano.
- **Deuda asumida:** no hay notificación a la gerencia ante un conflicto y no hay un procedimiento automático para resolverlo; hoy se revisa el resumen de sincronización

del terminal y la tabla `eventos_acceso_offline`.

Límites conocidos

- Para recargar la página sin conexión hace falta la PWA con su service worker instalado.
- Los registros de `jti` usados y de códigos cortos están en memoria del servidor: no sobreviven a un reinicio ni se comparten entre instancias. Durante la sincronización, el uso único del QR lo garantiza el índice único `uq_evento_offline_jti`.
- La ventana de revocación llega a 24 horas, por la vigencia del snapshot.
- Un timeout puede dejar al servidor con el ingreso registrado y al terminal sin la respuesta; el terminal valida localmente y encola el ingreso, y al sincronizar termina en un conflicto `SESION_ABIERTA`.
- La lista de ingresos offline que muestra la pantalla vive solo en el estado de la pantalla y se pierde al recargar; la cola de IndexedDB y el contador de pendientes se conservan.
- Sin conexión, el aforo muestra "No se pudo cargar el aforo".
- Los eventos que el servidor rechaza con error 400 por formato inválido no se reintentan solos: quedan guardados en el dispositivo y no cuentan como pendientes.

Requisitos relacionados

RNF-01, RN-01, RF-04

ADR-008 — Estrategia de pruebas, pirámide adoptada y umbral de cobertura

- **Estado:** Aceptado
- **Fecha:** 2026-11-09
- **Autor:** Retamoso, Máximo
- **Sprint:** 7

Contexto

Hasta el sprint 6 las pruebas eran unitarias con dobles en memoria (más de 700 casos en 39 suites), con buena cobertura de reglas de negocio, pero ninguna corría contra una base real: el bloqueo de fila y los índices únicos que garantizan RN-01 y RN-02 solo se simulaban

(lo advertía el ADR-022). Tampoco había un umbral de cobertura, así que el CI no bloqueaba un merge que la bajara.

Alternativas consideradas

Tabla 10

Alternativas consideradas (ADR-008)

Alternativa	A favor	En contra	Por qué se descartó
Pirámide: muchas unitarias, pocas de integración con Postgres real, E2E por pipeline de Docker	Las unitarias son rápidas y cubren reglas; las de integración prueban lo que los dobles no pueden (locks, índices); el job de Docker prueba que el sistema levanta	Las de integración necesitan una base y son más lentas	Elegida
Solo unitarias con dobles	Rápidas y sin infraestructura	No pueden probar concurrencia real	Deja sin verificar RN-01 y RN-02
Testcontainers dentro de Jest	Cada corrida crea su propio Postgres	Requiere Docker también en la máquina de cada integrante	El servicio Postgres de GitHub Actions resuelve lo mismo en CI
E2E de navegador (Playwright)	Prueba el recorrido del usuario	Lento y frágil; costo alto de mantener en el plazo	Se deja como mejora; el recorrido se prueba a mano (docs/PRUEBAS.md)

Decisión

Adoptamos una **pirámide de tres niveles**: (1) **unitarias** (`backend/src/**/*.spec.ts`, Jest + dobles en memoria) para reglas de negocio, estrategias de precio, permisos y casos límite; (2) **integración** (`backend/test/integracion/*.int-spec.ts`) contra Postgres real con los servicios y repositorios reales, para RN-01, RN-02 y cupo de clases bajo concurrencia, preparando la base con las mismas migraciones de producción; (3) **sistema**: el job `docker` del CI levanta `docker compose up --build` y verifica portal, API, Swagger y datos de prueba. **Umbral mínimo de cobertura** de las unitarias, declarado en `coverageThreshold` de `backend/package.json`: **70 % de sentencias y líneas, 60 % de ramas y 55 % de funciones**, excluyendo archivos sin lógica (módulos de Nest, DTOs, entidades, `main.ts`, scripts). El CI falla si se incumple.

Consecuencias

- **Positivas:** RN-01 y RN-02 quedan probadas con concurrencia real; la cobertura no puede bajar sin que el pipeline falle; el pipeline valida migraciones, contrato OpenAPI y arranque con Docker.
- **Negativas:** las pruebas de integración necesitan Postgres (en CI lo provee el servicio; en local hay que levantarlo); el umbral de funciones es bajo porque muchos controladores solo delegan.
- **Deuda asumida:** no hay pruebas automatizadas del cliente ni E2E de navegador; el umbral debería subir a medida que se cubran controladores.

Requisitos relacionados

Unidad V, RN-01, RN-02, RNF-03

Cómo se corren

```
cd server
npm test                  # unitarias
npm run test:cov          # unitarias con cobertura y umbral
INTEGRACION_DB=true DB_HOST=localhost DB_PORT=5433 DB_USERNAME=fitzone
DB_PASSWORD=fitzone-local DB_NAME=fitzone_test npm run test:integracion
```

ADR-009 — Despliegue en plataformas administradas, Docker para desarrollo y corrección, y procedimiento de reversión

- **Estado:** Aceptado
- **Fecha:** 2026-11-09
- **Autor:** Retamoso, Máximo
- **Sprint:** 7

Contexto

El sistema está desplegado desde el sprint 3 en planes gratuitos: el portal en Vercel, la API en Render y la base y los archivos en Supabase. La entrega exige además que alguien ajeno al equipo pueda levantar todo con `docker compose up --build`, con esquema y datos de prueba, y un procedimiento documentado de reversión. Hasta ahora no había Dockerfiles, el esquema se aplicaba a mano en el editor SQL de Supabase y los scripts de `backend/db/sql` no reproducían la base real en una base vacía.

Alternativas consideradas

Tabla 11

Alternativas consideradas (ADR-009)

Alternativa	A favor	En contra	Por qué se descartó
Plataformas administradas en producción + docker compose para desarrollo y corrección	Producción sin servidores que mantener; un solo comando reproduce el sistema completo sin cuentas externas	Dos formas de ejecutar el sistema que hay que mantener coherentes	Elegida
Todo en contenedores también en producción (VPS con Docker)	Mismo artefacto en todos lados	Costo y mantenimiento de un servidor propio	Fuera del presupuesto del proyecto
Kubernetes	Escalado y despliegues progresivos	Complejidad muy superior a la necesidad	Sobredimensionado
Sin Docker, solo instrucciones manuales	Nada que mantener	No cumple la condición de admisibilidad	Descartada

Decisión

Producción: Vercel (portal, con `/api` reenviado a la API), Render (API) y Supabase (Postgres y Storage), con despliegue automático al hacer merge en `main` después del CI. **Desarrollo y corrección:** `docker-compose.yml` en la raíz con tres contenedores (Postgres 17, API y portal servido por nginx con el mismo reenvío de `/api`); la API aplica las migraciones pendientes y carga los datos de prueba al arrancar, y guarda archivos en un volumen (ADR-023). **Migraciones:** `backend/db/migrar.js` aplica los scripts `db/sql/NNN-*.sql` en orden y registra cada uno en `schema_migraciones`; cada uno tiene su reversión en `db/sql/reversion/NNN-*.down.sql`. **Reversión:** (1) aplicación: volver al despliegue anterior desde el panel de Render ("Rollback") y de Vercel ("Promote" del deployment previo), o `git revert` del merge; (2) base: `npm run db:revertir` deshace la última migración (o `revertir N`), previa copia de seguridad.

Consecuencias

- **Positivas:** el sistema se reproduce en una máquina limpia con un comando; las migraciones son verificables (el CI las aplica, las revierte todas y las vuelve a aplicar); la reversión está documentada para código y para datos.

- **Negativas:** el compose no reproduce Supabase Storage ni el envío real de correo (se reemplazan por disco local y links en el log); el plan gratuito de Render suspende la API por inactividad.
- **Deuda asumida:** la base de Supabase se creó a mano: hay que marcar sus migraciones como aplicadas una vez (`node db/migrar.js marcar-aplicadas`) antes de usar el script contra producción.

Requisitos relacionados

RNF-04, Unidad V

ADR-010 — Tratamiento de los medios de pago: pasarela simulada, tokenización y sin datos de tarjeta

- **Estado:** Aceptado
- **Fecha:** 2026-09-06
- **Autor:** Zanandrea, Martín
- **Sprint:** 2

Contexto

RF-13 pide integrar una pasarela de pago simulada (Mercado Pago o MODO) y RNF-02 exige que los datos de tarjeta no se almacenen localmente: solo el token de la pasarela. Además el recepcionista cobra en efectivo en el mostrador, sin pasarela. El monto nunca debía depender de lo que envía el cliente (ver ADR-016).

Alternativas consideradas

Tabla 12

Alternativas consideradas (ADR-010)

Alternativa	A favor	En contra	Por qué se descartó
Interfaz `PasarelaPago` con implementación simulada; solo se guarda el token	La tarjeta nunca toca nuestro servidor ni la base; cambiar a una pasarela real es otra implementación	La pasarela simulada no prueba los casos reales de una pasarela (webhooks, pagos pendientes)	Elegida
Formulario propio de tarjeta que envía los datos a la API	Control total de la interfaz	Los datos de tarjeta pasarían por el servidor: alcance PCI-DSS completo	Viola RNF-02
Guardar la tarjeta cifrada para cobros recurrentes	Renovación automática real	Custodia de datos de tarjeta, aunque estén cifrados	Viola RNF-02; la renovación se simula

Decisión

Los pagos online pasan por la interfaz `PasarelaPago` (`backend/src/pagos/pasarela-pago.interface.ts`), implementada por `PasarelaMockService`, que devuelve `{ aprobado, token }` con un token opaco `MOCK-<método>-…`. La tabla `pagos` guarda solo método, monto, estado y `token_pasarela`: **no existe ningún campo de número de tarjeta, vencimiento ni código de seguridad**. Una restricción CHECK (exclusive arc) obliga a que un pago EFECTIVO tenga `registrado_por_id` y no token, y uno MERCADOPAGO/MODO tenga token y no `registrado_por_id`. El monto se calcula siempre en el servidor (ADR-016) y cada pago aprobado genera su comprobante PDF (RF-14).

Consecuencias

- **Positivas:** RNF-02 se cumple por construcción: no hay dónde guardar una tarjeta; integrar Mercado Pago real es agregar otra implementación de la interfaz.
- **Negativas:** la simulación aprueba siempre salvo que se fuerce el rechazo; no se modelan pagos pendientes ni notificaciones asíncronas de la pasarela.
- **Deuda asumida:** con una pasarela real habría que agregar el webhook de confirmación y el estado PENDIENTE que hoy no se usa.

Requisitos relacionados

RF-13, RF-14, RNF-02

ADR-011 — Aplicación web progresiva (PWA) en lugar de una app móvil nativa

- **Estado:** Aceptado
- **Fecha:** 2026-10-18
- **Autor:** Zanandrea, Martín
- **Sprint:** 5

Contexto

El enunciado plantea una app móvil para el socio (mostrar el QR, reservar) y la Unidad VI pide una app instalable con el QR visible. El portal del socio ya era una SPA responsive en React. Construir una app nativa implicaba un segundo cliente con otra base de código y otra capa de servicio, en la parte final del cuatrimestre, y publicar en tiendas

requiere cuentas de desarrollador. La PWA se implementó en el commit `8963659` (vite-plugin-pwa) y se completó con notificaciones push (ADR-021 y canal PUSH).

Alternativas consideradas

Tabla 13

Alternativas consideradas (ADR-011)

Alternativa	A favor	En contra	Por qué se descartó
PWA sobre el mismo portal (vite-plugin-pwa)	Una sola base de código; instalable en Android, iOS y escritorio; actualizaciones sin tienda; Web Push	En iOS el soporte de push y de almacenamiento es más limitado; sin acceso a APIs nativas avanzadas	Elegida
React Native / Expo	App nativa real, reutiliza conocimiento de React	Segundo cliente a mantener; la capa de servicio y los componentes se duplican; publicación en tiendas	Duplica trabajo en el último tramo
Flutter	Rendimiento y UI consistentes	Otro lenguaje (Dart) y otra base de código	Costo de aprendizaje
Envolver la web con Capacitor o Cordova	APK instalable reutilizando la web	Agrega una capa nativa que mantener sin beneficio funcional sobre la PWA	La PWA ya es instalable

Decisión

El portal se distribuye como **PWA**: manifest, iconos, service worker generado con `vite-plugin-pwa`, modo standalone, aviso de actualización, banner sin conexión y Web Push con VAPID para las notificaciones. El criterio determinante fue entregar la experiencia móvil sin duplicar la base de código ni la capa de servicio.

Consecuencias

- **Positivas:** un solo cliente para socio y personal; la misma URL sirve en el teléfono y en la recepción; el terminal de sede puede ser la propia PWA instalada (base para ADR-003).
- **Negativas:** la API no se cachea (estrategia `NetworkOnly`), así que hoy el QR no se muestra sin conexión; en iOS las notificaciones push requieren instalar la app en la pantalla de inicio.
- **Deuda asumida:** si la cátedra exigiera APK/IPA de tienda, se podría empaquetar la PWA con una TWA (Android) o Capacitor sin reescribir la app.

Requisitos relacionados

Unidad VI, RF-04, RNF-01

ADR-012 — GitHub como repositorio compartido (monorepo)

- **Estado:** Aceptado
- **Fecha:** 2026-09-04
- **Autor:** Zanandrea, Martín
- **Sprint:** 2

Contexto

El equipo necesita coordinar cambios de código realizados en paralelo por 3 integrantes sobre un mismo proyecto (frontend y backend), evitando sobrescrituras accidentales y manteniendo un historial auditable de quién hizo cada cambio.

Alternativas consideradas

Tabla 14

Alternativas consideradas (ADR-012)

Alternativa	A favor	En contra	Por qué se descartó
GitHub, monorepo	Historial único, PR, Actions y Projects integrados	Front y back comparten historial	Elegida
GitLab	CI integrado equivalente	El equipo ya usaba GitHub	Sin ventaja suficiente
Dos repositorios (front y back)	Historiales independientes	Coordinar cambios que cruzan API y cliente en dos PR	Más fricción para un equipo de 4

Decisión

Vamos a usar GitHub como repositorio remoto compartido, con Git como sistema de control de versiones, alojando en un único repositorio tanto el código de cliente como de servidor (monorepo), coordinado mediante ramas de trabajo y commits individuales por integrante.

Consecuencias

- **Positivas:** historial auditable por autor; trabajo en paralelo por ramas.
- **Negativas:** ambos proyectos comparten historial y pipeline.

- **Deuda asumida:** si el cliente y la API se versionaran por separado, habría que dividir el repositorio.

Requisitos relacionados

Todos (gestión)

Detalle de las consecuencias

Todo cambio queda historizado con autor, fecha y descripción, permitiendo revertir o auditar el desarrollo en cualquier momento. El equipo puede trabajar en paralelo sobre distintas partes del sistema sin bloquearse mutuamente. Mantener frontend y backend en el mismo repositorio (monorepo) simplifica la coordinación para un equipo de este tamaño, a costa de que ambos proyectos compartan siempre el mismo historial de versiones en lugar de evolucionar de forma completamente independiente.

Originalmente ADR 0001. Contenido sin cambios, adaptado a la plantilla del Anexo B.

ADR-013 — Modelado de datos: UUID, sede parcial, tipos exactos y exclusive arc

- **Estado:** Aceptado
- **Fecha:** 2026-09-06
- **Autor:** Zanandrea, Martín
- **Sprint:** 2

Contexto

El sistema debía escalar a 25 o más sucursales con interoperabilidad real entre ellas (RF-03, RNF-04) y reglas de negocio que dependen de la pertenencia de cada recurso a una sede concreta (RN-01). Con claves primarias enteras autoincrementales, dos sedes generando datos de forma independiente podrían producir colisiones de ID. El sistema además maneja valores monetarios (precio de canchas, monto de pagos, precio de membresías) donde un error de redondeo por el uso de punto flotante binario (FLOAT/REAL) podría generar diferencias de centavos acumulativas entre lo cobrado y lo registrado. Por último, un pago puede corresponder a exactamente una de tres cosas (membresía, reserva de clase o reserva de cancha), y una FK polimórfica genérica no permite que Postgres valide la existencia real de la fila referenciada.

Alternativas consideradas

Tabla 15

Alternativas consideradas (ADR-013)

Alternativa	A favor	En contra	Por qué se descartó
UUID + NUMERIC + ENUM + exclusive arc	Sin colisiones entre sedes, montos exactos, la base valida vocabularios y referencias	Claves más grandes; un ENUM nuevo requiere migración	Elegida
Claves enteras autoincrementales	Más chicas y legibles	Colisiones al combinar datos de sedes; revelan volumen de negocio	No escala a 25+ sedes
FLOAT para montos	Simple	Errores de redondeo acumulativos	Inaceptable en pagos
FK polimórfica genérica en pagos	Una sola columna	Postgres no puede validar la referencia	Pierde integridad referencial

Decisión

Vamos a adoptar cinco criterios de modelado en conjunto sobre las 17 entidades del sistema: (1) UUID (gen_random_uuid()) como clave primaria en todas las tablas, en lugar de enteros autoincrementales; (2) sede_id como columna directa únicamente en las entidades donde el recurso pertenece de forma primaria e inmutable a una sola sede (sedes, usuarios —nullable para roles sin sede fija—, clases, canchas, membresias vía sede_alta_id, franjas_horarias); el resto de las entidades (instructores, reservas_clase, reservas_cancha, bloqueos_cancha, pagos, comprobantes, precios_plan, auditoria, clase_horario_semanal, clase_ocurrencia) obtiene su sede por relación indirecta (join) hacia la entidad que sí la posee, evitando duplicar un dato derivable y el riesgo de que quede desincronizado si el recurso padre cambia de sede; (3) tipos ENUM nativos de Postgres para todo campo de vocabulario cerrado (tipo_actor, estado_membresia, estado_res_cancha, metodo_pago, etc.); (4) el tipo NUMERIC(10,2) —de precisión decimal exacta, no de punto flotante binario— para todo campo monetario (monto, precio, precio_final, costo_hora_base); (5) el patrón exclusive arc (columnas de clave foránea nullables más un CHECK) para representar que un pago referencia exactamente una de tres entidades posibles.

Consecuencias

- **Positivas:** scoping por sede con un WHERE; montos exactos; la base rechaza valores inválidos.
- **Negativas:** más almacenamiento por clave; tres columnas nullables en pagos.
- **Deuda asumida:** agregar un valor a un ENUM siempre requiere una migración (ver 005).

Requisitos relacionados

RF-03, RNF-04, RN-01, RF-13

Detalle de las consecuencias

El UUID elimina el riesgo de colisión de ID entre sedes y no revela volumen de negocio a través de la API. El sede_id parcial permite implementar el scoping por sede (ADR-014) con una simple cláusula WHERE. Los ENUM detectan valores inválidos a nivel de base de datos antes de que lleguen a la aplicación. NUMERIC(10,2) garantiza que un monto se almacene y se sume siempre con exactitud decimal, sin el riesgo de discrepancias de centavos propio de FLOAT/REAL, crítico en una tabla auditada financieramente como pagos. El exclusive arc permite que Postgres valide la existencia real de la fila referenciada en pagos. Como contrapartida: mayor tamaño de almacenamiento por clave UUID frente a un entero; agregar un valor a un ENUM requiere una migración de esquema; y la tabla de pagos usa tres columnas nullables en lugar de dos columnas fijas. El sede_id directo permite el scoping por sede (ADR-014) con una cláusula WHERE simple en las entidades que lo tienen; en el resto, el scoping se resuelve con un join adicional hacia la entidad relacionada, evitando la redundancia de repetir el dato.

Originalmente ADR 0004. Contenido sin cambios, adaptado a la plantilla del Anexo B.

ADR-014 — Autenticación con JWT en cookie httpOnly y autorización por rol, dueño y sede

- **Estado:** Aceptado
- **Fecha:** 2026-09-06
- **Autor:** Ava, Teo

- **Sprint:** 2

Contexto

El sistema autentica a cuatro tipos de actor (Socio, Externo, Recepcionista, Gerente) que comparten un único mecanismo de login pero requieren visibilidad y permisos distintos. Un cliente no tenía forma de encontrar el acceso administrativo mediante una única puerta de login. Un Recepcionista podía operar sobre sedes ajenas a la suya, porque el control de acceso solo filtraba por rol, no por pertenencia a sede. Y almacenar el token en localStorage habría expuesto la sesión a robo vía XSS.

Alternativas consideradas

Tabla 16

Alternativas consideradas (ADR-014)

Alternativa	A favor	En contra	Por qué se descartó
JWT en cookie httpOnly + guards por rol + scope por sede	El token no es accesible desde JavaScript; cada endpoint valida rol y sede	Requiere CORS con credenciales y trust proxy	Elegida
JWT Bearer en localStorage	Simple de implementar en el cliente	Expuesto a robo por XSS	Riesgo de seguridad
Sesiones en servidor (express-session)	Revocación inmediata	Estado en el servidor; más difícil con varias instancias	JWT alcanza para el caso
Auth0 / Supabase Auth	Gestión de identidad completa	Reescribir el login y los roles ya construidos	Costo de migración

Decisión

Vamos a: (1) transportar el JWT en una cookie httpOnly —nunca en localStorage—, con el atributo secure en producción y sameSite lax; (2) separar dos rutas de login (/login para clientes, /admin/login para personal), reutilizando el mismo componente parametrizado por una prop de audiencia que valida que el rol corresponda a esa puerta; (3) aplicar un mecanismo de scoping por sede (assertSedeScope) en todos los módulos donde un Recepcionista puede operar —acceso, clases, canchas, pagos, y los reportes agregados del panel administrativo—, de modo que solo pueda actuar sobre la sede indicada en su propio registro de usuario, propagada a través del JWT.

Consecuencias

- **Positivas:** sesión protegida de XSS; un recepcionista no puede operar otra sede ni por API directa.
- **Negativas:** configuración extra de CORS y proxy; la puerta de login por audiencia es solo experiencia de usuario.
- **Deuda asumida:** no hay revocación de tokens antes de su vencimiento.

Requisitos relacionados

RF-01, RF-03, RN-01

Detalle de las consecuencias

El JWT nunca es accesible desde JavaScript del lado del cliente, mitigando el robo de sesión vía XSS. Ningún visitante ve un formulario de login que no le corresponde. Un Recepcionista no puede operar sobre otra sede aunque lo intente vía API directa, en ningún módulo del sistema. Como contrapartida, requiere configuración adicional de CORS y de trust proxy detrás del proveedor de hosting, y la validación de audiencia del login es una capa de experiencia de usuario: el control de seguridad real sigue siendo el guard de rol del backend en cada endpoint.

Nota de refuerzo: se detectó y corrigió que dos implementaciones del scoping por sede (membresías, opciones de cobro en efectivo) fallaban "abierto" ante un Recepcionista sin sedeId asignado (devolvían datos de todas las sedes en vez de ninguna). Se ajustó el criterio a fail-closed explícito en ambos casos, y se agregó una pantalla dedicada en el frontend que bloquea a un Recepcionista sin sede antes de que use cualquier pantalla del panel, mostrando ese estado como transitorio y esperable, no como un error.

Originalmente ADR 0005 (FT1-19). Contenido sin cambios, adaptado a la plantilla del Anexo B.

ADR-015 — Flujo obligatorio de membresía y pago, con recuperación ante pago rechazado

- **Estado:** Aceptado
- **Fecha:** 2026-09-06
- **Autor:** Zanandrea, Martín

- **Sprint:** 2

Contexto

RF-02 exige que un Socio tenga un plan de membresía vigente para operar como tal, pero el registro público solo creaba el usuario, sin obligarlo a elegir un plan ni completar el pago. Al forzar ese flujo y bloquear la creación de una segunda membresía activa por usuario, apareció un caso borde: si el pago de una membresía recién creada es rechazado por la pasarela, la membresía queda activa sin ningún pago aprobado, y el usuario no puede reintentar sin chocar con el bloqueo de duplicados.

Alternativas consideradas

Tabla 17

Alternativas consideradas (ADR-015)

Alternativa	A favor	En contra	Por qué se descartó
Paso obligatorio de plan, sede y pago tras el registro	Ningún socio queda en estado incompleto	La redirección la decide el cliente	Elegida
Membresía solo en mostrador	Control total del personal	Sin autoservicio online	Contradice el flujo del Demo Day
Borrar la membresía si el pago falla	Sin filas huérfanas	Se pierde la trazabilidad del intento	Se prefirió SUSPENDIDO

Decisión

Vamos a intercalar un paso obligatorio de elegir plan, sede y pagar entre el registro y el ingreso al panel del Socio, relajando la creación de membresías de un permiso exclusivo de personal a una validación de propiedad (el propio usuario puede crear su membresía). Si el pago resulta rechazado, el frontend invoca automáticamente un endpoint de cancelación que cambia el estado de esa membresía a Suspendido, liberando el bloqueo de duplicados para permitir un nuevo intento.

Consecuencias

- **Positivas:** registro y pago en un solo recorrido; trazabilidad del intento fallido.
- **Negativas:** un usuario podría navegar al panel sin pagar (sin beneficios).
- **Deuda asumida:** con la vigencia por fechas (ADR-022) la membresía se crea recién con el pago aprobado.

Requisitos relacionados

RF-01, RF-02, RN-03

Detalle de las consecuencias

Ningún Socio queda en un estado de negocio incompleto tras su propio registro. El endpoint de creación de membresías gana un caso de uso de autoservicio sin perder el de alta en mostrador por parte del personal. El caso borde de pago rechazado queda resuelto sin perder trazabilidad, ya que la membresía fallida se conserva como Suspendido en lugar de borrarse. Como contrapartida, la redirección al flujo de pago es una decisión tomada en el cliente, no impuesta por el backend: un usuario podría navegar manualmente al panel sin completar el pago, aunque no accedería a ningún beneficio de socio activo mientras tanto. Además del control al momento de registrarse, la membresía activa también se valida como condición para reservar clases grupales (RN-03): reserva-clase.service.ts rechaza la reserva con un mensaje explícito si el usuario no tiene una membresía en estado ACTIVO al momento de reservar, aplicando el mismo criterio a reservas hechas por el propio socio o anotadas por personal de recepción en su nombre.

Originalmente ADR 0006. Contenido sin cambios, adaptado a la plantilla del Anexo B. Parcialmente superado por ADR-022.

ADR-016 — Cálculo de montos de pago exclusivamente en el servidor

- **Estado:** Aceptado
- **Fecha:** 2026-09-06
- **Autor:** Ava, Teo
- **Sprint:** 2

Contexto

El flujo de pago recibía el monto directamente del cliente para membresías, reservas de clase y reservas de cancha, sin validarlo contra ninguna fuente de verdad del backend, lo que permitía manipular el monto efectivamente pagado. Tampoco existía una forma de que el Gerente modificara el precio de los planes de membresía sin desplegar código nuevo.

Alternativas consideradas

Tabla 18

Alternativas consideradas (ADR-016)

Alternativa	A favor	En contra	Por qué se descartó
Tabla de precios + recálculo en el servidor	El cliente no puede manipular el monto; el gerente edita precios sin desplegar	Una tabla más	Elegida
Confiar en el monto del cliente	Simple	Permite pagar lo que el cliente quiera	Vulnerabilidad
Precios en constantes del código	Sin tabla	Cambiar un precio exige desplegar	Poco operable

Decisión

Vamos a usar una tabla de precios de membresía, editable exclusivamente por el rol Gerente, como fuente de verdad del precio de cada plan, y recalcular siempre el monto real del lado del servidor: desde esa tabla para membresías, y desde el precio final ya calculado y persistido al momento de reservar para las canchas, ignorando cualquier monto que envíe el cliente en ambos casos.

Consecuencias

- **Positivas:** cierra la manipulación de montos; precios editables por el gerente.
- **Negativas:** los precios de canchas se toman del valor persistido al reservar.
- **Deuda asumida:** ninguna relevante.

Requisitos relacionados

RF-02, RF-11, RF-13

Detalle de las consecuencias

Cierra el vector de manipulación de precios en membresías y canchas: el monto pagado ya no depende de lo que declare el cliente. El Gerente puede modificar el precio de los planes desde la aplicación, sin necesidad de un nuevo despliegue. Las clases no son cobrables de forma independiente porque el acceso viene incluido en la membresía (RN-03).

Originalmente ADR 0007 (FT1-21). Contenido sin cambios, adaptado a la plantilla del Anexo B.

ADR-017 — Sistema de auditoría en dos capas: interceptor de aplicación y trigger de base

- **Estado:** Aceptado
- **Fecha:** 2026-09-06
- **Autor:** Zanandrea, Martín
- **Sprint:** 2

Contexto

El sistema no contaba con ningún mecanismo para registrar quién realizó una acción sensible ni en qué momento. Se evaluó un interceptor a nivel de aplicación, que identifica al actor a través del JWT pero solo cubre los cambios que pasan por la API, frente a triggers de base de datos, que cubren cualquier cambio sobre una tabla mismo si ocurre por fuera de la aplicación, pero no tienen forma de identificar al usuario de la aplicación que lo originó.

Alternativas consideradas

Tabla 19

Alternativas consideradas (ADR-017)

Alternativa	A favor	En contra	Por qué se descartó
Interceptor + trigger en pagos	Contexto completo del actor y cobertura de cambios fuera de la API en la tabla más sensible	Dos lugares donde mirar	Elegida
Solo interceptor	Contexto rico	No ve cambios hechos directo en la base	Deja abierta la evasión de la API
Solo triggers	Cubre todo cambio	No identifica al usuario de la aplicación	Pierde el contexto

Decisión

Vamos a usar ambos mecanismos con responsabilidades distintas: un decorador de auditoría junto con un interceptor global como capa principal, que registra el contexto completo (actor, acción, entidad afectada) de toda acción sensible realizada a través de la API; y un trigger de Postgres en la tabla de pagos —la de mayor sensibilidad financiera del

sistema— como capa adicional, que deja constancia de cualquier cambio sobre esa tabla ocurrido fuera de la aplicación.

Consecuencias

- **Positivas:** trazabilidad de acciones sensibles; defensa ante cambios directos en pagos.
- **Negativas:** el trigger no tiene contexto de aplicación.
- **Deuda asumida:** los datos sensibles en `detalle` se redactan (migración 008).

Requisitos relacionados

RF-13, RNF-02

Detalle de las consecuencias

La capa de interceptor cubre el uso normal del sistema con contexto rico y legible. La capa de trigger cierra el vector de "alguien con acceso directo a la base de datos evade la aplicación", específicamente en la tabla más sensible. Como contrapartida, mantener dos mecanismos implica dos lugares distintos donde revisar qué ocurrió sobre la tabla de pagos, y el trigger no puede enriquecerse con contexto de aplicación, como identificar si un cambio provino de un recepcionista cobrando o de la propia pasarela de pago.

Originalmente ADR 0008. Contenido sin cambios, adaptado a la plantilla del Anexo B.

ADR-018 — Clases grupales como plantillas recurrentes, no eventos únicos

- **Estado:** Aceptado
- **Fecha:** 2026-09-09
- **Autor:** Zanandrea, Martín
- **Sprint:** 3

Contexto

El modelo original de `clases` representaba cada clase como un evento único con fecha y hora fija (horario_inicio/horario_fin como timestamptz puntual), obligando al Gerente a crear una fila nueva por cada ocurrencia. En la operación real de un gimnasio, las clases grupales (Yoga, Spinning, etc.) son recurrentes semanales y persisten indefinidamente hasta que se decide discontinuarlas, sin una fecha de fin planificada de antemano — el enunciado

original (RF-06 a RF-08) no específica este comportamiento ni si una sede permite clases simultáneas, dejando ambas decisiones abiertas.

Alternativas consideradas

Tabla 20

Alternativas consideradas (ADR-018)

Alternativa	A favor	En contra	Por qué se descartó
Plantilla semanal + ocurrencias generadas	Refleja la operación real; reservas atadas a una fecha concreta	Necesita un cron que mantenga la ventana	Elegida
Una fila por clase con fecha fija	Modelo simple	El gerente carga cada clase a mano	No escala
Reglas de recurrencia tipo iCal (RRULE)	Muy expresivo	Complejidad para calcular cupo y reservas	Más de lo necesario

Decisión

Vamos a modelar las clases como plantillas recurrentes semanales (tipo, instructor, sede, horas semanales totales), a partir de las cuales el sistema genera automáticamente las ocurrencias reservables concretas (fechas puntuales) manteniendo siempre una ventana móvil de semanas futuras generadas, sin intervención manual del Gerente. Al definir una plantilla, el Gerente indica solo las horas semanales totales; el sistema sugiere combinaciones de días (1 a 5), calculando las horas por día con redondeo a la media hora más cercana, y propone automáticamente en qué días y horarios concretos ubicarlas según los huecos libres de la sede — todo editable por el Gerente antes de confirmar. Asumimos que cada sede tiene una única "sala" compartida para clases grupales: nunca dos clases ocurren en simultáneo en la misma sede, y el pool de horas semanales disponibles (definido por el horario de apertura de la sede) se comparte entre todas sus clases.

Consecuencias

- **Positivas:** planificación por horas semanales; reglas de 48 h y 2 h sin ambigüedad.
- **Negativas:** supuesto de sala única por sede.
- **Deuda asumida:** si una sede necesitara clases simultáneas habría que introducir el concepto de sala.

Requisitos relacionados

RF-06, RF-07, RF-08

Detalle de las consecuencias

El Gerente planifica en términos de "cuántas horas semanales" en lugar de crear eventos uno por uno, reflejando cómo se gestiona un gimnasio real. Las reservas (RF-07) siguen atadas a una ocurrencia concreta (fecha y hora puntual), preservando la semántica de "reservá hasta 48hs antes" y "cancelá hasta 2hs antes" sin ambigüedad. Como contrapartida, el supuesto de sala única es una decisión nuestra no respaldada por el enunciado original: si en el futuro una sede necesitara ofrecer clases simultáneas en espacios distintos, haría falta introducir un concepto de "sala" o "espacio" que hoy no existe, lo cual implicaría revisar este ADR. La generación automática de ocurrencias hacia adelante requiere un mecanismo recurrente (cron) que mantenga la ventana de semanas futuras, sumando una pieza de infraestructura que el modelo anterior no necesitaba.

Nota de refuerzo: se detectó y corrigió una condición de carrera real en la reserva de clases — el chequeo de cupo y la creación de la reserva ocurrían en dos pasos sin transacción, permitiendo en teoría que dos solicitudes simultáneas al último cupo disponible resultaran ambas en RESERVADA, superando la capacidad declarada. Se cerró con el mismo patrón Repository ya aplicado a canchas (transacción con lock pesimista sobre la ocurrencia, más una implementación in-memory disponible para testing).

Originalmente ADR 0009. Contenido sin cambios, adaptado a la plantilla del Anexo B.

ADR-019 — Verificación de email con Gmail SMTP, en vez de Resend o Auth0

- **Estado:** Reemplazado por ADR-021
- **Fecha:** 2026-09-30
- **Autor:** Zanandrea, Martín
- **Sprint:** 4

Contexto

La verificación de email obligatoria al registrarse (ver contexto completo en la versión anterior de este ADR: se descartó migrar a Auth0 por el costo de reemplazar el sistema de

autenticación ya construido y documentado en el ADR-014). La primera implementación usó Resend como proveedor de email transaccional, pero su plan gratuito solo permite enviar a la casilla propia del dueño de la cuenta mientras no se verifique un dominio propio — y el proyecto no tiene un dominio propio (vercel.app y onrender.com son subdominios de terceros, sin DNS propio editable).

Alternativas consideradas

Tabla 21

Alternativas consideradas (ADR-019)

Alternativa	A favor	En contra	Por qué se descartó
Gmail SMTP con contraseña de aplicación	Envía a cualquier destinatario sin dominio propio	Límite de envíos; no apto para producción a escala	Elegida en su momento
Resend	Proveedor transaccional dedicado	El plan gratuito solo envía al dueño sin dominio verificado	Sin dominio propio
Auth0	Verificación de email incluida	Reemplazar la autenticación ya construida	Costo de migración

Decisión

Se reemplaza Resend por Gmail SMTP, usando una cuenta de Gmail creada específicamente para el proyecto (fitzonesports@gmail.com) con una contraseña de aplicación (no la contraseña real de la cuenta). Gmail SMTP permite enviar a cualquier destinatario real sin restricción de dominio verificado, dentro de un límite de aproximadamente 500 envíos diarios — suficiente para el volumen de un proyecto académico. El mecanismo de token de verificación (JWT con tipo 'email-verificacion', expiración 24hs) y el bloqueo de login hasta verificar se mantienen sin cambios; solo cambia el transporte de envío.

Consecuencias

- **Positivas:** flujo de verificación probado con destinatarios reales.
- **Negativas:** Render bloquea el SMTP saliente: no funciona en producción.
- **Deuda asumida:** reemplazado por el proveedor abstracto del ADR-021.

Requisitos relacionados

RF-01

Detalle de las consecuencias

Se pierde la infraestructura dedicada de email transaccional de Resend (métricas de entrega, reputación de IP separada), a cambio de poder probar el flujo completo con destinatarios reales sin restricción. El límite de envíos de Gmail es generoso para el contexto académico, pero no sería apto para un uso productivo real a escala — si el proyecto creciera más allá de la cátedra, volver a un proveedor dedicado (con dominio propio verificado) sería la decisión correcta.

Originalmente ADR 0010. Se conserva como registro histórico.

ADR-020 — Jerarquía de roles de gestión: Gerente General y Gerente de Sede

- **Estado:** Aceptado
- **Fecha:** 2026-10-07
- **Autor:** Zanandrea, Martín
- **Sprint:** 5

Contexto

El enunciado agregó un rol de gerente general que administra gerentes de sede: una persona con visibilidad total sobre todas las sedes, y gerentes operativos acotados a su propia sede. El rol GERENTE anterior era plano (acceso total sin sede) y no distinguía niveles.

Alternativas consideradas

Tabla 22

Alternativas consideradas (ADR-020)

Alternativa	A favor	En contra	Por qué se descartó
Tres niveles con scope por exclusión (fail-closed)	Cada actor ve solo lo suyo; sin sede se rechaza	Más casos de permisos que probar	Elegida
Rol GERENTE plano	Simple	No distingue la gerencia central de la de sede	No cubre el nuevo requisito
Permisos granulares configurables (RBAC)	Muy flexible	Interfaz y modelo de permisos completos	Sobredimensiona do

Decisión

Se divide el rol en tres niveles con scope por exclusión (fail-closed): si el actor no es GERENTE_GENERAL, se acota a su sedeId; sin sedeId se rechaza con 403. El scope se aplica también en el WHERE de listados y reportes, no solo en endpoints por id.

- GERENTE_GENERAL: ve y administra todas las sedes. Es el único que edita precios de planes de membresía, crea o elimina sedes, administra el catálogo de tipos de cancha, crea y gestiona gerentes de sede, activa o desactiva instructores y ejecuta marcar-vencidas.
- GERENTE_SEDE: administra todo lo de su sede salvo los precios de planes. Crea y edita canchas (incluido costoHoraBase), las pone en mantenimiento y las elimina físicamente solo si no tienen reservas en ningún estado; gestiona clases, instructores (alta y edición), membresías, accesos, pagos, franjas horarias, alta y baja de recepcionistas de su sede y reportes limitados a su sede.
- RECEPCIONISTA: opera en una sede. Ve aforo y quién está dentro, valida QR y código de 6 dígitos y registra egreso, cobra en efectivo (plan de membresía o reserva de cancha, no clases), anota socios a clases y canchas, ve el estado de membresía en solo lectura y un panel liviano sin dinero.
- Excepción documentada: las canchas admiten borrado físico condicionado (0 reservas), a diferencia del resto de las entidades que se dan de baja lógicamente, para corregir errores de carga.

Consecuencias

- **Positivas:** visibilidad acotada por sede también en listados y reportes.
- **Negativas:** el valor GERENTE queda en el enum sin uso.
- **Deuda asumida:** nuevos roles requieren migración del enum.

Requisitos relacionados

RF-03, RF-05, RF-09

Detalle de las consecuencias

El control de acceso por dueño o staff se acotó: el acceso a perfiles y membresías de otros usuarios depende del rol y de la sede, y el socio que reserva para sí mismo conserva el

uso multi-sede. Un pago de membresía APROBADO (pasarela o efectivo del staff) convierte al titular EXTERNO en SOCIO, en la misma transacción que el pago (única función: `PagosService.promoverExternoASocioSiCorresponde`). El rol del titular es el único que cambia; ningún otro rol se toca. Un pago rechazado no lo cambia. El valor GERENTE del enum se conserva por compatibilidad en Postgres y los usuarios existentes se migraron a GERENTE_GENERAL (migración 005).

Originalmente ADR 0011. Contenido sin cambios, adaptado a la plantilla del Anexo B.

ADR-021 — Envío de correo mediante proveedor abstracto (Brevo en producción, Gmail en desarrollo)

- **Estado:** Aceptado
- **Fecha:** 2026-10-12
- **Autor:** Zanandrea, Martín
- **Sprint:** 5

Contexto

El ADR-019 adoptó Gmail SMTP como transporte de correo para la verificación de email. Esa decisión funcionó en desarrollo local, pero en producción el backend corre en el plan gratuito de Render, que bloquea el SMTP saliente (puertos 25, 465 y 587). El resultado fue un error ETIMEDOUT en todos los envíos desde producción: tanto el mail de verificación de cuenta como las notificaciones por correo. Gmail SMTP seguía funcionando solo en local.

Los dos consumidores del correo (`EmailVerificacionService` y `EmailNotificador`) creaban o usaban directamente el transporte SMTP, de modo que cambiar de proveedor implicaba modificarlos.

Alternativas consideradas

Tabla 23

Alternativas consideradas (ADR-021)

Alternativa	A favor	En contra	Por qué se descartó
Interfaz `EnviadorCorreo` con Brevo (HTTPS) y Gmail (SMTP)	Funciona en Render; los consumidores no conocen al proveedor	Depende de otro proveedor externo	Elegida
Seguir con Gmail SMTP	Sin cambios	Render bloquea SMTP	No funciona en producción
Gmail API con OAuth	HTTPS	Flujo OAuth en lugar de contraseña de aplicación	Más complejo
Dominio propio	Mejor entrega (DKIM/DMARC)	El proyecto no tiene uno	Pendiente

Decisión

Se introduce la abstracción `EnviadorCorreo` (`backend/src/common/mail/`), con un único método `enviar({ para, asunto, html, texto? })`, y dos implementaciones:

- `BrevoEnviador`: envía por la API HTTPS de Brevo (`POST https://api.brevo.com/v3/smtp/email`) usando `fetch` nativo de Node, con un timeout de 10 segundos mediante `AbortController`. No agrega dependencias.
- `GmailSmtpEnviador`: el envío por SMTP con nodemailer ya existente, movido sin cambios funcionales y con las mismas variables `GMAIL_USER` y `GMAIL_APP_PASSWORD`. Se usa solo en desarrollo local.

La implementación activa se elige con la variable de entorno `MAIL_PROVIDER` (`brevo` o `gmail`; si no está definida, `gmail`), mediante un provider de NestJS con `useFactory`. La configuración se valida al arrancar: con `brevo` son obligatorias `BREVO_API_KEY` y `MAIL_FROM_EMAIL` (`MAIL_FROM_NAME` es opcional), y un valor desconocido de `MAIL_PROVIDER` impide iniciar el servidor. Los mensajes de error nombran las variables pero nunca sus valores.

Los consumidores dependen de la interfaz y no de un proveedor concreto. Un fallo de envío no interrumpe el flujo principal (registro de usuario, notificación en el panel): se registra y, en las notificaciones, se cuenta como envío fallido. Se mantiene la pausa de 300 ms

entre envíos de un mismo lote. Los logs registran solo el id de usuario, el status y el código del error y, en un timeout, la duración; nunca la API key, el contenido del mail ni la dirección completa del destinatario.

Patrón de diseño

La abstracción aplica el principio de inversión de dependencias: los módulos de alto nivel (verificación de cuenta, notificaciones) dependen de la interfaz `EnviadorCorreo`, y las implementaciones de bajo nivel dependen de ella, no al revés. Cambiar de proveedor se reduce a agregar una implementación y registrarla en la fábrica, sin tocar los consumidores. Es la misma lógica de desacople que ya usa el módulo de notificaciones con su fábrica de notificadores por canal.

Consecuencias

- **Positivas:** correo funcionando en producción; cambiar de proveedor no toca a los consumidores.
- **Negativas:** sin dominio propio los mails pueden ir a spam.
- **Deuda asumida:** conseguir un dominio propio para autenticar los envíos.

Requisitos relacionados

RF-01, RF-08

Detalle de las consecuencias

- Los envíos de producción dependen de un proveedor externo y de los límites de su plan gratuito.
- El remitente es una dirección de Gmail verificada como remitente individual (sender) en Brevo. Al no ser un dominio propio, no se puede autenticar con DKIM ni DMARC propios, por lo que los mensajes pueden ser clasificados como spam. Para mejorar la entrega habría que usar un dominio propio.
- La API key de Brevo vive únicamente en las variables de entorno de Render (y en el `.env` local, que no se versiona). Se documentan los nombres de las variables en `backend/.env.example`, sin valores.
- Gmail SMTP queda restringido a desarrollo local: en producción debe configurarse `MAIL_PROVIDER=brevo`.

- El envío por Brevo se verificó con pruebas unitarias con `fetch` simulado. No se probó el envío masivo con volumen real.
- Se mantiene lo ya decidido sobre el mecanismo de verificación (token JWT de tipo 'email-verificacion' con expiración de 24 horas y bloqueo de login hasta verificar): solo cambia el transporte.

Detalle de las alternativas

- **Seguir con Gmail SMTP:** descartada, porque Render bloquea el SMTP saliente y no es posible enviar desde producción.
- **Otro proveedor con API HTTPS:** es viable en general, porque la API HTTPS no está afectada por el bloqueo de puertos. Con la abstracción adoptada, incorporarlo no exigiría cambios en los consumidores.
- **Gmail API con OAuth:** también usa HTTPS, pero requiere un flujo de autorización OAuth en lugar de una contraseña de aplicación. No se adoptó.
- **Dominio propio:** el proyecto no tiene uno (ver ADR-019). Es la mejora pendiente para la entrega, descrita en las consecuencias.
  Originalmente ADR 0012. Contenido sin cambios, adaptado a la plantilla del Anexo B. Reemplaza al ADR-019.

ADR-022 — Vigencia de membresías por fechas y no superposición

- **Estado:** Aceptado
- **Fecha:** 2026-10-18
- **Autor:** Zanandrea, Martín
- **Sprint:** 5

Contexto

Comprar o renovar con otra membresía vigente creaba una fila ACTIVO más, sin cerrar la anterior ni impedir períodos superpuestos. Casi todos los lugares que leían la membresía (QR, reservas de clases, precio de canchas, reportes, listados) elegían por `estado = ACTIVO`, sin mirar fechas, y `renovar()` tomaba "la primera" ACTIVO sin orden. Además las fechas se calculaban con `toISOString()` (UTC; después de las 21:00 argentinas el inicio caía "mañana"), `setMonth` desbordaba (31/01 + 1 mes = 03/03) y la vigencia dependía del

cron nocturno, que puede no ejecutarse en el hosting gratuito. No había restricción en la base, y dos compras simultáneas podían duplicar períodos.

Alternativas consideradas

Tabla 24

Alternativas consideradas (ADR-022)

Alternativa	A favor	En contra	Por qué se descartó
A: cada compra es una fila; la vigencia se decide por fechas	Renovación anticipada; no depende del cron; la base impide superposiciones	Todos los consumidores tienen que usar el mismo módulo de vigencia	Elegida
B: estado nuevo (programada) activado al llegar la fecha	Estado explícito	Depende de un proceso que active la fila	Más piezas, mismo resultado
C: rechazar la compra con una vigente	Simple	Elimina la renovación anticipada; sigue la carrera sin restricción	Peor experiencia

Decisión

Alternativa A del análisis: cada compra crea su propia fila (con su propio pago) y la vigencia se decide siempre por fechas.

1. `fecha_fin` es **exclusiva**: el primer día en que la membresía ya no vale. Vigente hoy = `estado = ACTIVO` y `fecha_inicio <= hoy < fecha_fin`, con "hoy" en hora de Argentina (`ahoraArgentina()`). Próxima = ACTIVO con `fecha_inicio > hoy`.
2. Una compra con una membresía vigente o en cola no pisa nada: la nueva empieza en el **mayor** `fecha_fin` entre las ACTIVO que no terminaron, o hoy si no hay ninguna. Se permite un plan distinto.
3. Máximo una vigente y una próxima: con una próxima se rechaza con 409 ("Ya tenés una membresía programada. Podrás comprar otra cuando comience"), antes de cobrar en la pasarela y de nuevo, con bloqueo, dentro de la transacción del alta.
4. `fecha_fin` suma meses de calendario recortando al último día del mes (31/01 + 1 = 28 o 29/02), en fechas argentinas.
5. Único módulo de decisión: `backend/src/membresias/vigencia.ts` (`esVigente`, `esProxima`, `sumarMeses`, `obtenerVigente`, `obtenerProxima`, `obtenerCadena`). Los

consumidores (QR, reserva de clases, precio de cancha, pagos, listado de clientes, reportes) usan solo estas funciones o el fragmento SQL equivalente.

6. Alta única (`MembresiasService.alta`): toma `SELECT ... FOR UPDATE` sobre la fila del usuario dentro de la transacción del pago, calcula inicio y fin y aplica el tope. `create()` y `renovar()` dejan de existir.
7. Defensa en la base (script `010`, a ejecutar a mano): restricción de exclusión GiST sobre `usuario_id` y `daterange(fecha_inicio, fecha_fin, '[)')`, para filas con estado distinto de SUSPENDIDO (requiere la extensión `btree_gist`). El error 23P01 se traduce a 409.
8. El cron pasa a marcar VENCIDO con `fecha_fin <= hoy` (hora argentina). Es solo higiene: la corrección del sistema **no depende** de que corra, y las respuestas informan VENCIDO para una ACTIVO ya terminada.
9. `PATCH /membresias/:id` ya no asigna campos libres: solo `renovacionAuto` y ACTIVO <-> SUSPENDIDO. Reactivar exige que no se superponga ni exceda una vigente más una próxima (409 si no).
10. `GET /membresias/vigente/:id` conserva sus campos, tomados de la vigente (o de la última no próxima si no hay vigente), y agrega `proxima` (objeto o null).

Consecuencias

- **Positivas:** una sola definición de vigente; la base rechaza períodos superpuestos (23P01 → 409).
- **Negativas:** una segunda compra legítima queda como próxima y se cobra.
- **Deuda asumida:** la prueba de concurrencia real de la exclusión quedó cubierta por la estrategia del ADR-008 para RN-01 y RN-02, pero no para membresías.

Requisitos relacionados

RF-02, RN-03

Detalle de las consecuencias

- Con una vigente, una segunda compra legítima (o un doble clic que llega al servidor dos veces) queda como próxima y se cobra: la protección de doble clic sigue siendo la

del cliente (botón deshabilitado); el servidor solo garantiza que no haya más de una próxima ni superposiciones.

- Los reportes pasan a contar membresías vigentes hoy; se agregan `programadas` y, por socio, `fechaInicio`, `vigente` y `proxima`. Los campos existentes se conservan, pero el valor de `estado` ahora informa VENCIDO para una ACTIVO terminada.
- El "vencimiento" del listado de clientes es la `fecha_fin` (exclusiva) de la vigente, o de la más reciente no próxima.
- Hay que correr 009, limpiar datos superpuestos y recién entonces 010.
- Pendiente en el cliente: mostrar vigente y próxima, y tratar `fecha_fin` como exclusiva en textos y cuenta de días.

Alternativas descartadas

- **B — estado nuevo (programada/pendiente) activado al llegar su fecha:** cambia el enum de la base y depende de un proceso que active la fila; con el cron potencialmente apagado el socio quedaría sin acceso. Terminaría recalculando por fechas igual, con más piezas.
- **C — rechazar la compra mientras haya una vigente (permitir renovar solo a pocos días del vencimiento):** simple, pero elimina la renovación anticipada que la interfaz ofrece, y sin restricción en la base sigue habiendo carrera entre dos requests.

Limitación de las pruebas de concurrencia

Los tests usan repositorios simulados: verifican el orden (bloqueo antes de calcular), la lógica de encadenado, el tope y la traducción de 23P01, pero **no pueden probar** que el bloqueo de fila serialice dos transacciones reales ni que la restricción de exclusión rechace una inserción concurrente. Eso exige una base PostgreSQL real con dos conexiones simultáneas y `btree_gist` habilitada; queda como prueba manual o de integración pendiente.

Originalmente ADR 0013. Contenido sin cambios, adaptado a la plantilla del Anexo B.

ADR-023 — Almacenamiento de archivos intercambiable (Supabase Storage o disco local)

- **Estado:** Aceptado

- **Fecha:** 2026-11-09
- **Autor:** Retamoso, Máximo
- **Sprint:** 7

Contexto

Las fotos de usuario, las imágenes de tipos de cancha y los comprobantes PDF se subían directamente con `SupabaseStorageService`, inyectado por clase concreta en tres servicios. Para cumplir `docker compose up --build` sin cuentas externas (ADR-009), la API tenía que poder guardar archivos sin Supabase; sin eso, pagar una reserva fallaba al generar el comprobante.

Alternativas consideradas

Tabla 25

Alternativas consideradas (ADR-023)

Alternativa	A favor	En contra	Por qué se descartó
Clase abstracta `Almacenamiento` con dos implementaciones elegidas por configuración	Los servicios no conocen al proveedor; mismo patrón que el correo (ADR-021)	Una abstracción más	Elegida
MinIO en el compose (S3 compatible)	Más parecido a un storage real	Otro contenedor y cambiar el cliente de Supabase por uno S3	Más piezas para el mismo resultado
Pedir credenciales de Supabase para levantar el compose	Sin cambios de código	La cátedra no puede levantar el sistema sin cuentas del equipo	No cumple la condición de admisibilidad

Decisión

Los servicios dependen de la clase abstracta `Almacenamiento` (`backend/src/storage/almacenamiento.ts`: subir, eliminar y leer). `StorageModule` elige la implementación con `STORAGE_PROVIDER`: `SupabaseStorageService` por defecto (producción) o `LocalStorageService` con `local`, que guarda en `STORAGE_LOCAL_DIR` y sirve los archivos en `/archivos`. La descarga de comprobantes también pasa por el contrato y solo lee de nuestro propio almacenamiento.

Consecuencias

- **Positivas:** el sistema completo funciona en Docker sin Supabase; agregar otro proveedor es una clase nueva.
- **Negativas:** en modo local los archivos quedan en el volumen del contenedor, sin respaldo propio.
- **Deuda asumida:** las URLs públicas de comprobantes (en Supabase y en local) no exigen sesión: la descarga controlada es `GET /comprobantes/:id/archivo`.

Requisitos relacionados

RF-14, RNF-04

4. Modelado: clases, secuencia, estados, actividad, DER y despliegue

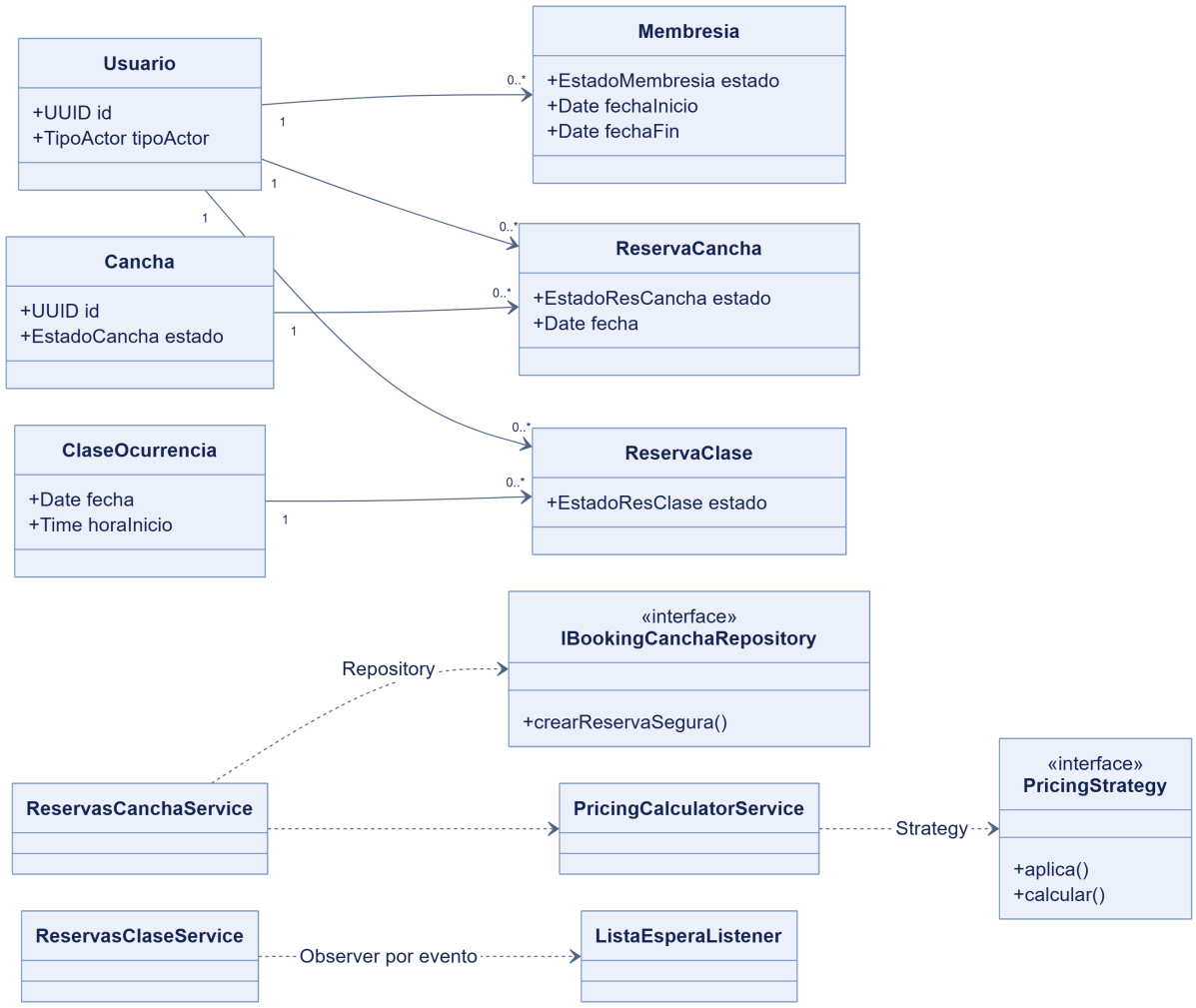
Los diagramas se mantienen como fuente Mermaid en el repositorio y se exportan a SVG y PNG; las imágenes de este capítulo son las exportaciones vigentes.

4.1 Diagrama de clases

El diagrama muestra las clases del dominio y sus relaciones. Las reglas de negocio se apoyan en estos tipos y en las restricciones de la base de datos.

Figura 5

Diagrama de clases del dominio



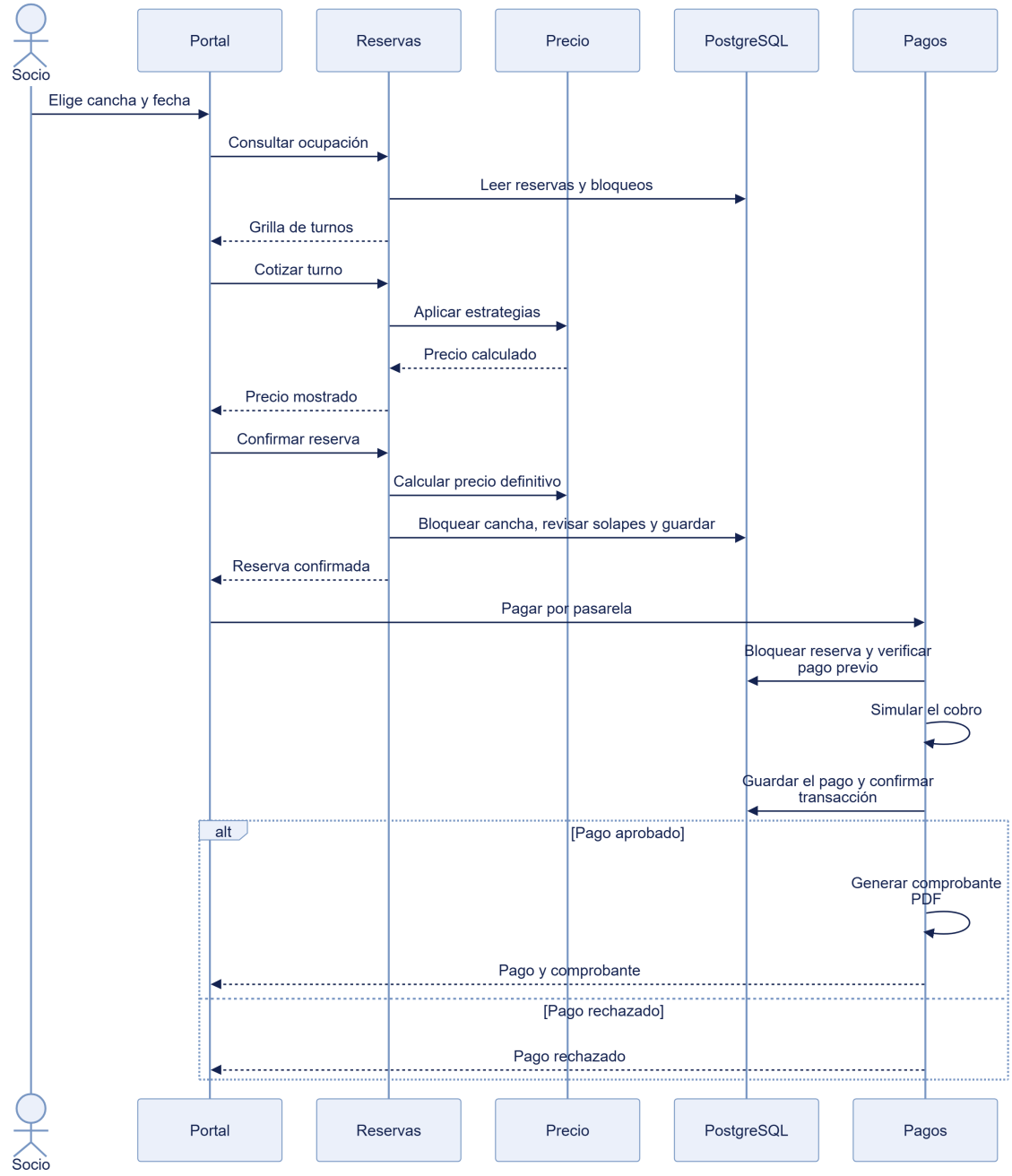
Nota. Elaboración del Equipo 08 (Retamoso, Máximo, 09/11/2026, v1.0) con Mermaid. Fuente editable: `docs/diagramas/uml-clases-dominio.mmd`. La imagen se incluye en su resolución original y puede ampliarse en el visor del PDF.

4.2 Diagramas de secuencia

Tres secuencias describen los recorridos más críticos: la reserva con pago, la reserva concurrente de una misma cancha (RN-02) y el ingreso por QR sin conexión (RNF-01).

Figura 6

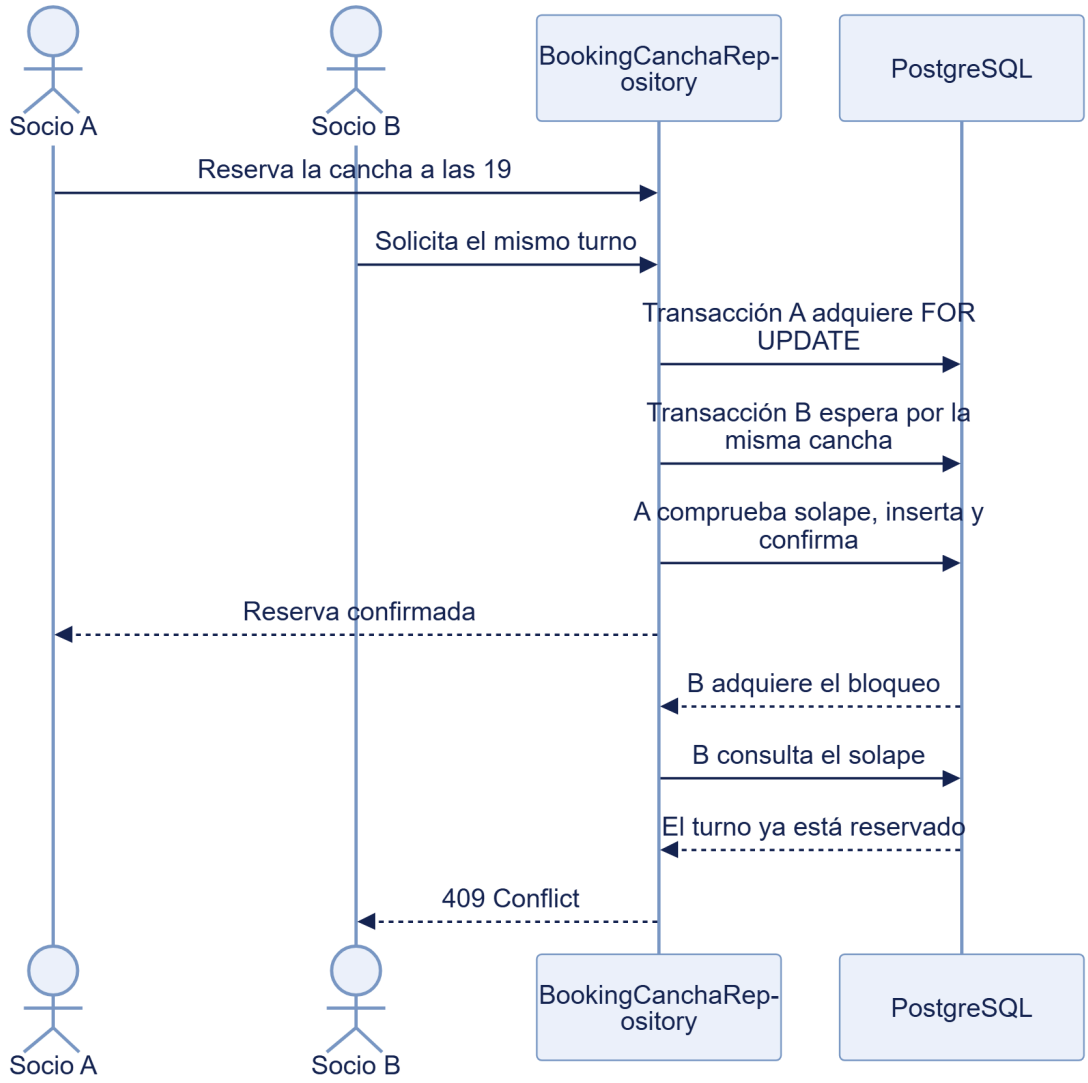
Secuencia: reserva y pago



Nota. Elaboración del Equipo 08 (Retamoso, Máximo, 09/11/2026, v1.0) con Mermaid. Fuente editable: `docs/diagramas/uml-secuencia-reserva-pago.mmd`. La imagen se incluye en su resolución original y puede ampliarse en el visor del PDF.

Figura 7

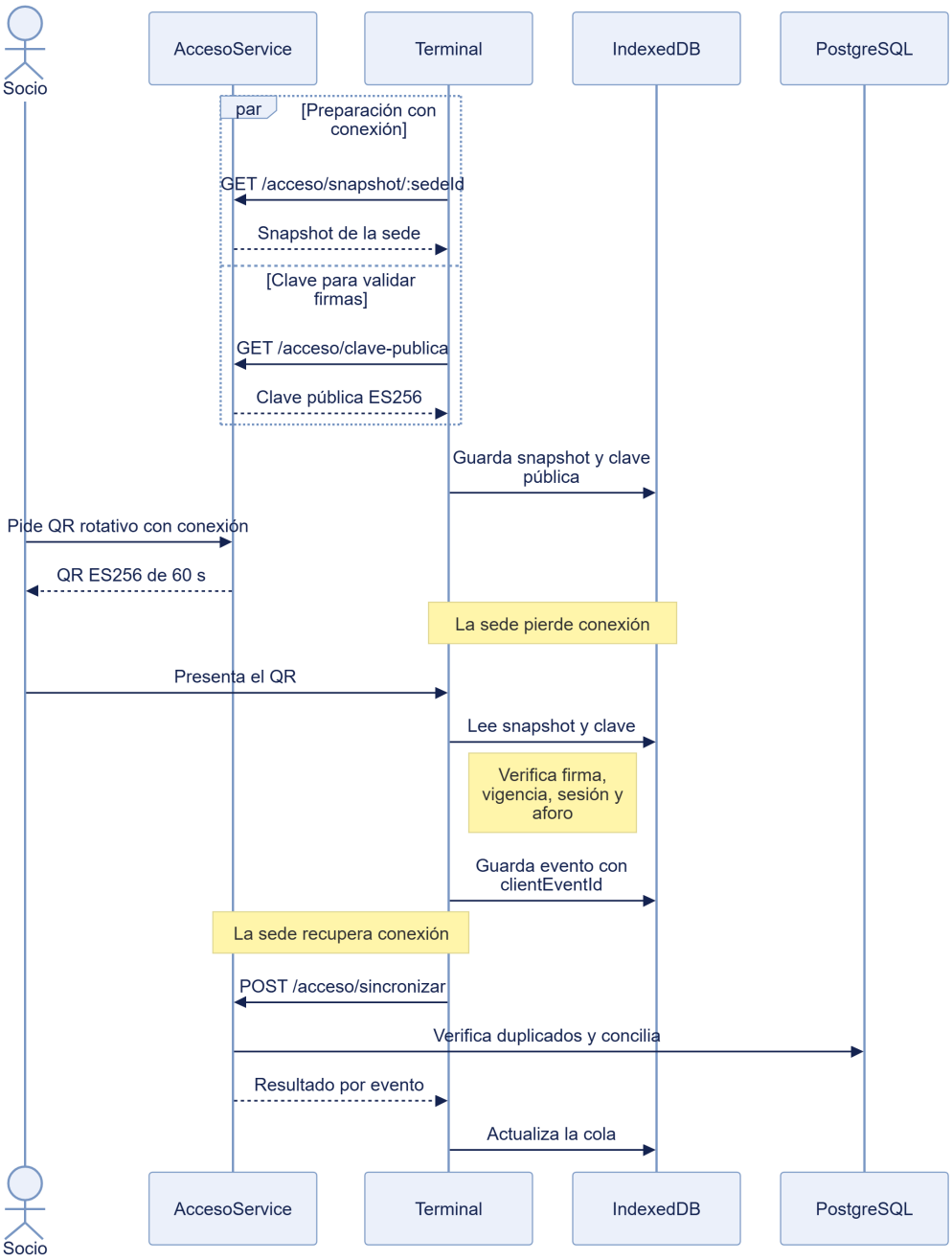
Secuencia: reserva concurrente (RN-02)



Nota. Elaboración del Equipo 08 (Retamoso, Máximo, 09/11/2026, v1.0) con Mermaid. Fuente editable: `docs/diagramas/uml-secuencia-reserva-concurrente.mmd`. La imagen se incluye en su resolución original y puede ampliarse en el visor del PDF.

Figura 8

Secuencia: ingreso con QR sin conexión (RNF-01)



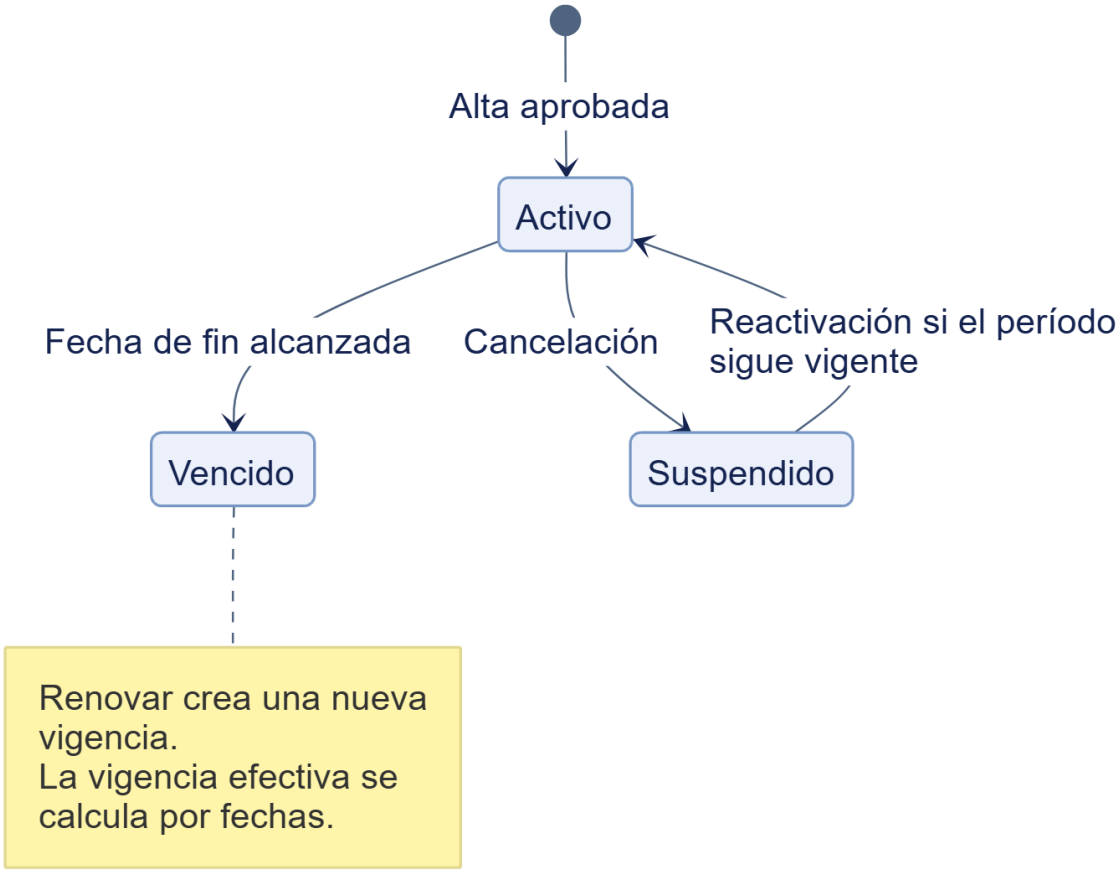
Nota. Elaboración del Equipo 08 (Retamoso, Máximo, 09/11/2026, v1.0) con Mermaid. Fuente editable: `docs/diagramas/uml-secuencia-acceso-offline.mmd`. La imagen se incluye en su resolución original y puede ampliarse en el visor del PDF. El diagrama asume una clave ES256 configurada y un snapshot vigente.

4.3 Diagramas de estados

El flujo de reserva de cancha persiste los estados CONFIRMADA y CANCELADA y modela el pago por separado; la reserva de clase incorpora además la lista de espera.

Figura 9

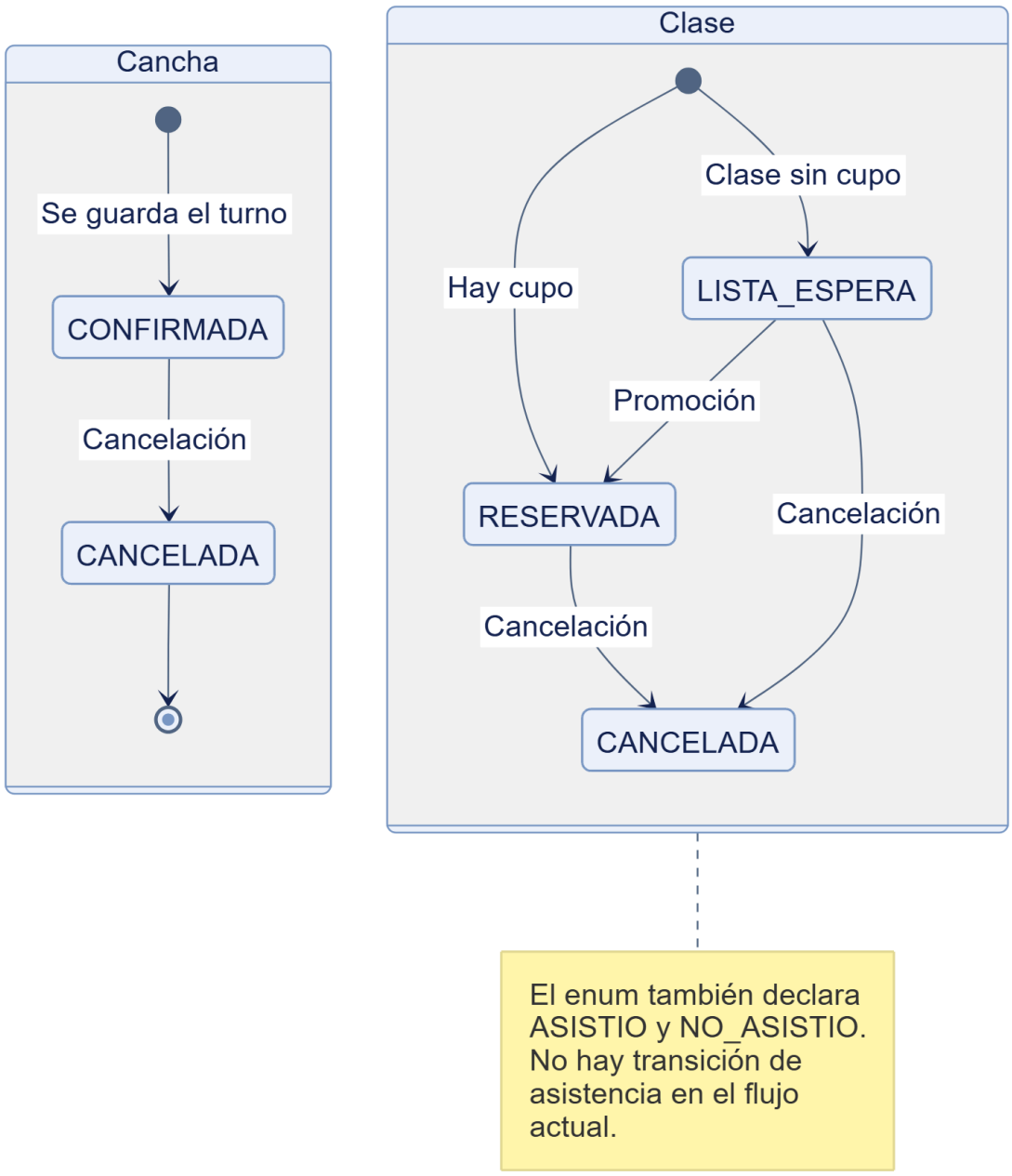
Estados de la membresía



Nota. Elaboración del Equipo 08 (Retamoso, Máximo, 09/11/2026, v1.0) con Mermaid. Fuente editable: `docs/diagramas/uml-estados-membresia.mmd`. La imagen se incluye en su resolución original y puede ampliarse en el visor del PDF.

Figura 10

Estados de la reserva



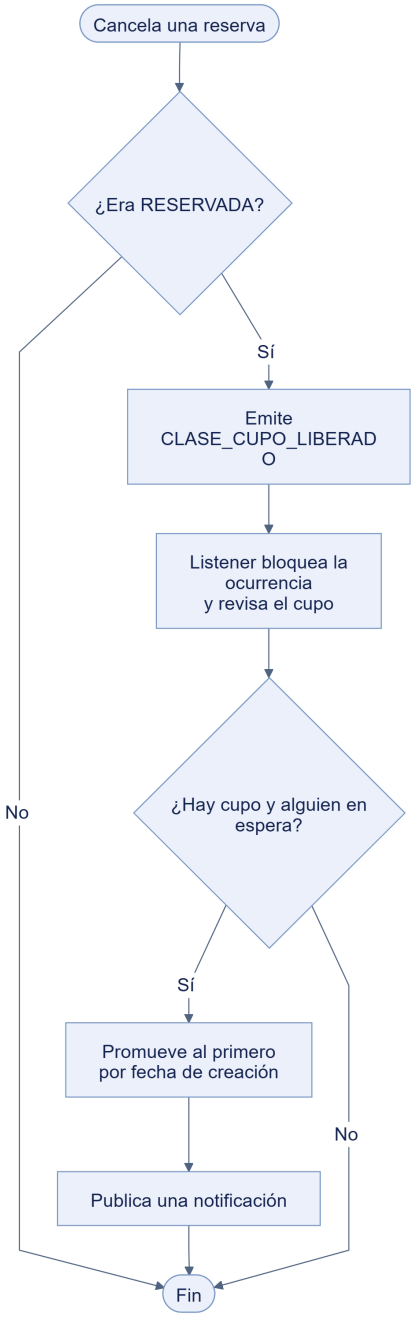
Nota. Elaboración del Equipo 08 (Retamoso, Máximo, 09/11/2026, v1.0) con Mermaid. Fuente editable: `docs/diagramas/uml-estados-reserva.mmd`. La imagen se incluye en su resolución original y puede ampliarse en el visor del PDF.

4.4 Diagrama de actividad

La actividad describe cómo se promueve a un socio desde la lista de espera cuando se libera un cupo de clase.

Figura 11

Actividad: lista de espera de clases



Nota. Elaboración del Equipo 08 (Retamoso, Máximo, 09/11/2026, v1.0) con Mermaid. Fuente editable: `docs/diagramas/uml-actividad-lista-espera.mmd`. La imagen se incluye en su resolución original y puede ampliarse en el visor del PDF.

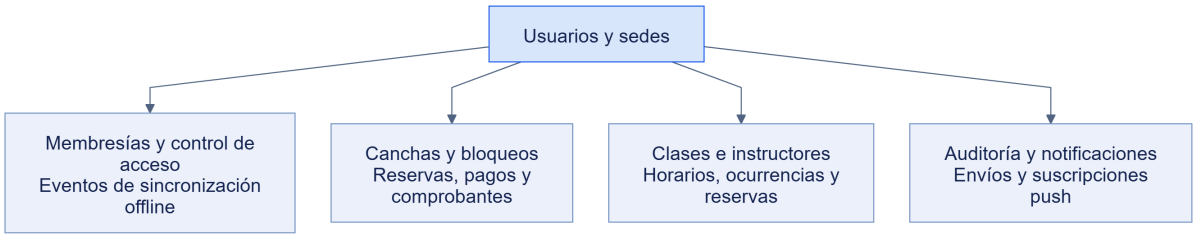
4.5 Diagrama entidad-relación (DER)

El mapa general funciona como índice del modelo físico. Las cuatro vistas de detalle conservan columnas, claves e índices; las tablas de referencia se repiten con su clave cuando una relación cruza de vista. Las restricciones que sostienen las reglas de negocio son

uq_cancha_horario_confirmado (RN-02), uq_acceso_abierto_por_usuario (RN-01) y uq_evento_offline_jti (detección de QR repetidos en la conciliación).

Figura 12

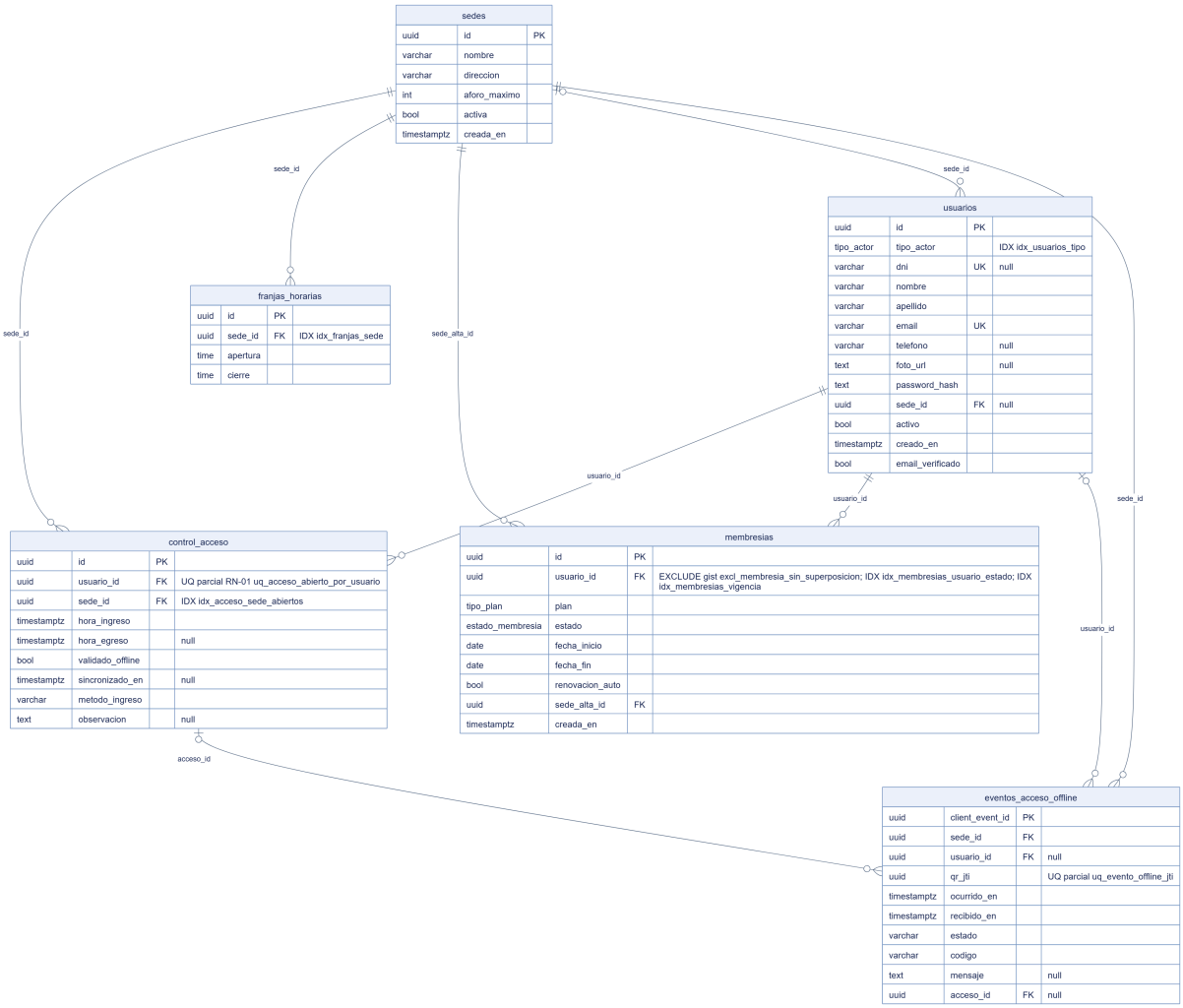
DER físico: mapa general



Nota. Elaboración del Equipo 08 (Retamoso, Máximo, 09/11/2026, v2.0) con Mermaid. Fuente editable: `docs/diagramas/der-fisico.mmd`. La imagen se incluye en su resolución original y puede ampliarse en el visor del PDF.

Figura 13

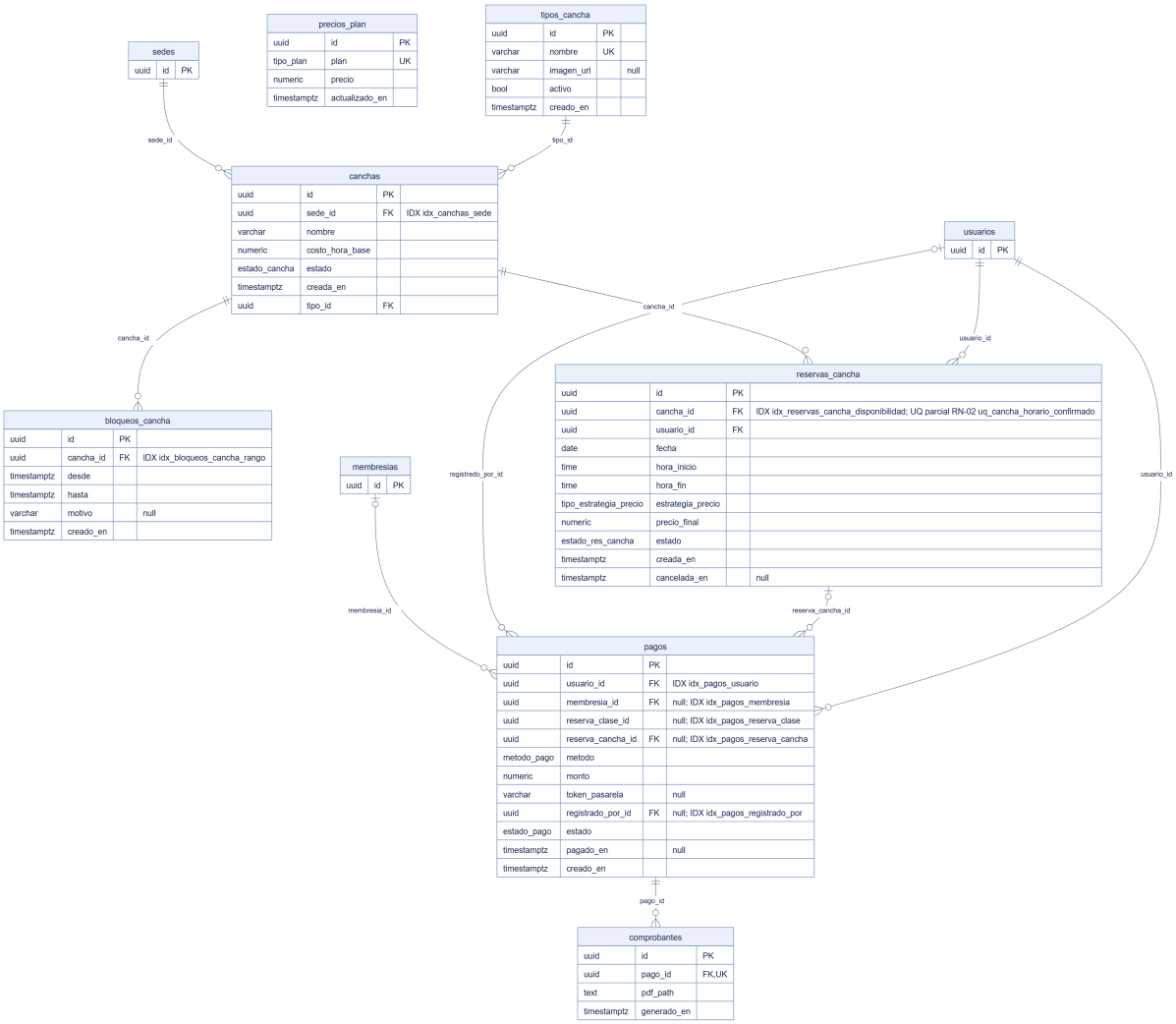
DER físico: sedes, socios y acceso



Nota. Elaboración del Equipo 08 (Retamoso, Máximo, 09/11/2026, v1.0) con Mermaid. Fuente editable: `docs/diagramas/der-acceso.mmd`. La imagen se incluye en su resolución original y puede ampliarse en el visor del PDF.

Figura 14

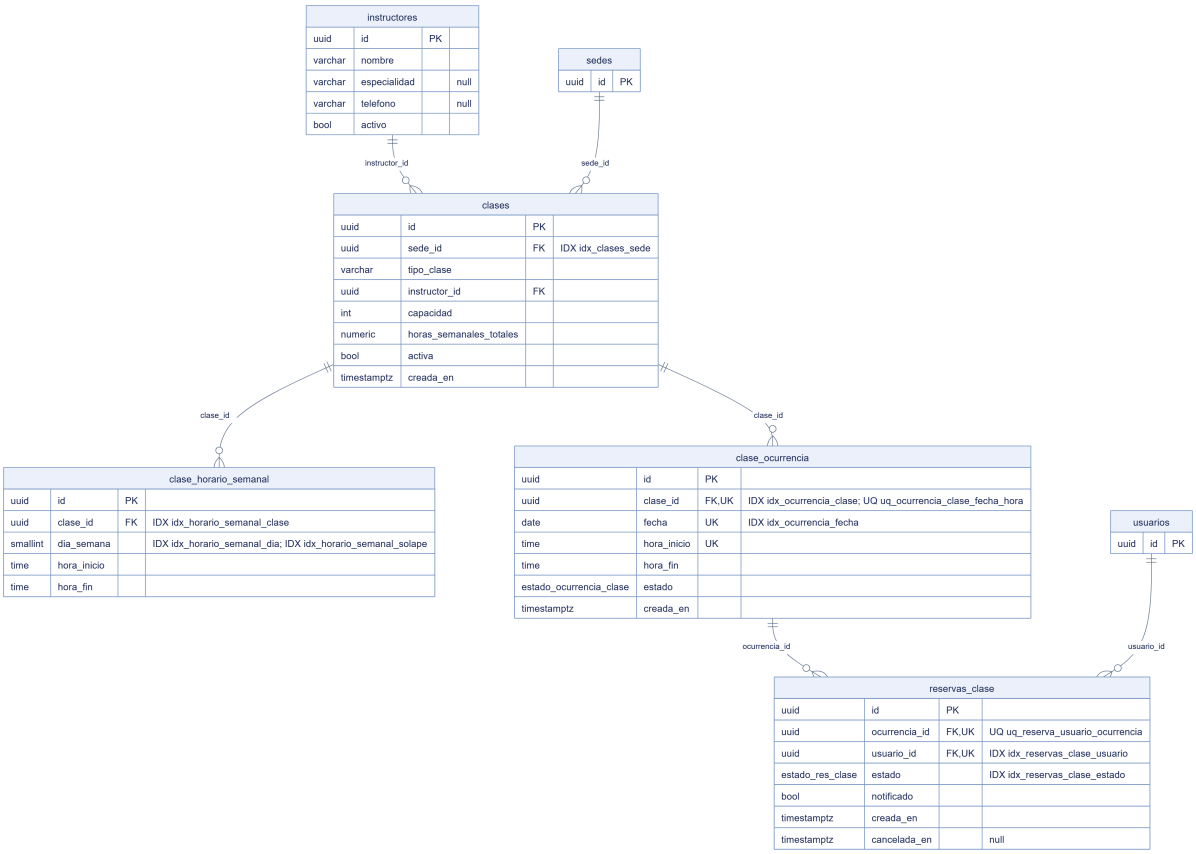
DER físico: canchas y pagos



Nota. Elaboración del Equipo 08 (Retamoso, Máximo, 09/11/2026, v1.0) con Mermaid. Fuente editable: `docs/diagramas/der-canchas-pagos.mmd`. La imagen se incluye en su resolución original y puede ampliarse en el visor del PDF.

Figura 15

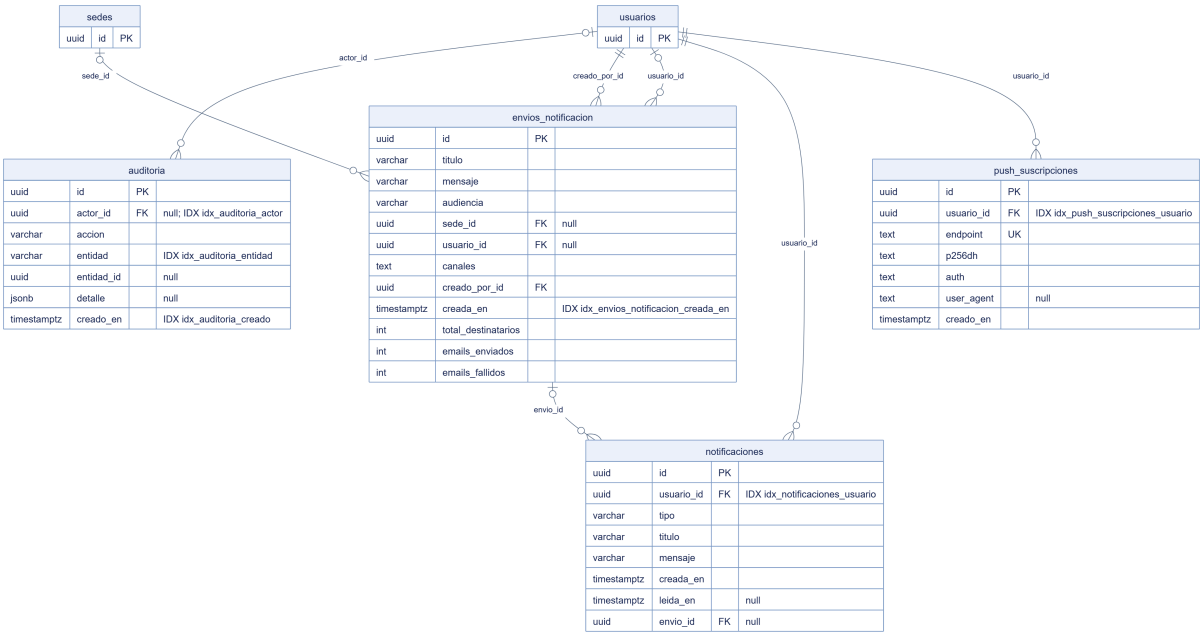
DER físico: clases y reservas



Nota. Elaboración del Equipo 08 (Retamoso, Máximo, 09/11/2026, v1.0) con Mermaid. Fuente editable: `docs/diagramas/der-clases.mmd`. La imagen se incluye en su resolución original y puede ampliarse en el visor del PDF.

Figura 16

DER físico: auditoría y avisos



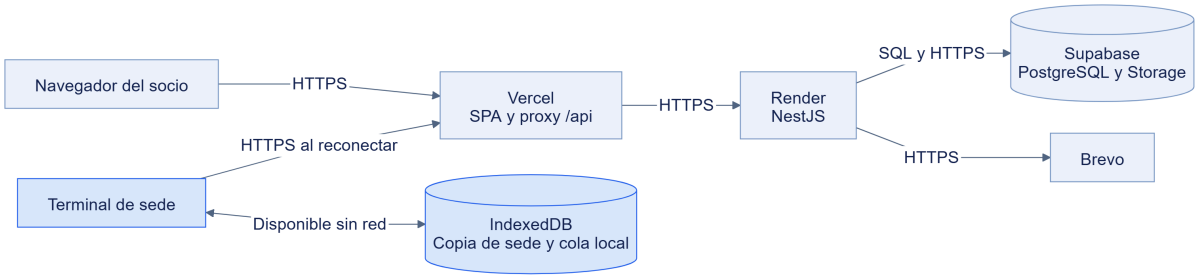
Nota. Elaboración del Equipo 08 (Retamoso, Máximo, 09/11/2026, v1.0) con Mermaid. Fuente editable: `docs/diagramas/der-auditoria-avisos.mmd`. La imagen se incluye en su resolución original y puede ampliarse en el visor del PDF.

4.6 Diagrama de despliegue

El despliegue muestra los nodos de ejecución y la ruta desde el repositorio hasta producción.

Figura 17

Diagrama de despliegue



Nota. Elaboración del Equipo 08 (Retamoso, Máximo, 09/11/2026, v2.0) con Mermaid. Fuente editable: `docs/diagramas/despliegue.mmd`. La imagen se incluye en su resolución original y puede ampliarse en el visor del PDF.

5. Patrones de diseño aplicados y su justificación

Los patrones se aplicaron donde resuelven un problema concreto del dominio, no por decoración (Gamma et al., 1994). Los tres patrones de comportamiento y persistencia exigidos por la unidad (Strategy, Observer y Repository) se describen con su ubicación en el código y la razón de su elección; los restantes se resumen en una tabla.

Strategy — precio dinámico de canchas

Tabla 26

Strategy — precio dinámico de canchas (Patrones)

Rol	Clase	Archivo
Interfaz	`PricingStrategy` (`aplica(contexto)`, `calcular(precio)`)	`canchas/pricing/pricing-strategy.interface.ts`
Estrategias concretas	`MemberDiscountPricing` (−15 %), `PeakHourPricing` (+15 % de 19 a 21 h)	`canchas/pricing/*.strategy.ts`
Contexto	`PricingCalculatorService`	`canchas/pricing/pricing-calculator.service.ts`
Registro	proveedor `PRICING_STRATEGIES` con `useFactory`	`canchas/canchas.module.ts`

El contexto recibe el arreglo de estrategias por inyección, aplica en orden las que dicen `aplica()` y devuelve el precio y la estrategia resultante (`ESTANDAR`, `SOCIO_DESCUENTO`, `HORA_PICO`, `SOCIO_HORA_PICO`), que queda guardada en la reserva. La **tarifa de externo** para el socio en mora (RN-03) sale de que `MemberDiscountPricing` no aplica cuando la membresía no está vigente; el estado lo decide el patrón **State** de `membresias/estado-membresia.ts`.

El mismo patrón se usa para los canales de notificación (`notificaciones/notificadores/`) y los proveedores de correo (`common/mail/`, ADR-021).

Justificación. Las reglas de precio (descuento de socio y recargo de hora pico) cambian con el negocio y se combinan entre sí. Encapsular cada una en una estrategia evita condicionales en el servicio de reservas y permite sumar una regla nueva registrándola como proveedor, sin modificar el calculador; el manual técnico lo demuestra con el caso resuelto de agregar una estrategia de precio.

Observer — lista de espera de clases

Tabla 27

Observer — lista de espera de clases (Patrones)

Rol	Elemento	Archivo
Evento	`CLASE_CUPO_LIBERADO`	`clases/events/clase.events.ts`
Publicador	`ReservasClaseService.cancelar()` emite con `EventEmitter2`	`clases/reserva-clase.service.ts`
Suscriptor	`ListaEsperaListener` (`@OnEvent`)	`clases/listeners/lista-espera.listener.ts`

Al cancelar una reserva, el servicio emite el evento sin saber quién lo escucha. El listener toma la primera reserva en `LISTA_ESPERA` de esa ocurrencia (por orden de creación), la pasa a `RESERVADA` y notifica al socio por los canales configurados. Agregar otra reacción (por ejemplo, avisar al instructor) es agregar otro listener, sin tocar el servicio.

Justificación. Cancelar una reserva debe liberar un cupo y promover al primero de la lista de espera, pero el servicio de reservas no debe conocer esa consecuencia. El evento desacopla ambos lados: agregar otra reacción es agregar otro listener, sin tocar el servicio.

Repository — abstracción de la persistencia

Tabla 28

Repository — abstracción de la persistencia (Patrones)

Interfaz	Implementaciones	Token
`IBookingCanchaRepository`	`TypeOrmBookingCanchaRepository`, `InMemoryBookingCanchaRepository`	`BOOKING_CANCHA_REPOSITORY`
`IBookingClaseRepository`	`TypeOrmBookingClaseRepository`, `InMemoryBookingClaseRepository`	`BOOKING_CLASE_REPOSITORY`

Los servicios dependen de la interfaz; el módulo decide la implementación. La de TypeORM encapsula la transacción y el bloqueo (sección 5); la de memoria permite probar los servicios sin base. El resto del acceso a datos usa `Repository<T>` de TypeORM, que ya es un repositorio genérico.

Justificación. La interfaz aísla la persistencia: la implementación de TypeORM encapsula la transacción y el bloqueo que garantizan RN-02 (ADR-004), y la implementación

en memoria permite probar los servicios sin base de datos, en línea con la pirámide de pruebas del ADR-008.

Patrones creacionales y estructurales

Tabla 29

Patrones creacionales y estructurales (Patrones)

Patrón	Dónde	Para qué
Factory	`crearEnviadorCorreo()` (`common/mail/crear-enviador.ts`), `NotificadorFactory` (`notificaciones/notificadores/notificador.factory.ts`), `StorageModule` (`useFactory`)	Elegir la implementación por configuración o por canal
Facade	`ReservasCanchaService.reservar()`	Un punto de entrada que orquesta validaciones, precio y reserva
Singleton	Proveedores de Nest (alcance por defecto)	Una instancia por aplicación (por ejemplo, el registro de QR usados)
Adapter	`SupabaseStorageService` y `LocalStorageService` sobre `Almacenamiento`; `PasarelaMockService` sobre `PasarelaPago`	Adaptar servicios externos a un contrato propio
Decorator	`@Auditable()` + `AuditoriaInterceptor`; `@Roles()`, `@CurrentUser()`	Agregar comportamiento transversal sin tocar los controladores
State	`membresias/estado-membresia.ts`	Comportamiento según el estado de la membresía (QR, tarifa)

Justificación. Factory, Adapter y Decorator aparecen donde el sistema depende de piezas intercambiables (proveedores de correo, almacenamiento, pasarela) o de comportamiento transversal (auditoría y roles); Facade ordena el flujo de reserva en un único punto de entrada; State concentra las reglas que dependen del estado de la membresía.

6. Estrategia de pruebas y reporte de cobertura

La estrategia sigue una pirámide de pruebas con umbral de cobertura declarado en el ADR-008 y verificado por el pipeline: pruebas unitarias con dobles en memoria, pruebas de integración contra PostgreSQL real para las reglas que dependen de la concurrencia y verificación del sistema completo con Docker.

6.1 Niveles de prueba y resultados

Tabla 30

Niveles de prueba y resultados

Nivel	Comando	Resultado
Unitarias del backend	`npm test` (en `backend/`)	848 pruebas en 42 suites, todas en verde
Unitarias del frontend	`npm test` (en `frontend/`)	153 pruebas en 7 archivos, todas en verde
Integración contra PostgreSQL (RN-01, RN-02, cupo de clases, sincronización offline)	`npm run test:integracion` (en `backend/`)	18 pruebas en 5 suites. Requieren una base PostgreSQL; las corre el job `integracion` del CI
Sistema	job `docker` del CI	Levanta con `docker compose up --build` y responde en el CI

Nota. Fuente: README del repositorio, sección 6, y `docs/manuales/manual-tecnico.md`, sección 8.

6.2 Cobertura del backend

La cobertura de las pruebas unitarias del backend se obtuvo con `npm run test:cov`. Las cuatro métricas superan el umbral mínimo declarado en el ADR-008 y configurado en `coverageThreshold`; si el resultado queda por debajo, el comando falla y el CI bloquea el merge. El frontend no tiene umbral ni informe de cobertura.

Tabla 31

Cobertura de las pruebas unitarias del backend

Métrica	Obtenido (%)	Umbral (%)	Cumple
Sentencias	73,65	70	Sí
Ramas	64,30	60	Sí
Funciones	58,99	55	Sí
Líneas	73,88	70	Sí

Nota. Informe detallado: `backend/coverage/lcov-report/index.html`, generado por `npm run test:cov`.

6.3 Pipeline de integración y entrega continuas

Archivo: `.github/workflows/ci.yml`. Se ejecuta en cada push y pull request a `main`.

Tabla 32

Jobs del pipeline de integración continua

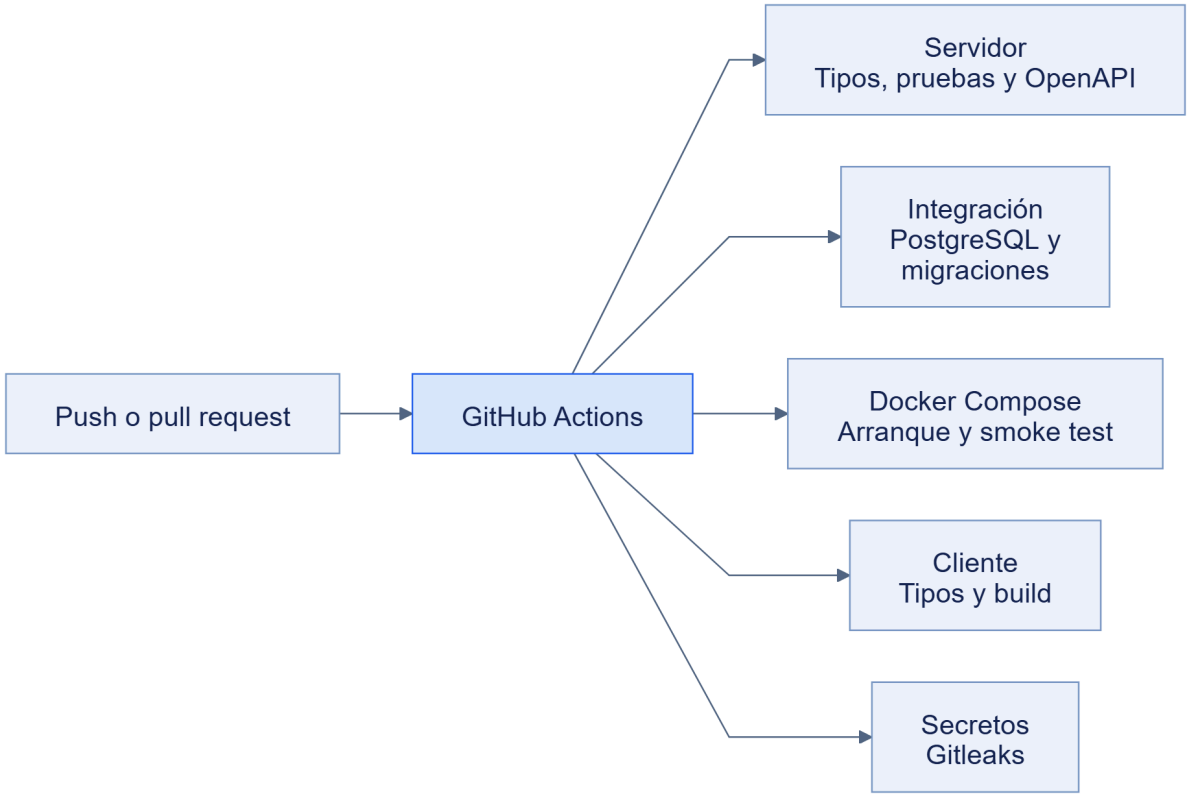
Job	Pasos	Falla si
`secretos`	gitleaks sobre todo el historial	Hay una credencial versionada
`servidor`	`npm ci`, `tsc --noEmit`, pruebas con cobertura, `npm run build`, `npm run swagger:export` y comparación	Error de tipos, prueba roja, cobertura bajo el umbral o `swagger.json` desactualizado
`integracion`	Servicio `postgres:17`, pruebas de integración, revertir todas las migraciones y volver a aplicarlas con el seed	Falla una regla bajo concurrencia o una migración no es reversible
`docker`	`docker compose up --build --wait`, chequeo de portal, API, Swagger y 25 sedes	El sistema no levanta desde cero
`cliente`	`npm ci`, `npm test` (vitest), `tsc -b`, `vite build`	Falla una prueba del portal o el portal no compila

Con la protección de la rama `main` configurada para exigir estos checks, un pull request no se puede mergear con el pipeline en rojo. Después del merge, Render y Vercel despliegan automáticamente. Ver el diagrama.

Contrato OpenAPI: si se cambia un endpoint, regenerar `backend/swagger.json` con `npm run swagger:export` y versionarlo junto con el cambio.

Figura 18

Pipeline de integración continua



Nota. Elaboración del Equipo 08 (Retamoso, Máximo, 09/11/2026, v2.0) con Mermaid. Fuente editable: `docs/diagramas/pipeline.mmd`. La imagen se incluye en su resolución original y puede ampliarse en el visor del PDF.

6.4 Otros informes de pruebas

El repositorio conserva, además, el reporte de QA del sprint 5 (`docs/PRUEBAS.md`), con pruebas de uso, de seguridad y de casos simultáneos realizadas sobre producción, y el informe de auditoría de testing del 29/10/2026 (`docs/testing/informe-auditoria-testing.md`), que documenta el estado previo a las pruebas de integración y al pipeline.

7. Cumplimiento de los requisitos no funcionales y su evidencia

Los cuatro requisitos no funcionales (RNF-01 a RNF-04) del enunciado (UNER, 2026a) se tradujeron en decisiones de arquitectura. La tabla resume cada requisito, cómo se cumple y dónde está la evidencia; el apartado siguiente transcribe el único informe que hoy existe en `docs/rnf`.

Tabla 33

Cumplimiento de los requisitos no funcionales

RNF	Requisito (según los ADR)	Cómo se cumple	Evidencia en el repositorio
RNF-01	El control de acceso funciona aunque se caiga la conexión principal (disponibilidad de 99,5 %). ADR-003.	QR firmado con ES256, réplica local de socios en el terminal, cola de ingresos y conciliación al reconectar (ADR-003, ADR-007).	`acceso-offline.spec.ts` y `acceso-sincronizacion.spec.ts`; pruebas de integración de la sincronización; manual técnico, sección 6. La prueba de punta a punta en producción figura como pendiente (FT1-130).
RNF-02	No se almacenan datos de tarjeta, solo el token de la pasarela. ADR-010.	Pasarela simulada que devuelve un token; ninguna entidad tiene campos de tarjeta.	`pagos/gateway/pasarela-mock.service.ts`; `pagos/pasarela-cancha.spec.ts`.
RNF-03	La consulta de disponibilidad de canchas responde en milisegundos. ADR-002.	Índices del esquema SQL, entre ellos `idx_reservas_cancha_disponibilidad` para la grilla de canchas.	34 sentencias `CREATE INDEX` en `backend/db/sql`. El repositorio no contiene una medición de tiempos de respuesta.
RNF-04	Se puede dar de alta una sede nueva sin detener el sistema. ADR-001.	Las sedes son datos: se crean por API (`POST /sedes`) o desde la consola, sin reiniciar ni desplegar.	Manual de instalación y operación, sección 9; datos de prueba con 25 sedes.

Nota. Elaboración propia a partir de los ADR, el manual técnico y el código del repositorio.

7.1 Evidencia disponible en docs/rnf

La carpeta `docs/rnf` contiene un único informe: el escaneo de secretos sobre el historial completo del repositorio, que respalda la condición de admisibilidad de no versionar credenciales. Se transcribe a continuación.

Escaneo de secretos en el historial completo

Fecha: 2026-11-09 · **Alcance:** las 112 versiones de todas las ramas (`git log --all -p`), no solo el estado actual.

Condición de admisibilidad: *no hay credenciales, claves ni tokens reales en el repositorio; solo* `.env.example`.

Método

Se recorrió cada línea agregada en cada commit buscando estos patrones:

Tabla 34

Método (Escaneo de secretos)

Patrón	Qué detecta
`eyJ….….…`	JWT, incluidas las claves `anon` y `service_role` de Supabase
`postgres://usuario:clave@`	Cadenas de conexión con contraseña
`xkeysib-[hex]`	Claves de API de Brevo
`AKIA…`, `ghp_…`, `AIza…`, `sk_live…`, `APP_USR-…`	AWS, GitHub, Google, Stripe, Mercado Pago
`-----BEGIN … PRIVATE KEY-----`	Claves privadas (incluida la VAPID)
`xxxx xxxx xxxx xxxx`	Contraseñas de aplicación de Gmail
`[a-z0-9]{20}.supabase.co`	Identificador del proyecto de Supabase
`*_SECRET`, `*_PASSWORD`, `*_API_KEY`, `*_TOKEN` con valor literal	Asignaciones de secretos en código o configuración

Además se listaron todos los archivos `.env*` que se agregaron alguna vez al historial.

Resultado

Sin secretos reales. Ningún archivo `.env` se versionó nunca (solo `backend/.env.example` y `frontend/.env.example`, sin valores) y no aparece ninguna clave, token ni cadena de conexión real.

Coincidencias revisadas una por una, todas falsos positivos:

Tabla 35

Resultado (Escaneo de secretos)

Archivo	Qué es
`backend/src/common/mail/brevo.enviador.spec.ts`	Constante `API_KEY` de prueba con el texto `xkeysib-clave-super-secreta-…`: no es hexadecimal ni tiene el formato de una clave de Brevo.
`backend/src/acceso/acceso.service.spec.ts`, `crear-enviador.spec.ts`, `email-verificacion.service.spec.ts`, `usuarios-duplicados.spec.ts`, `seguridad-lecturas.spec.ts`	Valores de prueba (`'secreto-solo-para-pruebas'`, `'clave-secreta'`, `'p'`, `'123…'`).
`.github/workflows/ci.yml`	Contraseña del Postgres efímero del job de integración, que solo existe durante la corrida.
`frontend/src/…/CrearStaff.tsx`, `VerificarEmail.tsx`, `ControlAcceso.tsx`	Nombres de campos (`password: form.password`, `qrToken`), no valores.
`backend/README.md`	Enlace de una imagen de la plantilla de NestJS.

Archivo	Qué es
`docker-compose.yml`	Valores por defecto solo de desarrollo local (`fitzone-local`, `solo-para-desarrollo-local-cambiar`), reemplazables con `.env`.
`backend/db/seed/datos-prueba.sql`	Hash bcrypt de la contraseña de los usuarios de prueba, documentada en el README.

Prevención

- `.env` está en los `.gitignore` de `backend/` y `frontend/`, y en los `.dockerignore`.
- El job `secretos` del CI corre gitleaks (https://github.com/gitleaks/gitleaks) sobre todo el historial en cada push y pull request; si encuentra algo, el pipeline falla.
- Si alguna vez se versiona una clave real, borrarla del archivo no alcanza: hay que **rotarla** en el proveedor (Supabase, Brevo, Gmail) porque queda en el historial.

7.2 Evidencia pendiente

El tablero de Jira mantiene la tarjeta FT1-117, «Evidencia de requisitos no funcionales en docs/rnf», en estado «Por hacer». Por ese motivo `docs/rnf` no incluye todavía mediciones de disponibilidad (RNF-01) ni de tiempo de respuesta (RNF-03), y la evidencia de la tabla anterior se apoya en las decisiones de arquitectura, el código y las pruebas.

8. Planificación: plan, cronograma, trazabilidad, seguimiento y riesgos

Se incorpora el plan de proyecto completo (`docs/planificacion/PLANIFICACION.md`), elaborado a partir de la exportación del tablero de Jira y del historial del repositorio. Sus secciones cubren alcance, supuestos, equipo, estructura de desglose del trabajo, cronograma, asignación, matriz de trazabilidad, seguimiento por sprint, riesgos y análisis de desvíos.

Plan de proyecto del Trabajo Integrador Final: caso FitZone Sports

Este documento presenta la planificación del Trabajo Integrador Final (Universidad Nacional de Entre Ríos [UNER], 2026a) y se elaboró el 09/11/2026 a partir de la exportación del tablero de Jira (proyecto FT1) y del historial del repositorio. Sus secciones siguen la plantilla de los lineamientos de entrega (UNER, 2026b).

Nota de transparencia: este plan se redactó al cierre del cuatrimestre. El tablero de Jira no registra estimaciones, horas trabajadas ni fechas de resolución, y parte de las tarjetas se creó de forma retroactiva (ver la sección 7 y el análisis de desvíos). Los datos de este documento son los que existen en el tablero; no se completaron con valores inventados.

1. Alcance

Incluido:

- Módulos M1 a M5 del enunciado: usuarios y membresías, acceso, clases grupales, canchas y pagos (RF-01 a RF-14), con las reglas de negocio RN-01 a RN-03 y los requisitos no funcionales RNF-01 a RNF-04.
- API REST como monolito modular (NestJS y TypeORM) sobre PostgreSQL (Supabase), con autenticación JWT, roles y alcance por sede.
- Portal responsive del socio, instalable como PWA, y consola de administración para recepción y gerencia.
- Validación del QR en el terminal de sede sin conexión (QR firmado con ES256, réplica local de socios, cola de ingresos y sincronización).
- Pasarela de pagos simulada y comprobante en PDF.
- Pipeline de integración y entrega continuas (GitHub Actions), contenedorización con Docker y documentación (ADR, diagramas, manuales).

Excluido, con su motivo:

- Aplicación móvil nativa (React Native o Flutter): se entrega el portal como PWA instalable (ADR-011) para no duplicar el cliente.
- Pasarela de pagos real y facturación fiscal: el enunciado pide una pasarela simulada.
- Cifrado del almacenamiento local del terminal y revocación de membresías en tiempo real mientras está sin conexión: la réplica local vence a las 24 horas; queda documentado como límite conocido.

2. Supuestos y restricciones

Supuestos:

- La infraestructura de planes gratuitos (Render, Supabase y Vercel) alcanza para la demostración.
- En el modo sin conexión, el teléfono del socio conserva datos móviles y es el terminal de la sede el que pierde la conexión.
- El reloj del dispositivo del terminal es razonablemente correcto.

Restricciones:

- Entrega del repositorio con el tag v1.0-entrega el jueves 19/11/2026 a las 18:00; la semana 16 queda para el Demo Day.
- Equipo de tres integrantes, conocido por la cátedra desde el inicio.
- Pila tecnológica asociada a las unidades de la materia (NestJS, React, PostgreSQL).

3. Equipo

Tabla 36

Equipo (Planificación)

Integrante	Rol	Horas comprometidas	Responsabilidad principal (según las tarjetas del tablero)
Martín Zanandrea	Backend, líder técnico	No registradas al planificar	Usuarios y membresías, canchas, autenticación y alcance por sede, patrones Strategy, Observer y Repository, CI/CD y despliegue, acceso offline
Teo Ava	Frontend	No registradas al planificar	Pantallas del socio y de administración, estado global, clases grupales, ADR de autenticación y de cálculo de montos
Máximo Retamoso	Calidad, DevOps y documentación	No registradas al planificar	Acceso y pagos, pruebas de integración y QA, datos de prueba, Docker, ADR, diagramas y manuales

El código de rol P1 a P3 de cada integrante se consigna en el README del repositorio.

4. Estructura de desglose del trabajo

Épica, historia y subtarea según el tablero de Jira. El tablero no registra estimaciones ni criterios de aceptación para las historias creadas antes del 09/11/2026; las tarjetas FT1-122 a FT1-132, creadas el 09/11/2026 a las 23:44, sí incluyen criterios de aceptación. Las estimaciones faltantes se analizan en la sección 8.

FT1-1 — Unidad I — Arquitectura (Listo)

Tabla 37

FT1-1 — Unidad I — Arquitectura (Listo) (Planificación)

Clave	Historia o subtarea	Responsable	Sprint	Estado
FT1-9	Debate grupal monolito modular vs microservicios (ADR)	Máximo Retamoso	Sprint 1	Listo
FT1-10	Niveles C4 y ADR fundacionales	Teo Ava	Sprint 1	Listo
FT1-16	ADR - Arquitectura monolito modular	Máximo Retamoso	Sprint 1	Listo
FT1-17	ADR - Selección de stack tecnológico principal	Martín Zanandrea	Sprint 1	Listo

FT1-2 — Unidad II — Frameworks (Listo)

Tabla 38

FT1-2 — Unidad II — Frameworks (Listo) (Planificación)

Clave	Historia o subtarea	Responsable	Sprint	Estado
FT1-11	Scaffolding NestJS, estructura modular e inyección de dependencias	Martín Zanandrea	Sprint 2	Listo
FT1-14	Documentación de la API con Swagger	Teo Ava	Sprint 2	Listo
FT1-15	ADR - GitHub como repositorio compartido	Máximo Retamoso	Sprint 2	Listo
FT1-18	ADR - Modelado de datos	Martín Zanandrea	Sprint 2	Listo
FT1-19	ADR - Autenticación JWT y autorización por rol/dueño/sede	Teo Ava	Sprint 2	Listo
FT1-20	ADR - Flujo obligatorio de membresía y pago	Martín Zanandrea	Sprint 2	Listo
FT1-21	ADR - Cálculo de montos server-side	Teo Ava	Sprint 2	Listo
FT1-22	ADR - Sistema de auditoría en dos capas	Máximo Retamoso	Sprint 2	Listo

Clave	Historia o subtarea	Responsable	Sprint	Estado
FT1-35	TypeORM + PostgreSQL (Supabase) y entidades	Martín Zanandrea	Sprint 2	Listo
FT1-36	M1 Usuarios y Membresías (RF-01, RF-02, RF-03)	Martín Zanandrea	Sprint 2	Listo
FT1-37	M2 Gimnasio y Acceso: QR, validación, egreso y aforo (RF-04, RF-05, RN-01)	Máximo Retamoso	Sprint 2	Listo
FT1-38	M3 Clases Grupales (RF-06, RF-07)	Teo Ava	Sprint 2	Listo
FT1-39	M4 Canchas: recursos, reservas y mantenimiento (RF-09, RF-10, RF-12)	Martín Zanandrea	Sprint 2	Listo
FT1-40	M5 Pagos: pasarela mock y comprobante PDF (RF-13, RF-14, RNF-02)	Máximo Retamoso	Sprint 2	Listo
FT1-41	Autenticación JWT, roles y alcance por sede	Martín Zanandrea	Sprint 2	Listo
FT1-42	Módulo de soporte: sedes, franjas, precios e instructores (RNF-04)	Máximo Retamoso	Sprint 2	Listo
FT1-43	Scripts SQL: esquema base y trigger de auditoría	Martín Zanandrea	Sprint 2	Listo
FT1-50	Crear nuevo rol Gerente de Sede	Máximo Retamoso	Sprint 4	Listo
FT1-51	Subtarea: Endpoints	Martín Zanandrea	Sprint 4	Listo
FT1-52	Subtarea: Modificar BD	Teo Ava	Sprint 4	Listo
FT1-53	Subtarea: Nuevas pantallas	Máximo Retamoso	Sprint 4	Listo
FT1-70	Primera versión del cobro en efectivo en mostrador	Teo Ava	Sprint 2	Listo
FT1-71	Corrección del límite de intentos de login	Teo Ava	Sprint 2	Listo
FT1-72	Alcance por sede fail-closed en membresías y pagos	Martín Zanandrea	Sprint 2	Listo
FT1-73	Clases como plantillas recurrentes (ADR-018)	Martín Zanandrea	Sprint 2	Listo
FT1-74	Paginación en los listados de la API	Martín Zanandrea	Sprint 2	Listo

FT1-3 — Unidad III — Patrones (Listo)

Tabla 39

FT1-3 — Unidad III — Patrones (Listo) (Planificación)

Clave	Historia o subtarea	Responsable	Sprint	Estado
FT1-23	Patrón Strategy para precio dinámico de canchas (RF-11)	Martín Zanandrea	Sprint 3	Listo

Clave	Historia o subtarea	Responsable	Sprint	Estado
FT1-24	Patrón Observer para lista de espera (RF-08)	Martín Zanandrea	Sprint 3	Listo
FT1-25	Patrón Repository con control de concurrencia (RN-02)	Martín Zanandrea	Sprint 3	Listo
FT1-26	Justificación escrita de los patrones de diseño	Máximo Retamoso	Sprint 3	Listo
FT1-75	Cambio de contraseña	Martín Zanandrea	Sprint 3	Listo

FT1-4 — Unidad IV — Componentes (Listo)

Tabla 40

FT1-4 — Unidad IV — Componentes (Listo) (Planificación)

Clave	Historia o subtarea	Responsable	Sprint	Estado
FT1-28	Estado global y comunicación entre componentes	Teo Ava	Sprint 4	Listo
FT1-32	Reorganizar componentes con Atomic Design	Máximo Retamoso	Sprint 4	Listo
FT1-34	Capa de servicio frontend (consumo de la API)	Martín Zanandrea	Sprint 4	Listo
FT1-45	Pantallas de administración (Gerente y Recepcionista)	Martín Zanandrea	Sprint 4	Listo
FT1-47	Pantallas del socio/cliente	Teo Ava	Sprint 4	Listo
FT1-48	Pantallas del home/inicio	Máximo Retamoso	Sprint 4	Listo
FT1-49	Componentes base compartidos (ui.tsx)	Martín Zanandrea	Sprint 4	Listo
FT1-76	Panel de administración: sedes, usuarios y reportes	Teo Ava	Sprint 4	Listo
FT1-77	Gestión de reservas de clases y canchas para el gerente	Teo Ava	Sprint 4	Listo
FT1-78	Verificación de email obligatoria (RF-01)	Martín Zanandrea	Sprint 4	Listo
FT1-79	Catálogo de tipos de cancha gestionable (RF-09)	Martín Zanandrea	Sprint 4	Listo
FT1-80	Paso de Externo a Socio al pagar la membresía (ADR-020)	Martín Zanandrea	Sprint 4	Listo
FT1-81	Sesión sincronizada entre pestañas	Martín Zanandrea	Sprint 4	Listo
FT1-82	Cobro del recepcionista, personal y clientes	Martín Zanandrea	Sprint 4	Listo
FT1-83	Notificaciones por panel y email, historial de pagos y comprobantes	Martín Zanandrea	Sprint 4	Listo
FT1-84	Correo con proveedor abstracto (Brevo)	Martín Zanandrea	Sprint 4	Listo

Clave	Historia o subtarea	Responsable	Sprint	Estado
FT1-85	Vigencia de membresías por fechas (ADR-022)	Martín Zanandrea	Sprint 4	Listo

FT1-5 — Unidad V — Testing y CI/CD (Por hacer)

Tabla 41

FT1-5 — Unidad V — Testing y CI/CD (Por hacer) (Planificación)

Clave	Historia o subtarea	Responsable	Sprint	Estado
FT1-30	Testing y TDD (pruebas unitarias e integración)	Máximo Retamoso	Sprint 5	Listo
FT1-57	Subtarea: Pruebas unitarias del acceso por QR y RN01 (duplicada de FT1-58)	Sin asignar	Sin sprint	Por hacer
FT1-31	Pipeline CI/CD	Martín Zanandrea	Sprint 5	Listo
FT1-54	Pruebas de uso	Teo Ava	Sprint 5	Listo
FT1-55	Pruebas de seguridad	Martín Zanandrea	Sprint 5	En curso
FT1-56	Reporte de pruebas	Máximo Retamoso	Sprint 5	Listo
FT1-86	Escaneo de secretos del historial y gitleaks en el CI	Máximo Retamoso	Sprint 5	Listo
FT1-87	CI con Node 22	Martín Zanandrea	Sprint 5	Listo

FT1-6 — Unidad VI — Móvil (Por hacer)

Tabla 42

FT1-6 — Unidad VI — Móvil (Por hacer) (Planificación)

Clave	Historia o subtarea	Responsable	Sprint	Estado
FT1-88	Portal como PWA instalable (ADR-011)	Martín Zanandrea	Sprint 6	Listo
FT1-89	Notificaciones push con VAPID	Martín Zanandrea	Sprint 6	Listo
FT1-90	Estados sin conexión en el cliente	Martín Zanandrea	Sprint 6	Listo
FT1-91	Validación del QR sin conexión en el terminal de sede (RNF-01)	Martín Zanandrea	Sprint 7	En curso
FT1-105	Subtarea: Claves ES256 del QR y verificación dual (HS256/ES256)	Martín Zanandrea	Sprint 7	Listo
FT1-106	Subtarea: Migración 012: columnas de acceso offline y tabla de eventos	Martín Zanandrea	Sprint 7	Listo

Clave	Historia o subtarea	Responsable	Sprint	Estado
FT1-107	Subtarea: Endpoints de clave pública y snapshot de la sede	Martín Zanandrea	Sprint 7	Listo
FT1-108	Subtarea: Endpoint de sincronización de ingresos offline	Martín Zanandrea	Sprint 7	Listo
FT1-109	Subtarea: Terminal: réplica local, verificación del QR y cola de ingresos	Martín Zanandrea	Sprint 7	Listo
FT1-110	Subtarea: Terminal: integración en la pantalla de control de acceso	Martín Zanandrea	Sprint 7	Listo
FT1-111	Subtarea: Documentación del modo offline (ADR-003, ADR-007, secuencia y manual)	Martín Zanandrea	Sprint 7	Por hacer

FT1-7 — Cierre — Demo Day (Por hacer)

Tabla 43

FT1-7 — Cierre — Demo Day (Por hacer) (Planificación)

Clave	Historia o subtarea	Responsable	Sprint	Estado
FT1-92	Ingreso manual por DNI	Martín Zanandrea	Sprint 7	Listo
FT1-93	Reestructura a backend/frontend	Martín Zanandrea	Sprint 7	Listo
FT1-94	Contrato OpenAPI exportado (swagger.json)	Martín Zanandrea	Sprint 7	Listo
FT1-95	Migraciones reproducibles con reversión	Máximo Retamoso	Sprint 7	Listo
FT1-96	Datos de prueba con 25 sedes	Máximo Retamoso	Sprint 7	Listo
FT1-97	Docker compose y almacenamiento local de archivos	Máximo Retamoso	Sprint 7	Listo
FT1-98	Los diez ADR obligatorios en la plantilla del Anexo B	Máximo Retamoso	Sprint 7	Listo
FT1-99	Diagramas C4, DER, despliegue y pipeline	Máximo Retamoso	Sprint 7	Listo
FT1-100	Corrección de la carga de clases (429 con 25 sedes)	Máximo Retamoso	Sprint 7	Listo
FT1-101	Plazos de cancelación iguales a los del servidor	Máximo Retamoso	Sprint 7	Listo
FT1-102	Manuales de usuario, de instalación y técnico	Máximo Retamoso	Sprint 7	Listo
FT1-103	README con los seis puntos de la cátedra	Máximo Retamoso	Sprint 7	Listo
FT1-104	Integración con la reestructura	Máximo Retamoso	Sprint 7	Listo
FT1-112	Planificación del proyecto: exportar Jira y armar Planificacion.pdf	Martín Zanandrea	Sprint 7	Por hacer

Clave	Historia o subtarea	Responsable	Sprint	Estado
FT1-113	Bitácora individual de Martín Zanandrea	Martín Zanandrea	Sprint 7	Por hacer
FT1-114	Bitácora individual de Teo Ava	Teo Ava	Sprint 7	Por hacer
FT1-115	Bitácora individual de Máximo Retamoso: completar y verificar hashes	Máximo Retamoso	Sprint 7	Listo
FT1-116	Diagramas faltantes: clases, secuencia, estados y actividad	Máximo Retamoso	Sprint 7	Por hacer
FT1-117	Evidencia de requisitos no funcionales en docs/rnf	Máximo Retamoso	Sprint 7	Por hacer
FT1-118	Verificar docker compose up desde cero y agregar smoke test en el CI	Máximo Retamoso	Sprint 7	Por hacer
FT1-120	Cierre de entrega: checklist del Anexo G, tag v1.0-entrega y subida a Moodle	Martín Zanandrea	Sprint 7	Por hacer
FT1-121	Restaurar Supabase, verificar el backend en Render y monitor de actividad	Martín Zanandrea	Sprint 7	Por hacer
FT1-122	Auditoría (@Auditable) de acciones sensibles	Martín Zanandrea	Sprint 7	Por hacer
FT1-123	Actualizar dependencias vulnerables del backend (npm audit)	Martín Zanandrea	Sprint 7	Por hacer
FT1-124	Actualizar dependencias vulnerables del frontend (npm audit)	Teo Ava	Sprint 7	Por hacer
FT1-125	CI: ejecutar las pruebas del frontend (Vitest)	Máximo Retamoso	Sprint 7	Por hacer
FT1-126	Protección de main con checks obligatorios y colaboradora de solo lectura para la cátedra	Martín Zanandrea	Sprint 7	Por hacer
FT1-127	README: roles, declaración de IA y bibliotecas, credenciales del seed	Máximo Retamoso	Sprint 7	Por hacer
FT1-128	Actualizar documentación obsoleta (manual de instalación, informe histórico)	Máximo Retamoso	Sprint 7	Por hacer
FT1-129	Segunda ronda de auditoría contra el Anexo G	Máximo Retamoso	Sprint 7	Por hacer
FT1-130	Prueba de punta a punta del modo offline en producción y su evidencia	Martín Zanandrea	Sprint 7	Por hacer
FT1-131	Estructura del frontend frente al punto 4: verificar y documentar	Teo Ava	Sprint 7	Por hacer
FT1-132	Lint: errores del frontend y saltos de línea del backend	Teo Ava	Sprint 7	Por hacer
FT1-133	Preparación de la defensa: demo, plan B y escenario de canchas de tenis	Martín Zanandrea	Sin sprint	Por hacer

Historias sin épica asignada

Tabla 44

Historias sin épica asignada (Planificación)

Clave	Historia	Responsable	Sprint	Estado
FT1-58	Pruebas unitarias del acceso por QR y RN01	Teo Ava	Sprint 5	Listo
FT1-59	Pruebas de integración contra PostgreSQL real (RN01 y RN02)	Máximo Retamoso	Sprint 5	Listo
FT1-60	Pruebas de autenticación y estrategia JWT	Martín Zanandrea	Sprint 5	Por hacer
FT1-61	Checklist de pruebas en producción por rol	Máximo Retamoso	Sprint 5	Listo
FT1-62	Reporte de QA y corrección de hallazgos	Máximo Retamoso	Sprint 5	Listo
FT1-63	Auditoría de roles y alcance por sede (prueba automática)	Martín Zanandrea	Sprint 5	Listo
FT1-64	Corregir error 500 en ingresos simultáneos y endurecer el QR	Máximo Retamoso	Sprint 5	Listo
FT1-65	Informe de auditoría del testing y cobertura	Teo Ava	Sprint 5	Listo
FT1-66	Informe de cobertura generado en cada ejecución del CI	Martín Zanandrea	Sprint 5	Listo
FT1-67	Workflow de GitHub Actions (tsc, pruebas, build)	Martín Zanandrea	Sprint 5	Listo
FT1-68	Despliegue del backend condicionado al CI (Render)	Martín Zanandrea	Sprint 5	Listo
FT1-69	Pruebas unitarias con Vitest	Martín Zanandrea	Sprint 5	Por hacer

5. Cronograma

Tabla 45

Cronograma (Planificación)

Sprint	Fechas (Jira)	Objetivo	Entregable de cierre	Estado en Jira
Sprint 1	13/08 – 27/08	Arquitectura base: monolito modular, PostgreSQL y C4 en tres niveles	Documento de arquitectura: C4 y ADR fundacionales	Cerrado
Sprint 2	03/09 – 17/09	Scaffolding NestJS + TypeORM, esquema de base de datos y primera API	API base de los módulos M1 a M5 con Swagger	Cerrado
Sprint 3	15/09 – 24/09	Patrones Strategy, Observer y Repository con control de concurrencia	Patrones implementados y justificados	Cerrado

Sprint	Fechas (Jira)	Objetivo	Entregable de cierre	Estado en Jira
Sprint 4	15/09 – 08/10	Frontend con componentes, estado global y pantallas por rol	Portal del socio y consola de administración	Cerrado
Sprint 5	09/10 – 22/10	Pruebas unitarias e integración, CI/CD y Docker	Pipeline operativo, cobertura sobre el umbral, compose	Activo
Sprint 6	23/10 – 05/11	Acceso móvil sin conexión (PWA)	Portal instalable y notificaciones push	Futuro en Jira
Sprint 7	06/11 – 19/11	Integración final y cierre (acceso offline, reestructura, documentación)	Repositorio unificado, manuales, ADR y diagramas	Futuro en Jira
Sprint 8	23/11 – 29/11	Demo Day y defensa técnica	Demo funcional y defensa	Futuro en Jira

Capacidad comprometida: el tablero no registra capacidad ni horas. Las fechas de los sprints 5 a 8 se ordenaron el 09/11/2026 para que no se superpongan; las de los sprints 1 a 4 son las registradas al abrirlos, y el sprint 1 figura dos veces en Jira (identificadores 1 y 11, mismo contenido). La semana 16 (23/11 a 29/11) corresponde al Demo Day.

Figura 1. Diagrama de Gantt: ver Planificacion.pdf (la figura se genera en el PDF).

6. Asignación y carga

Cantidad de tarjetas (historias y subtareas, sin épicas) por integrante y sprint, y estado actual. El tablero no registra horas, por lo que la carga se mide en tarjetas.

Tabla 46

Asignación y carga (Planificación)

Integrante	S1	S2	S3	S4	S5	S6	S7	Sin sprint	Total
Martín Zanandrea	1	11	4	12	9	3	19	1	60
Teo Ava	1	6	0	5	3	0	4	0	19
Máximo Retamoso	2	5	1	4	7	0	18	0	37
Sin asignar	0	0	0	0	0	0	0	1	1

Desbalance visible: de 117 tarjetas sin contar épicas, Martín Zanandrea tiene 60, Máximo Retamoso 37, Teo Ava 19 y 1 no tienen responsable.

Tabla 47

Asignación y carga (Planificación)

Integrante	Listo	En curso	Por hacer
Martín Zanandrea	46	2	12
Teo Ava	15	0	4

Integrante	Listo	En curso	Por hacer
Máximo Retamoso	30	0	7

7. Matriz de trazabilidad

Requisito, historia que lo implementa (según el título de la tarjeta de Jira) y prueba que lo verifica. El detalle por archivo de prueba está en docs/PRUEBAS.md y en las carpetas test y *.spec.ts de cada módulo.

Tabla 48

Matriz de trazabilidad (Planificación)

Requisito	Historia	Prueba que lo verifica
RF-01	FT1-36, FT1-78	Pruebas unitarias del módulo de usuarios y verificación de email
RF-02	FT1-36, FT1-85	Pruebas unitarias de membresías y de vigencia (consumidores-vigencia)
RF-03	FT1-36, FT1-37	Pruebas de acceso multi-sede; integración de RN-01 contra PostgreSQL
RF-04	FT1-37, FT1-91, FT1-92	acceso.service.spec.ts, acceso-dni.spec.ts, acceso-offline.spec.ts, acceso-sincronizacion.spec.ts e integración de sincronización
RF-05	FT1-37	Pruebas unitarias del servicio de acceso (aforo por sede)
RF-06	FT1-38, FT1-73	Pruebas unitarias del módulo de clases
RF-07	FT1-38, FT1-101	Pruebas unitarias del módulo de clases (reserva y cancelación)
RF-08	FT1-24	Pruebas unitarias del patrón Observer de la lista de espera
RF-09	FT1-39, FT1-79	Pruebas unitarias del módulo de canchas
RF-10	FT1-39	Pruebas unitarias del módulo de canchas (reservas y cancelación)
RF-11	FT1-23	Pruebas unitarias de las estrategias de precio (PricingStrategy)
RF-12	FT1-39	Pruebas unitarias del módulo de canchas (mantenimiento)
RF-13	FT1-40, FT1-70, FT1-82	Pruebas unitarias del módulo de pagos
RF-14	FT1-40, FT1-83	Pruebas unitarias del módulo de pagos (comprobantes)
RN-01	FT1-37, FT1-59	acceso-rn01.int-spec.ts: integración contra PostgreSQL real (FT1-59)
RN-02	FT1-25, FT1-59	Pruebas de integración contra PostgreSQL real (FT1-59)
RN-03	FT1-23, FT1-85	Pruebas unitarias de precio y de vigencia de membresías
RNF-01	FT1-90, FT1-91, FT1-130	Pruebas de verificación ES256, cola y sincronización (backend y frontend); prueba de punta a punta en FT1-130
RNF-02	FT1-40	Pasarela simulada con token; no se persisten datos de tarjeta
RNF-03	FT1-43	Índices del esquema SQL; evidencia en docs/rnf (FT1-117)
RNF-04	FT1-42, FT1-95, FT1-96	Alta de sede por configuración en base de datos; datos de prueba con 25 sedes

8. Seguimiento por sprint

El tablero no registra estimaciones ni horas trabajadas, por lo que no puede calcularse el desvío en horas. Se muestra lo que sí existe: tarjetas asignadas al sprint (sin épicas) y su estado actual.

Tabla 49

Seguimiento por sprint (Planificación)

Sprint	Asignadas	Completadas	Trabajado (h)	Desvío	Observación
Sprint 1	4	4	No registrado	Sin desvío	Cerrado
Sprint 2	22	22	No registrado	Sin desvío	Cerrado
Sprint 3	5	5	No registrado	Sin desvío	Cerrado
Sprint 4	21	21	No registrado	Sin desvío	Cerrado
Sprint 5	19	16	No registrado	3 tarjetas sin completar	Activo; pendientes de pruebas de seguridad, de autenticación y de Vitest
Sprint 6	3	3	No registrado	Sin desvío	Alcance móvil sin conexión (PWA)
Sprint 7	41	20	No registrado	21 tarjetas sin completar	Concentra la integración y el cierre; incluye las tarjetas creadas el 09/11

Velocidad: los sprints 2 y 4 concentran la mayor cantidad de tarjetas completadas; el sprint 7 acumula el trabajo de integración y documentación de cierre.

9. Registro de riesgos

Registro elaborado el 09/11/2026 a partir de lo ocurrido; no se llevó actualizado durante el cuatrimestre.

Tabla 50

Registro de riesgos (Planificación)

Riesgo	Prob.	Impacto	Mitigación	Qué ocurrió
Concurrencia en reservas se resuelve tarde	Media	Alto	Patrón Repository con control de concurrencia en el sprint 3 (FT1-25) y pruebas contra PostgreSQL real	Se implementó en el sprint 3 y se verificó con pruebas de integración en el sprint 5 (FT1-59).
Concentración de la carga en pocos integrantes	Alta	Alto	Repartir tarjetas por módulo	Se materializó: ver sección 6. Un integrante concentra la mayor parte de las tarjetas.

Riesgo	Prob.	Impacto	Mitigación	Qué ocurrió
Tablero y planificación no se sostienen durante el cuatrimestre	Alta	Alto	Actualizar Jira al cierre de cada sprint	Se materializó: 48 tarjetas se crearon de forma retroactiva para registrar trabajo ya hecho (13 el 29/10 y 35 el 09/11 a las 17:58); otras 29 del 09/11 son planificación de cierre y brechas de la auditoría. No hay estimaciones ni horas.
Acceso sin conexión (RNF-01) no se prueba de punta a punta	Alta	Alto	Diseño con QR firmado ES256, réplica local, cola y sincronización; desglose en FT1-105 a FT1-111	La implementación se completó el 09/11. La prueba de punta a punta en producción está planificada en FT1-130.
Vulnerabilidades de seguridad en el acceso por QR	Media	Alto	Pruebas de seguridad y endurecimiento del QR (FT1-64)	Se corrigió el error 500 en ingresos simultáneos y el uso del QR como token de sesión; se rechaza el payload de tipo qr-acceso en la estrategia JWT.
Cambios de versión rompen el CI	Media	Medio	Integración continua con pruebas y compilación	El CI falló porque supabase-js exige WebSocket nativo (Node 22); se actualizó el CI, los Dockerfiles y engines a Node 22 (FT1-87).
Pérdida de conexión del backend desplegado con la base de datos gratuita	Media	Alto	Despliegue condicionado al CI y verificación manual en producción	El backend en Render dejó de conectarse a Supabase. Se verificó el proyecto el 09/11 y el despliegue volvió a funcionar; se creó FT1-121 para un monitor de actividad.
Esquema de producción divergente de las migraciones	Media	Medio	Migraciones con reversión y job de integración en el CI	La tabla schema_migraciones de producción estaba vacía; la migración 012 se aplicó a mano dentro de una transacción el 09/11.

10. Análisis de desvíos

1. ¿Dónde se estimó mal y por qué? No se registraron estimaciones, de modo que el desvío no puede medirse en horas; ese es el primer desvío. Los datos del tablero muestran qué no se había visto: la carga documental (ADR, diagramas, manuales, bitácoras y planificación) quedó para el sprint de cierre, en lugar de producirse sprint a sprint; el sprint 7 acumula 42 tarjetas frente a 3 del sprint 6; y la carga no estuvo balanceada: de 117 tarjetas, 60 son de Martín Zanandrea, 37 de Máximo Retamoso y 19 de Teo Ava.
2. ¿Qué riesgo se materializó y cómo se absorbió? Se materializaron la concentración de carga y la planificación no sostenida, esta última sin absorber: el tablero se completó al cierre y su estado se declara en este documento. Se absorbieron el fallo del CI por

la versión de Node (se subió a Node 22), la pérdida de conexión del backend con la base (se verificó el proyecto y se abrió un monitor) y las fallas del acceso por QR (se endureció y se reescribió la validación con firma ES256 para el modo sin conexión).

3. ¿Qué se replanificaría si el proyecto empezara de nuevo el lunes? Se mantendría el tablero vivo desde el sprint 1, con estimación en puntos y horas registradas por tarjeta y fecha de cierre. Se escribirían los ADR, los diagramas y las bitácoras al cierre de cada sprint y no al final. Se repartiría la carga con un tope visible por integrante y revisión en cada sprint,. Se priorizaría el acceso sin conexión (RNF-01) y su prueba de punta a punta antes del sprint de integración, y se instalaría desde el principio un monitor y un respaldo de la infraestructura.

9. Bitácoras individuales de los integrantes

Cada integrante lleva una bitácora con la plantilla del Anexo C de los lineamientos (UNER, 2026b). Los hashes, fechas y mensajes provienen del historial de git; las horas son estimaciones, con la regla de cálculo declarada en cada bitácora.

Bitácora — Zanandrea, Martín — Rol: Backend lead

Bitácora individual según el mismo formato que `docs/bitacoras/LOG-retamoso.md` (plantilla del Anexo C citada allí). Los hashes, fechas y mensajes provienen de `git log` real. La tarjeta se indica cuando figura en el mensaje del commit o cuando `docs/bitacoras/LOG-retamoso.md` o `docs/LOG.md` la asocian a ese hash; si no, se dice "Sin tarjeta citada en el commit". Los sprints y sus fechas son los de `docs/bitacoras/LOG-retamoso.md`.

Las horas son estimaciones calculadas a partir de los timestamps de los commits. Regla: por cada fecha, se ordenan los commits de Martín (sin merges) por hora local; se suman los intervalos entre commits consecutivos de 90 minutos o menos; se suman 30 minutos por cada commit como tiempo previo de trabajo; se redondea a media hora, con mínimo 0,5 h. En las fechas con varias entradas (2026-11-09), cada intervalo se asigna a la entrada del commit posterior. `a566405` no se cuenta porque repite el mensaje y la hora de `3abeccf`. Las horas de cada entrada se calculan solo con los commits de esa fecha, aunque la entrada cite commits de otros días.

Usuario de Git: `martin` y `Martin Zanandrea` (martinzanandrea09@gmail.com). Ambas variantes de nombre usan el mismo correo.

El README del repositorio no asigna el código de rol (P1 a P4) a ningún integrante, por eso el encabezado indica solo el rol.

Sprint 1 — Arquitectura — Unidad I — 2026-08-13 al 2026-08-27

2026-08-21 · 4,5 h

Tarjeta(s) Jira: según `docs/bitacoras/LOG-retamoso.md` (commits "del equipo, relacionados con la tarea"): `7862d54` con FT1-37, FT1-40 y FT1-42; `5552f69` y `cdca2b6` con FT1-26; `e22cde0` con FT1-9 y FT1-16. `f3cba13`, `1c30c96` y `06e2be4`: sin tarjeta citada en el commit.

Actividad

Construcción del backend con NestJS y TypeORM: creación de módulos y entidades (`7862d54`, 2026-08-15), módulos de usuarios, clases, instructores y membresías (`5552f69`, 2026-08-20), lógica de negocio de canchas (`cdca2b6`, 2026-08-20), pasarela de pagos, pago en efectivo y comprobantes (`06e2be4`, 2026-08-21) y cierre y despliegue del backend (`f3cba13`, `1c30c96`, 2026-08-21). Según `docs/LOG.md`, 13 módulos con Swagger documentado.

Decisión

Monolito modular con NestJS y PostgreSQL (ADR-0002, `e22cde0`, 2026-08-24), motivado en `docs/LOG.md` por la necesidad de transacciones ACID para la regla RN-02 (anti-sobreventa de canchas).

Problema o bloqueo

Según `docs/LOG.md`, el límite de peticiones global estaba demasiado estricto (20 por minuto) y producía errores 429 en el uso normal de desarrollo; se ajustó a 200 por minuto, manteniendo 5 por minuto en el login. Después, en `14ed42d` (2026-09-07), un comentario en `auth.controller.ts` deja constancia de que el límite de intentos del login no se aplicaba: el endpoint aceptaba POST sin límite y sospechaba de la versión de NestJS o de TypeScript. El commit no registra la solución.

Commits

- `7862d54`, `5552f69`, `cdca2b6`, `06e2be4`, `f3cba13`, `1c30c96`, `e22cde0`, `14ed42d`

Sprint 2 — Frameworks — Unidad II — 2026-09-03 al 2026-09-17

2026-09-08 · 1 h

Tarjeta(s) Jira: Sin tarjeta citada en el commit (`cba51e9`)

Actividad

Corrección de seguridad de alcance por sede (`cba51e9`): `membresias.service.ts` y `pagos.service.ts` dejan de devolver datos de todas las sedes a un recepcionista sin `sedeId` y responden 403 explícito. Se agrega la pantalla `SinSedeAsignada.tsx` con redirección desde `ProtectedRoute`, se construye `/admin/membresias` con filtros por estado y sede (antes era un enlace roto), se ajusta el ADR de alcance por sede y se versionan en el repositorio el esquema de base de datos y el trigger de auditoría.

Decisión

Denegar por defecto (fail-closed) cuando el usuario no tiene sede asignada, en lugar de mostrar todas las sedes.

Problema o bloqueo

Dos servicios (membresías y opciones de cobro en efectivo) tenían el mismo patrón fail-open: ante un recepcionista sin sede asignada devolvían datos de todas las sedes en vez de ninguno (`docs/LOG.md` y mensaje de `cba51e9`). Se corrigió en ambos con el mismo criterio. Además, el esquema de la base y el trigger de auditoría solo existían aplicados en Supabase, sin respaldo en el repositorio; el mismo commit los versionó.

Commits

- `cba51e9`

Sprint 4 — Componentes — Unidad IV — 2026-09-15 al 2026-10-08

2026-09-30 · 4 h

Tarjeta(s) Jira: según `docs/bitacoras/LOG-retamoso.md`, `f4bbfdc` se relaciona con FT1-37. Los commits `a846741`, `4f3a75a`, `f55e1c9` y `70511b0`: sin tarjeta citada en el commit.

Actividad

Verificación de correo obligatoria antes de poder iniciar sesión, con reenvío limitado por tasa y mensaje distinguible en el login; el chequeo de membresía activa para socios se movió del registro al login. Rediseño del área de Socio (dashboard, "Mi cuenta", endpoint `PATCH /usuarios/:id/foto`), QR de ingreso con código corto de 6 dígitos, lector de QR con cámara y correcciones previas a producción (`f4bbfdc`). Ese mismo día: `a846741` (atributos de la cookie), `4f3a75a` (`vercel.json`) y `f55e1c9` y `70511b0` (corrección visual de `MiQr.tsx`).

Decisión

Mover el chequeo de membresía del registro al login, porque el registro ya no inicia sesión de inmediato. Para la condición de carrera del último cupo de una clase, un repositorio con transacción y bloqueo pesimista, el mismo patrón que ya se usaba para canchas.

Problema o bloqueo

El mensaje de `f4bbfdc` registra cinco problemas y su solución: una regresión en el chequeo de membresía de los socios al dejar de iniciar sesión el registro; una condición de carrera real al reservar el último cupo de una clase con dos solicitudes simultáneas (se resolvió con transacción y bloqueo pesimista); una escalación de privilegios en `PATCH /usuarios/:id`, que aceptaba `tipoActor`, `sedeId` y `dni` en el cuerpo (se cerró y el DNI pasó a un endpoint dedicado solo para el Gerente); la ausencia de timeout en Axios, que causaba carga infinita ante fallos de red; y la cookie JWT, que en producción exige `secure` y `sameSite: none` por usar dominios distintos (Vercel y Render) y en desarrollo `lax` sin `secure`. En `a846741` los atributos de la cookie quedaron en un solo lugar (`opcionesCookie`), usado por el login y el logout. `4f3a75a` agregó a `vercel.json` un rewrite de `/api` hacia el backend en Render. `f55e1c9` y `70511b0` corrigen la pantalla del QR; los mensajes no detallan la causa.

Commits

- `f4bbfdc`, `a846741`, `4f3a75a`, `f55e1c9`, `70511b0`

Sprint 5 — Testing y CI/CD — Unidad V — 2026-10-29 al 2026-11-12

2026-10-29 · 1 h

Tarjeta(s) Jira: Sin tarjeta citada en el commit (`ac9c961`, `a0c3e5b`)

Actividad

Pruebas unitarias del servicio de acceso (`ac9c961`): generación y validación de QR por token y por código corto, regla RN-01 (un segundo ingreso con sesión abierta responde 409), aforo, alcance por sede y egreso; informe `docs/testing/informe-auditoria-testing.md`. Pipeline de GitHub Actions (`a0c3e5b`): instalación, verificación de tipos, pruebas con informe de cobertura y compilación del servidor, y compilación del cliente, en cada push y pull request a `main`.

Decisión

Documentar con pruebas dos comportamientos pendientes de corregir, en lugar de ocultarlos (mensaje de `ac9c961`): el error de índice único (23505) sin traducir a 409 y el token QR reutilizable dentro de su vigencia.

Problema o bloqueo

Esas dos fallas son el hallazgo de la entrada. Según `docs/bitacoras/LOG-retamoso.md` (entrada del 2026-11-09, FT1-59 y FT1-64), el token de un solo uso y la traducción del 23505 a 409 los aplicó Máximo en `009914b` y `c2d8e0e`. El informe de testing de la misma fecha señala además que el servicio de acceso tenía cobertura baja y que ninguna prueba usaba base de datos real, y que no existía CI hasta `a0c3e5b`.

Commits

- `ac9c961`, `a0c3e5b`
- Del equipo, relacionados con la tarea: `009914b`, `c2d8e0e`

2026-11-09 (18:18) · 2 h

Tarjeta(s) Jira: FT1-64 y FT1-87 (citadas en `3abeccf`). `cd24628`: sin tarjeta citada en el commit (cita `[FT1-nn]`). Sprint: `docs/bitacoras/LOG-retamoso.md` ubica FT1-64 en el sprint 5.

Actividad

Endurecimiento del acceso y CI (`3abeccf`): `JwtStrategy` rechaza los tokens de tipo `qr-acceso` como sesión (`jwt.strategy.ts`, con 46 líneas de pruebas nuevas en `jwt.strategy.spec.ts`), el código corto usa `randomInt` de `crypto` en `acceso.service.ts`, y Node 22 en `ci.yml` y en los Dockerfiles de `backend/` y `frontend/`. Antes, `cd24628` había subido Node a 22 en la CI por el requisito de supabase-js. `a566405` repite el mismo mensaje y la misma hora.

Decisión

Un QR firmado no debe poder usarse como cookie de sesión: se valida el tipo del token en la estrategia. El código corto pasa a generarse con un generador criptográfico.

Problema o bloqueo

El token del QR de acceso (60 s) se firma con el mismo secreto que la sesión, por lo que `JwtStrategy` lo aceptaba como sesión (comentario en `jwt.strategy.ts`); se rechaza el tipo `qr-acceso` sin consultar al usuario. El código corto se generaba con `Math.random`, que no es criptográfico (diff de `acceso.service.ts`). Las pruebas nuevas de `jwt.strategy.spec.ts` cubren el rechazo.

Commits

- 3abeccf, cd24628

Sprint 7 — Integración — Unidad Cierre — 2026-11-05 al 2026-11-19

Las entradas siguientes son del 2026-11-09. Ningún commit ni documento del repositorio asigna sprint a las tarjetas FT1-105 a FT1-110; se usa el sprint 7 por ser el que, según docs/bitacoras/LOG-retamoso.md, registra el trabajo de integración de ese día.

2026-11-09 (11:26) · 1,5 h

Tarjeta(s) Jira: Sin tarjeta citada en el commit (5885c68 y 1eec536 citan [FT1-nn] sin número). docs/bitacoras/LOG-retamoso.md relaciona 1eec536 con FT1-95 a FT1-104.

Actividad

Ingreso manual por DNI para recepción con previsualización y auditoría (5885c68): endpoints POST /acceso/buscar-dni y POST /acceso/validar-dni con validar-dni.dto.ts, modo DNI en ControlAcceso.tsx y acceso-dni.spec.ts (442 líneas). El mismo commit agrega docs/entregables/auditoria-estado-entrega.md. Después, 1eec536 adaptó la estructura del repositorio a los lineamientos de entrega (server/ a backend/, client/ a frontend/).

Decisión

Implementado en el código: el ingreso por DNI pasa por la misma lógica que el QR y el código corto (registrarIngresoValidado: sede, vigencia, aforo y RN-01) y queda auditado como INGRESO_MANUAL_DNI. La previsualización devuelve solo nombre, apellido, foto y estado de la membresía y no registra nada. Un DNI inexistente y el de alguien que no es socio dan el mismo 404, para que la búsqueda no revele datos de otros tipos de usuario. Los endpoints están limitados a 20 y 30 peticiones por minuto y solo para Recepcionista, Gerente de Sede y Gerente General.

Problema o bloqueo

docs/bitacoras/LOG-retamoso.md (entrada del 2026-11-09, FT1-59 y FT1-64) registra que esa versión chocó con el endurecimiento de Máximo del mismo día: se conservó la de Martín y se volvió a aplicar el endurecimiento sobre su código. Allí se vio que el 23505 se traducía a 409 solo en el ingreso por DNI (validarDni, como muestra el diff de 5885c68) y que con QR y con código seguía saliendo 500.

Commits

- `5885c68`, `1eec536`

2026-11-09 (19:46) · 2 h

Tarjeta(s) Jira: FT1-105, FT1-106 y FT1-107

Actividad

Base del acceso sin conexión en el backend (`9b9529e`): servicio `claves-qr.service.ts` que firma el QR con ES256 (clave P-256 por configuración, `kid` en el encabezado, con las variables documentadas sin valores en `.env.example`), migración `012-acceso-offline.sql` con su reversión `.down.sql`, entidad `evento-acceso-offline` y columnas nuevas en `control-acceso`, y endpoints de clave pública y de snapshot, con `swagger.json` regenerado. Pruebas en `acceso-offline.spec.ts` (554 líneas).

Decisión

Firma asimétrica del QR (ES256) en lugar de HS256 para que el terminal pueda verificarlo sin conexión con solo la clave pública. Sin clave configurada se sigue firmando con HS256 y el acceso offline no queda disponible (comentarios de `.env.example`). La migración es aditiva: solo agrega columnas con valor por defecto y una tabla, de modo que el código anterior sigue funcionando con ella aplicada. Los ADR-003 y ADR-007 en `docs/adr/` tratan la estrategia offline.

Problema o bloqueo

Al firmar, `@nestjs/jwt` prefiere el `secret` del módulo (HS256) sobre `privateKey`; para no tocar el `JwtModule`, la firma ES256 pasa el `secret` en cada llamada (comentario en `acceso.service.ts`). Además, en variables de entorno el PEM suele llegar en una línea con `\n` literales, y `claves-qr.service.ts` lo normaliza. Antes de este commit, `docs/entregables/auditoria-estado-entrega.md` registraba que el acceso offline no existía y que solo había columnas sin uso.

Commits

- `9b9529e`

2026-11-09 (19:58) · 0,5 h

Tarjeta(s) Jira: FT1-108

Actividad

Endpoint de sincronización de ingresos offline (`59218ce`): `POST /acceso/sincronizar` con `sincronizar-acceso.dto.ts`, procesamiento por evento en su propia transacción, resultados `CREADO`, `OBSERVADO`, `CONFLICTO` y `RECHAZADO`, y traducción de las violaciones de índice único (23505) mediante `restriccionViolada` en `db-errors.helper.ts`. Pruebas unitarias en `acceso-sincronizacion.spec.ts` (764 líneas) y de integración contra PostgreSQL en `acceso-sincronizacion.int-spec.ts` (212 líneas), con `swagger.json` actualizado.

Decisión

Una transacción por evento, y primero se registra el evento y después el ingreso, para que los índices únicos detecten el QR reusado (`jti`) y la sesión abierta. Idempotencia por identificador de evento del cliente: un evento ya procesado devuelve lo guardado.

Problema o bloqueo

Un 23505 aborta la transacción en curso, por lo que el resultado del evento se registra en una transacción nueva y corta; si en el medio otro proceso tomó el `jti`, se guarda sin él, y si ganó el mismo identificador de evento se devuelve lo de ese (comentarios de `acceso.service.ts`). Los rechazos y conflictos no reservan el `jti`, para que otro evento válido del mismo QR no quede bloqueado por un envío mal formado.

Commits

- `59218ce`

2026-11-09 (20:07 a 20:29) · 2 h

Tarjeta(s) Jira: FT1-109 (los tres commits) y FT1-69 (`458a14c`)

Actividad

Frontend del acceso sin conexión en tres commits. `458a14c`: verificación ES256 del QR en el navegador (`verificarQr.ts`), almacén IndexedDB (`almacenOffline.ts`) y configuración de Vitest con jsdom. `b3b10d5`: el arranque conserva la sesión ante un fallo de red o un servidor caído, con una copia local solo de los campos del usuario, sin tokens (`sesionLocal.ts`, `AuthContext.tsx`). `281446f`: descarga de snapshot (`snapshot.ts`), cola

de ingresos con sincronización por lotes (`colaIngresos.ts`) y hook `useModoOffline`. Cada módulo tiene su archivo de pruebas.

Decisión

La sesión sigue siendo la cookie httpOnly del backend: la copia local no guarda credenciales ni tokens y vence a las 24 horas (`VIGENCIA_COPIA_MS` en `sesionLocal.ts`), de modo que el terminal pueda abrir sin red sin debilitar la sesión real. Un 401 o 403 sigue cerrando la sesión.

Problema o bloqueo

Hasta `b3b10d5`, el arranque de `AuthContext.tsx` cerraba la sesión ante cualquier fallo al consultar al servidor (`.catch(() => aplicarUsuario(null))`), así que un terminal sin red no podía abrir la app. El commit distingue la falta de respuesta (red, timeout, 5xx) de una respuesta de sesión inválida (`decidirTrasFalloAlArrancar` en `clasificarError.ts`) y hace que el interceptor de Axios nunca cierre la sesión por un error de red. Con la sesión sin verificar se reintenta al volver la conexión y cada 30 s.

Commits

- `458a14c`, `b3b10d5`, `281446f`

2026-11-09 (20:35) · 0,5 h

Tarjeta(s) Jira: FT1-110

Actividad

Integración del modo sin conexión en la pantalla de control de acceso (`000cdc7`): `ControlAcceso.tsx` usa `useModoOffline`, descarga el snapshot al abrir y lo renueva si tiene más de una hora, valida el QR localmente sin conexión, muestra el aviso de modo sin conexión con "Sincronizar ahora", el resumen de la última sincronización y la insignia "Sin conexión". Las decisiones y los textos están en funciones puras en `decisionesPantalla.ts`, con 30 pruebas en `decisionesPantalla.test.ts`.

Decisión

Pasar a validación local solo ante falta de respuesta, timeout o error 5xx; un 4xx del servidor se muestra como rechazo real (comentarios de `ControlAcceso.tsx`). Ingreso por código, por DNI y egreso quedan deshabilitados sin conexión, con un texto que lo explica.

Problema o bloqueo

Antes de este commit, la pantalla bloqueaba toda validación sin conexión: mostraba "Sin conexión a internet" y el texto de que se requiere conexión (`docs/entregables/auditoria-estado-entrega.md`, informe de acceso offline). Sin bloqueos adicionales registrados en el historial.

Commits

- `000cdc7`

Trabajo previo o entre sprints

Las fechas siguientes caen fuera de las ventanas de sprint documentadas en `docs/bitacoras/LOG-retamoso.md`.

2026-08-29 · 1,5 h

Tarjeta(s) Jira: según `docs/bitacoras/LOG-retamoso.md` y `docs/LOG.md`, `1802fc4` se relaciona con FT1-22 (el commit no la cita). Fecha entre el sprint 1 (termina el 2026-08-27) y el sprint 2 (empieza el 2026-09-03).

Actividad

Sistema de auditoría (`1802fc4`): módulo con interceptor global `@Auditable` y trigger de Postgres sobre la tabla de pagos, que registra acciones sensibles en las 9 entidades principales. En el mismo commit: corrección del fallo de `GET /clases/:id/reservas` por falta de `relations`, prevención de membresías duplicadas tras un pago rechazado con `PATCH /membresias/:id/cancelar`, flujo de renovación (`POST /membresias/renovar`), indicador `simularRechazo` en la pasarela simulada para QA y manejo del duplicado de reserva de clase con `ConflictException`.

Decisión

Auditoría en dos capas: el interceptor aporta el contexto de la aplicación y el trigger cubre los cambios directos sobre pagos (ADR-017 en `docs/adr/`).

Problema o bloqueo

El mensaje de `1802fc4` registra: `GET /clases/:id/reservas` fallaba por falta de `relations` (la clase no venía poblada); reintentar tras un pago rechazado creaba membresías duplicadas; un duplicado de reserva de clase dejaba pasar el error crudo de Postgres; el

dashboard del socio no distinguía un error de red de "sin membresía activa"; y la pantalla de completar membresía navegaba como éxito con el pago rechazado. Todos se corrigieron en el mismo commit. Según `docs/bitacoras/LOG-retamoso.md` (FT1-22), el detalle JSON de la auditoría guardaba tokens, y se agregó redacción automática de secretos; el script `008-redactar-secretos-auditoria.sql` llegó en `26dfa17`.

Commits

- `1802fc4`

2026-10-09 · 1,5 h

Tarjeta(s) Jira: según `docs/bitacoras/LOG-retamoso.md`, `26dfa17` se relaciona con FT1-40 y FT1-42; según `docs/LOG.md` (entrada de Teo Ava), con FT1-38. El commit no cita tarjeta. Fecha posterior al sprint 4 (termina el 2026-10-08).

Actividad

Notificaciones y gestión (`26dfa17`): sistema de notificaciones por panel y correo con patrón Factory para los canales y Observer para los eventos, campanita para el Socio, sección de envíos para el Gerente General con vista previa, confirmación e historial, promoción de la lista de espera que ahora notifica, script `006-notificaciones.sql`, configuración de clases (edición, cambio de instructor, regeneración de fechas, baja con aviso) y cobro presencial de canchas por el Recepcionista (`POST /pagos/efectivo/cancha`) con precio calculado en el servidor. Ese mismo día, `4533a9f` eliminó un script auxiliar.

Decisión

Factory para los canales de notificación y Observer para los eventos; el precio del cobro presencial se calcula en el servidor (ADR-016).

Problema o bloqueo

El script `server/scripts/listar-endpoints.ts`, agregado en `26dfa17` para auditar los endpoints, alteraba la estructura de `dist` en el build; `4533a9f` lo eliminó. El mensaje de `26dfa17` declara además que las consultas SQL escritas a mano y los bloqueos reales no se probaron contra Postgres; el informe `docs/testing/informe-auditoria-testing.md` (2026-10-29) confirma que ninguna prueba usaba base de datos real.

Commits

- `26dfa17`, `4533a9f`

2026-10-12 · 1,5 h

Tarjeta(s) Jira: Sin tarjeta citada en el commit (`a72b7ef`, `ddeb86f`). Fecha entre el sprint 4 (termina el 2026-10-08) y el sprint 6 (empieza el 2026-10-22).

Actividad

Abstracción del transporte de correo (`a72b7ef`): interfaz `EnviadorCorreo` con `BrevoEnviador` (API HTTPS, timeout de 10 s) para producción y `GmailSmtpEnviador` para desarrollo, elegidos con `MAIL_PROVIDER`, validación de configuración al arrancar y logs que registran solo estado y código. Con tests de Brevo (éxito, 400/401, timeout, red) y de selección de proveedor. Luego, refactor de `NotificadorFactory` por registro de canales con el token `NOTIFICADORES` (`ddeb86f`), con tests de extensibilidad, de la fábrica y del grafo de inyección del módulo.

Decisión

Cambiar a una API HTTPS porque, según el mensaje de `a72b7ef`, Render bloquea el SMTP saliente. La fábrica deja de conocer clases concretas para que agregar un canal solo requiera una clase y su registro.

Problema o bloqueo

El envío de correo fallaba en producción con `ETIMEDOUT`: Render bloquea el SMTP saliente, de modo que la conexión por SMTP no llegaba al servidor de correo (mensaje de `a72b7ef`). Dos días antes, `27102e5` (2026-10-10) había agregado al log el código técnico del error (`err.code`, `err.responseCode`, `err.command` y la demora en caso de timeout) para distinguir un bloqueo de red, credenciales inválidas o variables faltantes en producción, sin registrar direcciones ni credenciales. La solución fue enviar por la API HTTPS de Brevo, con un timeout de 10 s (`BREVO_TIMEOUT_MS`) que aborta la llamada y falla con `ETIMEDOUT` sin exponer la clave de la API (prueba en `brevo.enviador.spec.ts`), y dejar el SMTP de Gmail solo para desarrollo.

Commits

- `a72b7ef`, `ddeb86f`, `27102e5`

2026-10-18 · 8 h

Tarjeta(s) Jira: Sin tarjeta citada en el commit (`73bbf51`, `ad8d386`, `8963659`, `65ecc2d`, `fadb12b`, `34a9ca9`). `8963659` figura en `docs/adr/ADR-011-pwa-en-lugar-de-app-nativa.md`, sin tarjeta. Fecha entre el sprint 4 (termina el 2026-10-08) y el sprint 6 (empieza el 2026-10-22).

Actividad

Vigencia de membresías por fechas (`73bbf51`, adaptada en el cliente por `ad8d386`): función central de vigencia usada por acceso, reservas, precios, pagos, clientes y reportes; una compra con membresía vigente queda como próxima y empieza al terminar la anterior (máximo una vigente y una próxima); `fecha_fin` exclusiva con fechas en hora de Argentina; alta con bloqueo de fila por usuario y error de exclusión de Postgres traducido a 409. Cliente como PWA instalable (`8963659`) con service worker que solo precachea archivos estáticos y deja las llamadas a la API siempre por red. Manejo de errores de carga y de falta de conexión (`65ecc2d`). Canal push con Web Push y claves VAPID (`fadb12b`, `34a9ca9`), deshabilitado si no hay claves configuradas.

Decisión

Service worker sin caché de la API y push opcional por configuración, según los mensajes de `8963659` y `fadb12b`. La vigencia se documenta en ADR-022.

Problema o bloqueo

El mensaje de `65ecc2d` registra el problema: una carga fallida se mostraba como "sin membresía" o "sin datos", y en la reserva de canchas, si fallaba la consulta de ocupación del día, se mostraban todos los horarios como libres. Se resolvió con un estado de error con motivo y botón "Reintentar", reintento automático al volver la conexión, un banner global de "Sin conexión a internet" y el bloqueo de la grilla de reserva cuando falla la ocupación. Mi QR no se muestra ni se regenera sin conexión. `73bbf51` resuelve a nivel de base la superposición de membresías con el bloqueo de fila y la traducción a 409 del error de exclusión de Postgres.

Commits

- `73bbf51`, `ad8d386`, `8963659`, `65ecc2d`, `fadb12b`, `34a9ca9`

Sprint 8 — Demo Day — Unidad Cierre — 2026-11-19 al 2026-12-03

Sprint todavía no iniciado.

Bitácora — Ava, Teo — Rol: Frontend

Bitácora individual con el mismo formato que `docs/bitacoras/LOG-zanandrea.md` y `docs/bitacoras/LOG-retamoso.md` (plantilla del Anexo C). Los hashes, fechas y mensajes provienen de `git log --author` real: son los commits de autoría de Teo Ava. Los sprints y sus fechas son los de `LOG-retamoso.md`.

Las horas son estimaciones calculadas a partir de los timestamps de los commits. Regla: por cada fecha, se ordenan los commits (sin merges) por hora local; se suman los intervalos entre commits consecutivos de 90 minutos o menos; se suman 30 minutos por cada commit como tiempo previo de trabajo; se redondea a media hora, con mínimo 0,5 h. La entrada de la madrugada del 6 al 7 de septiembre agrupa cuatro commits consecutivos que cruzan la medianoche.

Tarjetas. Ningún commit de Teo cita una tarjeta. Cuando una entrada indica una tarjeta "por coincidencia de título", la asociación sale de comparar el mensaje del commit con el título de la tarjeta en `docs/planificacion/PLANIFICACION.md`, y está **por confirmar** contra el tablero. `docs/LOG.md` asocia a Teo con otras tarjetas (por ejemplo FT1-10, FT1-19, FT1-21, FT1-28 y FT1-38) citando commits cuya autoría en git es de Martín; esas entradas no se repiten acá.

Usuario de Git: `Teo` (teoava3@gmail.com). Sus commits tocan el frontend y también archivos del backend (en ese momento las carpetas se llamaban `client/` y `server/`).

Sprint 1 — Arquitectura — Unidad I — 2026-08-13 al 2026-08-27

2026-08-26 · 0,5 h (estimación)

Tarjeta(s) Jira: Sin tarjeta citada en el commit. Por coincidencia de título: FT1-76 (Panel de administración: sedes, usuarios y reportes), por confirmar

Actividad

Sección de sedes y usuarios en el panel de administración (`64817da`): páginas `SedesPage.tsx` y `CrearStaff.tsx` con sus capas de servicio (`sedes.api.ts`,

`usuarios.api.ts`), ajuste de `Login.tsx` y de `AppRoutes.tsx`, y en el backend un DTO `assign-role.dto.ts` y el método `assignRole` del servicio de usuarios para asignar un rol.

Decisión

Sin decisión registrada en el mensaje del commit ni en su diff.

Problema o bloqueo

Sin bloqueos registrados en el historial.

Commits

- `64817da`

Sprint 2 — Frameworks — Unidad II — 2026-09-03 al 2026-09-17

2026-09-04 · 0,5 h (estimación)

Tarjeta(s) Jira: Sin tarjeta citada en el commit

Actividad

Cambios en las reservas de cancha por parte de los usuarios (`25718bf`): `ReservarCanchas.tsx` y `ReservarClases.tsx` en el frontend, y `reserva-cancha.service.ts` y `reservas-cancha.controller.ts` en el backend.

Decisión

El servicio de reserva de cancha rechaza la reserva de un horario que ya comenzó (diff de `reserva-cancha.service.ts`). El mensaje del commit no registra por qué.

Problema o bloqueo

Sin bloqueos registrados en el historial. El diff muestra que se agregaron las validaciones "No se puede reservar un horario que ya comenzó" y la comprobación de existencia de la cancha y del usuario.

Commits

- `25718bf`

2026-09-06 al 2026-09-07 (madrugada) · 3,5 h (estimación)

Tarjeta(s) Jira: Sin tarjeta citada en los commits. Por coincidencia de título, por confirmar: `782b39a` con FT1-70 (Primera versión del cobro en efectivo en mostrador); `e9e8d65` y `c564a90` con FT1-38 (M3 Clases Grupales); `9abc4b1` con FT1-71 (Corrección del límite de intentos de login)

Actividad

Sesión continua de cuatro commits, de las 23:53 a las 01:12:

- `782b39a` (23:53): cobro en efectivo en `CobrarEfectivo.tsx`, `pagos.api.ts`, `pagos.controller.ts` y `pagos.service.ts`.
- `e9e8d65` (00:01): cambios en las reservas de clases (`GestionReservasClases.tsx` y `reserva-clase.service.ts`).
- `c564a90` (00:12): cambios al crear clases con horario en `ClasesPage.tsx`.
- `9abc4b1` (01:12): corrección del límite de intentos de login en `auth.controller.ts`.

Decisión

Sin decisión registrada en los mensajes ni en los diffs.

Problema o bloqueo

El mensaje de `782b39a` deja constancia de que quedó pendiente: "Falta modificar la parte de Membresia" del cobro en efectivo. `9abc4b1` ("error throttle solucionando") quita del endpoint de login el `@SkipThrottle()` y el comentario con el que `14ed42d` (commit de otro integrante del 2026-09-07) había marcado que el límite de intentos del login no se aplicaba. Cómo se verificó la corrección no consta en el commit.

Commits

- `782b39a`, `e9e8d65`, `c564a90`, `9abc4b1`

2026-09-07 · 0,5 h (estimación)

Tarjeta(s) Jira: Sin tarjeta citada en el commit. `docs/LOG.md` lo asocia con FT1-14 (Documentación de la API con Swagger), que coincide con el título del tablero

Actividad

Documentación de la API con Swagger (`48c3add`): decoradores y descripciones en los controladores de acceso, administración, auditoría, autenticación, canchas y clases, y en sus DTO (`validar-qr.dto.ts`, `registrar-egreso.dto.ts`, `login.dto.ts`, `create-cancha.dto.ts`, entre otros).

Decisión

Según `docs/LOG.md`, Swagger en lugar de una colección de Postman suelta, porque vive con el código y no se desactualiza. El mensaje del commit no lo registra.

Problema o bloqueo

Sin bloqueos registrados en el historial.

Commits

- `48c3add`

Sprint 3 — Patrones — Unidad III — 2026-09-15 al 2026-09-24

Sin actividad propia registrada para este sprint. Los commits de Teo del 2026-09-23 se asignan al sprint 4, que también está en curso en esa fecha.

Sprint 4 — Componentes — Unidad IV — 2026-09-15 al 2026-10-08

2026-09-23 · 0,5 h (estimación)

Tarjeta(s) Jira: Sin tarjeta citada en el commit. Por coincidencia de título: FT1-77 (Gestión de reservas de clases y canchas para el gerente), por confirmar

Actividad

Arreglo en la gestión de reservas de clases (`df91a7a`): cambios en `GestionReservasClases.tsx` y en `CalendarioSede.tsx` (88 líneas agregadas y 25 quitadas).

Decisión

Sin decisión registrada en el mensaje del commit ni en su diff.

Problema o bloqueo

El mensaje habla de un "arreglo" en la pantalla, sin describir el defecto. Sin más detalle registrado en el historial.

Commits

- `df91a7a`

Sprint 5 — Testing y CI/CD — Unidad V — 2026-10-29 al 2026-11-12

Sin actividad propia registrada para este sprint.

Sprint 6 — Móvil — Unidad VI — 2026-10-22 al 2026-11-05

Sin actividad propia registrada para este sprint.

Sprint 7 — Integración — Unidad Cierre — 2026-11-05 al 2026-11-19

2026-11-09 · 0,5 h (estimación)

Tarjeta(s) Jira: Sin tarjeta citada en el commit. Sprint: no hay una tarjeta asociada; se usa el sprint 7 por la fecha

Actividad

Se agrega al repositorio el SQL de la estructura de la base de datos, esquema `public`, con fecha 9/11/2026 (`9f1e2e2`): un volcado de 1226 líneas en `fitzone-estructura-20261109.sql`, en la raíz del repositorio.

Decisión

Sin decisión registrada en el mensaje del commit. El archivo está en la raíz del repositorio y no en `docs/`.

Problema o bloqueo

Sin bloqueos registrados en el historial.

Commits

- `9f1e2e2`

Trabajo previo o entre sprints

Las fechas siguientes caen fuera de las ventanas de sprint documentadas en `docs/bitacoras/LOG-retamoso.md`.

2026-08-28 · 0,5 h (estimación)

Tarjeta(s) Jira: Sin tarjeta citada en el commit. Por coincidencia de título: FT1-77 (Gestión de reservas de clases y canchas para el gerente), por confirmar. Fecha entre el sprint 1 (termina el 2026-08-27) y el sprint 2 (empieza el 2026-09-03)

Actividad

Reservas y gestión de clases y canchas para el gerente (`7b33ea7`): páginas `ReservasPage.tsx`, `ClasesPage.tsx`, `CanchasPage.tsx` y `GerenteDashboard.tsx` del panel de administración, sus capas de servicio (`admin.api.ts`, `canchas.api.ts`, `clases.api.ts`, `instructores.api.ts`, `usuarios.api.ts`), `PersonalPage.tsx`, `AdminLayout.tsx` y `AppRoutes.tsx` (20 archivos, 768 líneas agregadas).

Decisión

Sin decisión registrada en el mensaje del commit ni en su diff.

Problema o bloqueo

Sin bloqueos registrados en el historial.

Commits

- `7b33ea7`

2026-08-29 · 0,5 h (estimación)

Tarjeta(s) Jira: Sin tarjeta citada en el commit. Por coincidencia de título: FT1-76 (Panel de administración: sedes, usuarios y reportes), por confirmar. Fecha entre el sprint 1 y el sprint 2

Actividad

Agrega el acceso a Reportes en el menú del panel de administración (`2e4c417`: una línea en `AdminLayout.tsx`).

Decisión

Sin decisión registrada en el mensaje del commit ni en su diff.

Problema o bloqueo

Sin bloqueos registrados en el historial.

Commits

- `2e4c417`

2026-10-14 · 0,5 h (estimación)

Tarjeta(s) Jira: Sin tarjeta citada en el commit. Fecha entre el sprint 4 (termina el 2026-10-08) y el sprint 6 (empieza el 2026-10-22)

Actividad

Completa las entradas de Teo en la bitácora grupal `docs/LOG.md` para las unidades I, II y IV (`f12acdb`: 39 líneas agregadas y 3 quitadas). Esas entradas citan commits cuya autoría en git es de otro integrante, razón por la cual esta bitácora individual se rehízo sobre los commits propios.

Decisión

Sin decisión registrada en el mensaje del commit.

Problema o bloqueo

Sin bloqueos registrados en el historial. Las unidades III, V y VI de `docs/LOG.md` quedaron como pendientes para Teo en ese archivo.

Commits

- `f12acdb`

Sprint 8 — Demo Day — Unidad Cierre — 2026-11-19 al 2026-12-03

Sprint todavía no iniciado.

Bitácora — Retamoso, Máximo — Rol P4

Bitácora individual según la plantilla del Anexo C. Los sprints, sus fechas y las tarjetas (FT1-nn) son los del tablero Jira del equipo. Las entradas de los sprints 1 al 8.

Usuario de Git: `maxi` (maxiretamosotrotta@gmail.com).

Sprint 1 — Arquitectura — Unidad I — 2026-08-13 al 2026-08-27

2026-08-24 · 3 h

Tarjeta(s) Jira: FT1-9 (2 pts) y FT1-16 (3 pts), ambas asignadas a MR

Actividad

Debate grupal sobre Monolito Modular vs Microservicios (FT1-9): sostuve transacciones ACID fuertes para RN-02 (anti-sobreventa de canchas) como criterio decisivo, con equipo de 4 y time-to-market como criterios secundarios.

Decisión

Monolito Modular con NestJS, documentado en ADR-0002 (FT1-16).

Problema o bloqueo

posiciones iniciales divididas en el equipo; se cerró con matriz de criterios (ACID, equipo, deploy) acordada entre los tres.

Commits

- Del equipo: `e22cde0`

Trabajo de diseño: el debate y el contenido del ADR-0002 se acordaron con el equipo por mensajes; el ADR lo integró Martín al repositorio.

Sprint 2 — Frameworks — Unidad II — 2026-09-03 al 2026-09-17

2026-09-02 · 1 h

Tarjeta(s) Jira: FT1-15 (1 pt); FT1-16 es del sprint 1

Actividad

Redacción de ADR-0001 GitHub como repositorio compartido (FT1-15) y de ADR-0002 Monolito Modular (FT1-16): ramas por integrante y versionado acoplado como costo aceptado.

Decisión

monorepo único cliente+servidor con formato Nygard de 5 campos.

Problema o bloqueo

numeración de ADRs con gaps tras varias iteraciones; se resolvió con renumeración basada en git log.

Commits

- Del equipo: `612d9ce`, `068e8b6`, `e22cde0`

2026-09-10 · 4 h

Tarjeta(s) Jira: FT1-37 (4 pts)

Actividad

Definición de las reglas del módulo de Gimnasio y Acceso (RF-04, RF-05, RN-01) junto con Martín: QR dinámico que vence a los 60 segundos, código corto de 6 dígitos de un solo uso como alternativa, control de aforo por sede y sesión única por usuario. Martín implementó el módulo; yo lo probé con cuentas de prueba (generación del QR, validación por código y aforo) y le reporté los ajustes necesarios.

Decisión

JWT firmado de corta vida con revalidación en el servidor, en vez de un QR estático que se puede fotografiar y compartir.

Problema o bloqueo

La ventana de 60 s entre que se genera el QR y se valida obligó a chequear la membresía dos veces, porque puede vencer en el medio. El aforo se resolvió contando las sesiones abiertas, con un índice parcial.

Commits

- Del equipo: `7862d54`, `23cf2ff`, `f4bbfdc`

2026-09-12 · 5 h

Tarjeta(s) Jira: FT1-40 (6 pts) y FT1-42 (5 pts)

Actividad

Definición con Martín del funcionamiento de los pagos (RF-13, RF-14) y de los módulos de soporte (sedes, franjas, precios e instructores): pasarela simulada que devuelve un token, membresía creada solo con el pago aprobado, paso de externo a socio y comprobante PDF por pago. Martín los implementó; yo los probé con cuentas de prueba (pago de membresía, pago de cancha, comprobante, alta de sedes y precios).

Decisión

Los montos se calculan siempre en el servidor, y las clases vienen incluidas en la membresía (RN-03).

Problema o bloqueo

El doble clic al pagar podía cobrar dos veces. Martín lo resolvió con un bloqueo de la fila y verificando si ya había un pago (el segundo intento recibe 409); en las pruebas verifiqué que no se duplica el cobro.

Commits

- Del equipo: `7862d54`, `13b0d95`, `f07fa62`, `26dfa17`

2026-09-14 · 3 h

Tarjeta(s) Jira: FT1-22 (5 pts)

Actividad

Redacción de ADR-0008 sistema de auditoría en dos capas (FT1-22): interceptor con @Auditable para contexto de app + trigger en Postgres sobre pagos para cubrir cambios directos, con redacción automática de secretos.

Decisión

dos capas complementarias (interceptor + trigger).

Problema o bloqueo

el detalle JSON guardaba tokens; se agregó redacción automática de secretos + migración.

Commits

- Del equipo: `1802fc4`, `068e8b6`

Sprint 3 — Patrones — Unidad III — 2026-09-15 al 2026-09-24

2026-09-18 · 3 h

Tarjeta(s) Jira: FT1-26 (4 pts)

Actividad

Definí con Martín cómo armar el documento de justificación de los tres patrones GoF: Strategy en el precio dinámico (RF-11), Observer en la lista de espera (RF-08) y Repository con bloqueo pesimista contra la sobreventa (RN-02). Para cada uno, qué problema resuelve, por qué se eligió frente a las alternativas y qué costo tiene. La redacción la hizo casi toda Martín.

Decisión

Documentar para cada patrón el porqué y la alternativa descartada, en lugar de solo describir la implementación.

Problema o bloqueo

Al repasar los patrones para justificarlos noté que la primera versión de Strategy no era polimórfica de verdad, porque el calculador conocía las clases concretas. Se lo planteé a Martín, que hizo el refactor a la forma canónica: arreglo de estrategias inyectado por el token PRICING_STRATEGIES y Repository con interfaz e implementación en memoria para las pruebas.

Commits

- Del equipo: `cdca2b6`, `5552f69`, `2c6c8d7`

Sprint 4 — Componentes — Unidad IV — 2026-09-15 al 2026-10-08

2026-10-05 · 2 h

Tarjeta(s) Jira: FT1-48

Actividad

Definí con Martín el contenido y el diseño de la landing pública multipágina (inicio, clases, canchas, sedes, nosotros, con navegación compartida), que programó él. Al probar la navegación encontré que al cambiar de ruta la página no volvía arriba, y lo arreglé con un componente global ScrollToTop.

Decisión

Un layout público único (PublicLayout) en vez de repetir la navegación en cada página, y un arreglo global del scroll en vez de parchear página por página.

Problema o bloqueo

React Router conserva el scroll entre rutas y el frontend no tenía ningún manejo de scroll. Lo diagnostiqué comparando el scroll más un clic contra una recarga de la página.

Commits

- `b3ec7c3`
- Del equipo: `9f46ceb`

2026-10-06 · 4 h

Tarjeta(s) Jira: FT1-32 (4 pts)

Actividad

Reorganización con Atomic Design (FT1-32): átomos (10 piezas, un archivo por componente), moléculas (12), organismos (layouts) con barrels index.ts + README del sistema; formatMoney a shared/utils; eliminado código muerto (ui.tsx partido, Donut duplicado, Reveal/Layout sin uso). Rama refactor/atomic-design, build limpio.

Decisión

un archivo por componente + barrels por nivel en vez de cajón único; mover sin cambiar comportamiento.

Problema o bloqueo

el build marcó dos errores tras el move (rutas relativas e import de barrel); se corrigieron y el build quedó limpio.

Commits

- `772b856`

2026-10-11 · 2 h

Tarjeta(s) Jira: FT1-50 (5 pts), subtarea FT1-53

Actividad

Discutí con Martín las pantallas del nuevo rol Gerente de Sede, que programó él. Las verifiqué con una cuenta de prueba: login, panel propio y las 10 pantallas del rol (personal,

canchas, clases, reservas, reportes, pagos, membresías, instructores y acceso), cargando los datos de su sede.

Decisión

Cerrar la tarea como Listo después de una verificación funcional por rol, y no solo revisando el código.

Problema o bloqueo

En la verificación encontré algunas cosas para corregir y se las pasé a Martín. También revisé el alcance por sede: el gerente de sede solo ve datos de su sede en personal, reservas y reportes.

Commits

- Del equipo: `13b0d95`, `f07fa62`

La tarjeta está en el sprint 4 aunque la verificación se cerró el 11/10, después de la fecha de fin del sprint.

Sprint 5 — Testing y CI/CD — Unidad V — 2026-10-29 al 2026-11-12

2026-10-13 · 4 h

Tarjeta(s) Jira: FT1-30 (6 pts), FT1-61, FT1-62 y FT1-56

Actividad

Pruebas manuales en producción (FT1-30/54/55: 20 ejecutadas, 2 parciales) con cuentas de prueba por rol, y reporte en docs/PRUEBAS.md con tablas por grupo, plantilla y hallazgos (FT1-56). Incluye registro con mail real verificado, compra en cuenta fresca, doble clic en cuenta propia sin duplicado y baja de clase TEST con aviso. Limpieza verificada en base: sin sesiones ni reservas abiertas.

Decisión

probar contra producción (como pide el plan) con cuentas de prueba, cancelando/cerrando todo lo creado en el acto; no ejecutar mail masivo por riesgo a socios reales.

Problema o bloqueo

sin clase casi llena no hay última plaza real (se probó carrera en paralelo sin duplicados); throttle de login confirmado al 6.º intento.

Commits

- `5371015`

2026-11-09 · 2 h

Tarjeta(s) Jira: FT1-59 y FT1-64

Actividad

Pruebas de integración contra PostgreSQL real para RN-01 y RN-02 (FT1-59): diez reservas simultáneas de la misma cancha confirman exactamente una, dos ingresos simultáneos del mismo socio en sedes distintas registran uno solo, y el cupo de una clase no se excede bajo concurrencia; umbral de cobertura en `package.json` y jobs de integración y Docker en el CI. Endurecimiento del QR (FT1-64): token de un solo uso con `jti` y la violación del índice de RN-01 (23505) devuelta como 409 en lugar de 500 en los tres tipos de ingreso.

Decisión

Probar la concurrencia contra PostgreSQL real y no solo con repositorios en memoria, porque los dobles no reproducen el SELECT … FOR UPDATE ni los índices únicos parciales. Para el QR de un solo uso, guardar los jti usados en memoria hasta que vencen (viven 60 segundos) en vez de una tabla, y traducir el error 23505 en la función común de registro del ingreso, así cubre QR, código y DNI.

Problema o bloqueo

Ese mismo día Martín subió el ingreso por DNI y chocaba con mi versión: me quedé con la suya y volví a aplicar solo el endurecimiento sobre su código. Ahí apareció que su versión traducía el 23505 únicamente para el DNI, y con QR y con código seguía saliendo 500. Además, el script de las pruebas de integración borra la base, así que le agregué un resguardo: exige INTEGRACION_DB=true y se niega a correr contra Supabase.

Commits

- `009914b`, `c2d8e0e`

Sprint 6 — Móvil — Unidad VI — 2026-10-22 al 2026-11-05

Sin actividad propia registrada para este sprint.

Sprint 7 — Integración — Unidad Cierre — 2026-11-05 al 2026-11-19

2026-11-09 · 4 h

Tarjeta(s) Jira: FT1-95 a FT1-104 (y FT1-86, escaneo de secretos, que está en el sprint 5)

Actividad

Cierre de los requisitos de admisibilidad de la entrega: migraciones reproducibles desde una base vacía con su reversión y script de migración; datos de prueba con 25 sedes; almacenamiento de archivos intercambiable y `docker compose up --build`; contrato OpenAPI exportado; escaneo de secretos del historial completo; los diez ADR obligatorios en la plantilla del Anexo B; diagramas C4, DER, despliegue y pipeline; corrección de la carga de clases (429 con 25 sedes) y de los plazos de cancelación mostrados; integración con la reestructura backend/frontend; los tres manuales; README.

Decisión

Escribir un script de migraciones propio sobre los SQL que ya existían, en lugar de las migraciones de TypeORM, porque el esquema usa restricciones que el ORM no genera (índices únicos parciales, exclusión GiST, triggers). Hacer intercambiable el almacenamiento de archivos para que Docker funcione sin Supabase.

Problema o bloqueo

Las migraciones no levantaban una base vacía: 002 repetía lo del 001 y faltaban GERENTE_SEDE y la tabla de push. Con 25 sedes, la pantalla de reservar clases hacía más de 100 pedidos y la API respondía 429.

Commits

- `b101898`, `5f5f7b6`, `a63719c`, `6b33468`, `ab9b759`, `532fb99`, `849e354`, `74894a6`, `6713502`, `458225c`, `7a7b3a3`, `8b605ad`, `86452ab`, `a5b6a03`, `79faad7`, `32a2ca8`
- Del equipo: `1eec536`

Revisión de bitácora (FT1-115, 09/11/2026). Se verificaron los 38 hashes citados contra el historial de `main`. El sprint corresponde a la tarjeta de Jira y la fecha de cada entrada corresponde a la actividad registrada. Las revisiones anteriores de la bitácora quedaron en `72f4f57` y `7766707`.

Sprint 8 — Demo Day — Unidad Cierre — 2026-11-19 al 2026-12-03

Sprint todavía no iniciado.

10. Índice de los manuales entregados y su ubicación en el repositorio

Los manuales no se copian en este Dossier: se entregan en el repositorio, en la carpeta `docs/manuales`, dentro del tag `v1.0-entrega`. Las capturas del manual de usuario están en `docs/manuales/img`.

Tabla 51

Manuales entregados

Manual	Destinatario	Ubicación en el repositorio	Índice según
Manual de usuario	Socio, cliente externo, recepcionista y gerentes	`docs/manuales/manual-de-usuario.md`	Anexo D de los lineamientos
Manual de instalación y operación	Quien instala, despliega y opera el sistema	`docs/manuales/manual-de-instalacion-y-operacion.md`	Anexo E de los lineamientos
Manual técnico	Desarrolladores que mantienen o extienden el sistema	`docs/manuales/manual-tecnico.md`	Anexo F de los lineamientos

Nota. Los índices de cada manual se transcriben a continuación desde sus encabezados.

Manual de usuario

Versión: 1.0 · **Fecha:** 2026-11-09 · **Sistema:** FitZone Sports v1.0 (portal del socio y consola de administración)

1. Qué es FitZone Sports y para qué sirve
2. Cómo leer este manual · Glosario
3. Manual del Socio
4. Manual del Recepcionista / Administrador de sede
5. Manual del Gerente central
6. Preguntas frecuentes

Manual de instalación y operación

Versión: 1.0 · **Fecha:** 2026-11-09 · **Aplica a:** tag `v1.0-entrega`

1. Requisitos previos · Versiones exactas de software
2. Puesta en marcha en entorno de desarrollo
3. Puesta en marcha en entorno de producción
4. Variables de entorno: tabla completa
5. Base de datos: migraciones, reversión y datos semilla

6. Despliegue: estrategia adoptada y procedimiento de reversión
7. Respaldo y restauración
8. Operación degradada: caída de la conexión principal y resincronización
9. Alta de una sede nueva sin detener el sistema
10. Monitoreo: registros, métricas y verificación de estado
11. Resolución de problemas de instalación

Manual técnico

Versión: 1.0 · **Fecha:** 2026-11-09 · **Aplica a:** tag `v1.0-entrega`

1. Visión arquitectónica · Diagramas C4 · Índice de ADR
2. Los cinco módulos funcionales y sus responsabilidades
3. Modelo de datos y convenciones de persistencia
4. Patrones de diseño aplicados y su ubicación en el código
5. Control de concurrencia en las reservas
6. Arquitectura offline del control de acceso
7. Guía de extensión
8. Pruebas: cómo correrlas y cómo leer la cobertura
9. Pipeline de integración y entrega continuas
10. Convenciones de código, ramas y mensajes de commit

11. Anexo: tablero del proyecto exportado

El archivo `docs/entregables/PROGV-2026-TIF-Equipo08-Registro.xlsx` contiene el tablero FT1 exportado de Jira. Se reproducen sus hojas como tablas, comenzando por la nota de transparencia que acompaña a la exportación.

11.1 Nota de transparencia

Nota de transparencia sobre este tablero (exportado el 09/11/2026 desde Jira, proyecto FT1)

1. Este archivo refleja el tablero tal como está en Jira. No se modificaron fechas de creación.

2. Parte del tablero se completó de forma retroactiva, y las fechas de creación lo muestran: 13 tarjetas se crearon el 29/10/2026 y 35 el 09/11/2026 entre las 17:58 y las 17:59, para registrar trabajo ya realizado. Otras 17 tarjetas del 09/11 a las 18:51 y 12 a las 23:44 corresponden a la planificación de cierre y a las brechas detectadas en la auditoría.

3. Jira no guarda en este proyecto la fecha de resolución de las tarjetas, ni estimaciones ni horas trabajadas. La hoja Tarjetas no las incluye porque no existen en el tablero.

4. Las épicas de las unidades I a IV y los sprints 2, 3 y 4 se marcaron como cerrados el 09/11/2026. Los sprints 1 y 11 son el mismo sprint creado dos veces.

5. Las fechas de los sprints 5, 6, 7 y 8 se superponen entre sí en el tablero (el sprint 6 figura antes que el 5). Se conservan tal como están registradas.

6. La hoja Resumen usa fórmulas sobre la hoja Tarjetas.

11.2 Resumen por responsable

Resumen del tablero FT1 (calculado desde la hoja Tarjetas).

Tabla 52

Tarjetas por responsable y estado

Responsable	Listo	En curso	Por hacer	Total
Martín Zanandrea	48	2	12	62
Teo Ava	16	0	3	19
Máximo Retamoso	33	0	5	38
Sin asignar	3	0	3	6

Responsable	Listo	En curso	Por hacer	Total
Total	100	2	23	125

Nota. Valores calculados en la hoja Resumen del archivo, con fórmulas sobre la hoja Tarjetas.

Tabla 53

Fecha de creación de las tarjetas

Cuándo se crearon las tarjetas	Cantidad
Septiembre de 2026	37
1 y 2 de octubre de 2026	8
13 de octubre de 2026	3
29/10/2026	13
09/11/2026 a las 17:58 y 17:59	35
09/11/2026 a las 18:51	17
09/11/2026 a las 23:44	12
Total de tarjetas	125

11.3 Sprints

Tabla 54

Sprints del tablero FT1

ID	Sprint	Estado en Jira	Inicio	Fin	Cierre registrado	Objetivo (resumen)	Tarjetas asignadas
1	Sprint 1- Arquitectura	closed	2026-08-13	2026-08-26	2026-09-03	Primer sprint creado; reemplazado luego por el sprint 11 (mismo nombre lógico)	4
11	Sprint 1 — Arquitectura	closed	2026-08-13	2026-08-27	2026-11-09	Arquitectura base: monolito modular, PostgreSQL y C4 en 3 niveles	4
2	Sprint 2 - Frameworks	closed	2026-09-03	2026-09-17	2026-11-09	Scaffolding NestJS + TypeORM, esquema de BD y primera API con Swagger	22
3	Sprint 3 - Patrones	closed	2026-09-15	2026-09-24	2026-11-09	Strategy, Observer y Repository con control de concurrencia	5
4	Sprint 4 - Componentes	closed	2026-09-15	2026-10-08	2026-11-09	Frontend React + TypeScript + Tailwind, Atomic Design, estado global y pantallas por rol	21
5	Sprint 5 — Testing y CI/CD	active	2026-10-29	2026-11-12		Pruebas unitarias e integración, pipeline de CI/CD y Docker	19
6	Sprint 6 — Móvil	future	2026-10-22	2026-11-05		Acceso móvil offline-first (PWA)	3
7	Sprint 7 — Integración	future	2026-11-05	2026-11-19		Integración final, reestructura backend/frontend y cierre	42
8	Sprint 8 — Demo Day	future	2026-11-19	2026-12-03		Demo funcional y defensa técnica	0

Nota. Hoja Sprints del archivo exportado.

11.4 Tarjetas

Tabla 55

Tarjetas del tablero FT1 (125)

Clave	Tipo	Resumen	Padre	Estado	Responsable	Creada (hora de Argentina)	Sprint
FT1-1	Epic	Unidad I — Arquitectura		Listo	Martín Zanandrea	2026-09-03 23:09	Sin sprint
FT1-2	Epic	Unidad II — Frameworks		Listo	Sin asignar	2026-09-03 23:09	Sin sprint
FT1-3	Epic	Unidad III — Patrones		Listo	Sin asignar	2026-09-03 23:10	Sin sprint
FT1-4	Epic	Unidad IV — Componentes		Listo	Sin asignar	2026-09-03 23:10	Sin sprint
FT1-5	Epic	Unidad V — Testing y CI/CD		Por hacer	Máximo Retamoso	2026-09-03 23:10	Sin sprint
FT1-6	Epic	Unidad VI — Móvil		Por hacer	Sin asignar	2026-09-03 23:10	Sin sprint
FT1-7	Epic	Cierre — Demo Day		Por hacer	Sin asignar	2026-09-03 23:11	Sin sprint
FT1-9	Historia	Debate grupal monolito modular vs microservicios (ADR)	FT1-1	Listo	Máximo Retamoso	2026-09-03 23:24	Sprint 1
FT1-10	Historia	Niveles C4 y ADR fundacionales	FT1-1	Listo	Teo Ava	2026-09-03 23:27	Sprint 1
FT1-11	Historia	Scaffolding NestJS, estructura modular e inyección de dependencias	FT1-2	Listo	Martín Zanandrea	2026-09-03 23:32	Sprint 2
FT1-14	Historia	Documentación de la API con Swagger	FT1-2	Listo	Teo Ava	2026-09-07 19:52	Sprint 2
FT1-15	Historia	ADR - GitHub como repositorio compartido	FT1-2	Listo	Máximo Retamoso	2026-09-08 11:02	Sprint 2
FT1-16	Historia	ADR - Arquitectura monolito modular	FT1-1	Listo	Máximo Retamoso	2026-09-08 11:03	Sprint 1
FT1-17	Historia	ADR - Selección de stack tecnológico principal	FT1-1	Listo	Martín Zanandrea	2026-09-08 11:04	Sprint 1
FT1-18	Historia	ADR - Modelado de datos	FT1-2	Listo	Martín Zanandrea	2026-09-08 11:05	Sprint 2
FT1-19	Historia	ADR - Autenticación JWT y autorización por rol/dueño/sede	FT1-2	Listo	Teo Ava	2026-09-08 11:06	Sprint 2
FT1-20	Historia	ADR - Flujo obligatorio de membresía y pago	FT1-2	Listo	Martín Zanandrea	2026-09-08 11:07	Sprint 2
FT1-21	Historia	ADR - Cálculo de montos server-side	FT1-2	Listo	Teo Ava	2026-09-08 11:08	Sprint 2
FT1-22	Historia	ADR - Sistema de auditoría en dos capas	FT1-2	Listo	Máximo Retamoso	2026-09-08 11:09	Sprint 2
FT1-23	Historia	Patrón Strategy para precio dinámico de canchas (RF-11)	FT1-3	Listo	Martín Zanandrea	2026-09-15 00:19	Sprint 3
FT1-24	Historia	Patrón Observer para lista de espera (RF-08)	FT1-3	Listo	Martín Zanandrea	2026-09-15 00:22	Sprint 3
FT1-25	Historia	Patrón Repository con control de concurrencia (RN-02)	FT1-3	Listo	Martín Zanandrea	2026-09-15 00:23	Sprint 3
FT1-26	Historia	Justificación escrita de los patrones de diseño	FT1-3	Listo	Máximo Retamoso	2026-09-15 00:26	Sprint 3
FT1-28	Historia	Estado global y comunicación entre componentes	FT1-4	Listo	Teo Ava	2026-09-15 00:32	Sprint 4

Clave	Tipo	Resumen	Padre	Estado	Responsable	Creada (hora de Argentina)	Sprint
FT1-30	Historia	Testing y TDD (pruebas unitarias e integración)	FT1-5	Listo	Máximo Retamoso	2026-09-15 00:35	Sprint 5
FT1-31	Historia	Pipeline CI/CD	FT1-5	Listo	Martín Zanandrea	2026-09-15 00:38	Sprint 5
FT1-32	Historia	Reorganizar componentes con Atomic Design	FT1-4	Listo	Máximo Retamoso	2026-09-15 00:42	Sprint 4
FT1-34	Historia	Capa de servicio frontend (consumo de la API)	FT1-4	Listo	Martín Zanandrea	2026-09-15 00:43	Sprint 4
FT1-35	Historia	TypeORM + PostgreSQL (Supabase) y entidades	FT1-2	Listo	Martín Zanandrea	2026-09-21 01:07	Sprint 2
FT1-36	Historia	M1 Usuarios y Membresías (RF-01, RF-02, RF-03)	FT1-2	Listo	Martín Zanandrea	2026-09-21 01:10	Sprint 2
FT1-37	Historia	M2 Gimnasio y Acceso: QR, validación, egreso y aforo (RF-04, RF-05, RN-01)	FT1-2	Listo	Máximo Retamoso	2026-09-21 01:11	Sprint 2
FT1-38	Historia	M3 Clases Grupales (RF-06, RF-07)	FT1-2	Listo	Teo Ava	2026-09-21 01:12	Sprint 2
FT1-39	Historia	M4 Canchas: recursos, reservas y mantenimiento (RF-09, RF-10, RF-12)	FT1-2	Listo	Martín Zanandrea	2026-09-21 01:12	Sprint 2
FT1-40	Historia	M5 Pagos: pasarela mock y comprobante PDF (RF-13, RF-14, RNF-02)	FT1-2	Listo	Máximo Retamoso	2026-09-21 01:15	Sprint 2
FT1-41	Historia	Autenticación JWT, roles y alcance por sede	FT1-2	Listo	Martín Zanandrea	2026-09-21 01:16	Sprint 2
FT1-42	Historia	Módulo de soporte: sedes, franjas, precios e instructores (RNF-04)	FT1-2	Listo	Máximo Retamoso	2026-09-21 01:20	Sprint 2
FT1-43	Historia	Scripts SQL: esquema base y trigger de auditoría	FT1-2	Listo	Martín Zanandrea	2026-09-21 01:27	Sprint 2
FT1-45	Historia	Pantallas de administración (Gerente y Recepcionista)	FT1-4	Listo	Martín Zanandrea	2026-10-01 13:06	Sprint 4
FT1-47	Historia	Pantallas del socio/cliente	FT1-4	Listo	Teo Ava	2026-10-01 13:07	Sprint 4
FT1-48	Historia	Pantallas del home/inicio	FT1-4	Listo	Máximo Retamoso	2026-10-01 13:07	Sprint 4
FT1-49	Historia	Componentes base compartidos (ui.tsx)	FT1-4	Listo	Martín Zanandrea	2026-10-01 13:07	Sprint 4
FT1-50	Historia	Crear nuevo rol Gerente de Sede	FT1-2	Listo	Máximo Retamoso	2026-10-02 19:58	Sprint 4
FT1-51	Subtask	Endpoints	FT1-50	Listo	Martín Zanandrea	2026-10-02 19:59	Sprint 4
FT1-52	Subtask	Modificar BD	FT1-50	Listo	Teo Ava	2026-10-02 19:59	Sprint 4
FT1-53	Subtask	Nuevas pantallas	FT1-50	Listo	Máximo Retamoso	2026-10-02 19:59	Sprint 4
FT1-54	Historia	Pruebas de uso	FT1-5	Listo	Teo Ava	2026-10-13 17:42	Sprint 5
FT1-55	Historia	Pruebas de seguridad	FT1-5	En curso	Martín Zanandrea	2026-10-13 17:44	Sprint 5
FT1-56	Historia	Reporte de pruebas	FT1-5	Listo	Máximo Retamoso	2026-10-13 17:46	Sprint 5
FT1-57	Subtask	Pruebas unitarias del acceso por QR y RN01 (duplicada de FT1-58)	FT1-30	Por hacer	Sin asignar	2026-10-29 13:54	Sin sprint
FT1-58	Historia	Pruebas unitarias del acceso por QR y RN01		Listo	Teo Ava	2026-10-29 13:55	Sprint 5

Clave	Tipo	Resumen	Padre	Estado	Responsable	Creada (hora de Argentina)	Sprint
FT1-59	Historia	Pruebas de integración contra PostgreSQL real (RN01 y RN02)		Listo	Máximo Retamoso	2026-10-29 13:55	Sprint 5
FT1-60	Historia	Pruebas de autenticación y estrategia JWT		Por hacer	Martín Zanandrea	2026-10-29 13:55	Sprint 5
FT1-61	Historia	Checklist de pruebas en producción por rol		Listo	Máximo Retamoso	2026-10-29 13:57	Sprint 5
FT1-62	Historia	Reporte de QA y corrección de hallazgos		Listo	Máximo Retamoso	2026-10-29 13:57	Sprint 5
FT1-63	Historia	Auditoría de roles y alcance por sede (prueba automática)		Listo	Martín Zanandrea	2026-10-29 13:58	Sprint 5
FT1-64	Historia	Corregir error 500 en ingresos simultáneos y endurecer el QR		Listo	Máximo Retamoso	2026-10-29 13:58	Sprint 5
FT1-65	Historia	Informe de auditoría del testing y cobertura		Listo	Teo Ava	2026-10-29 13:58	Sprint 5
FT1-66	Historia	Informe de cobertura generado en cada ejecución del CI		Listo	Martín Zanandrea	2026-10-29 13:58	Sprint 5
FT1-67	Historia	Workflow de GitHub Actions (tsc, pruebas, build)		Listo	Martín Zanandrea	2026-10-29 13:59	Sprint 5
FT1-68	Historia	Despliegue del backend condicionado al CI (Render)		Listo	Martín Zanandrea	2026-10-29 14:00	Sprint 5
FT1-69	Historia	Pruebas unitarias con Vitest		Por hacer	Martín Zanandrea	2026-10-29 19:36	Sprint 5
FT1-70	Historia	Primera versión del cobro en efectivo en mostrador	FT1-2	Listo	Teo Ava	2026-11-09 17:58	Sprint 2
FT1-71	Historia	Corrección del límite de intentos de login	FT1-2	Listo	Teo Ava	2026-11-09 17:58	Sprint 2
FT1-72	Historia	Alcance por sede fail-closed en membresías y pagos	FT1-2	Listo	Martín Zanandrea	2026-11-09 17:58	Sprint 2
FT1-73	Historia	Clases como plantillas recurrentes (ADR-018)	FT1-2	Listo	Martín Zanandrea	2026-11-09 17:58	Sprint 2
FT1-74	Historia	Paginación en los listados de la API	FT1-2	Listo	Martín Zanandrea	2026-11-09 17:59	Sprint 2
FT1-75	Historia	Cambio de contraseña	FT1-3	Listo	Martín Zanandrea	2026-11-09 17:59	Sprint 3
FT1-76	Historia	Panel de administración: sedes, usuarios y reportes	FT1-4	Listo	Teo Ava	2026-11-09 17:59	Sprint 4
FT1-77	Historia	Gestión de reservas de clases y canchas para el gerente	FT1-4	Listo	Teo Ava	2026-11-09 17:59	Sprint 4
FT1-78	Historia	Verificación de email obligatoria (RF-01)	FT1-4	Listo	Martín Zanandrea	2026-11-09 17:59	Sprint 4
FT1-79	Historia	Catálogo de tipos de cancha gestionable (RF-09)	FT1-4	Listo	Martín Zanandrea	2026-11-09 17:59	Sprint 4
FT1-80	Historia	Paso de Externo a Socio al pagar la membresía (ADR-020)	FT1-4	Listo	Martín Zanandrea	2026-11-09 17:59	Sprint 4
FT1-81	Historia	Sesión sincronizada entre pestañas	FT1-4	Listo	Martín Zanandrea	2026-11-09 17:59	Sprint 4
FT1-82	Historia	Cobro del recepcionista, personal y clientes	FT1-4	Listo	Martín Zanandrea	2026-11-09 17:59	Sprint 4
FT1-83	Historia	Notificaciones por panel y email, historial de pagos y comprobantes	FT1-4	Listo	Martín Zanandrea	2026-11-09 17:59	Sprint 4
FT1-84	Historia	Correo con proveedor abstracto (Brevo)	FT1-4	Listo	Martín Zanandrea	2026-11-09 17:59	Sprint 4
FT1-85	Historia	Vigencia de membresías por fechas (ADR-022)	FT1-4	Listo	Martín Zanandrea	2026-11-09 17:59	Sprint 4
FT1-86	Historia	Escaneo de secretos del historial y gitleaks en el CI	FT1-5	Listo	Máximo Retamoso	2026-11-09 17:59	Sprint 5

Clave	Tipo	Resumen	Padre	Estado	Responsable	Creada (hora de Argentina)	Sprint
FT1-87	Historia	CI con Node 22	FT1-5	Listo	Martín Zanandrea	2026-11-09 17:59	Sprint 5
FT1-88	Historia	Portal como PWA instalable (ADR-011)	FT1-6	Listo	Martín Zanandrea	2026-11-09 17:59	Sprint 6
FT1-89	Historia	Notificaciones push con VAPID	FT1-6	Listo	Martín Zanandrea	2026-11-09 17:59	Sprint 6
FT1-90	Historia	Estados sin conexión en el cliente	FT1-6	Listo	Martín Zanandrea	2026-11-09 17:59	Sprint 6
FT1-91	Historia	Validación del QR sin conexión en el terminal de sede (RNF-01)	FT1-6	En curso	Martín Zanandrea	2026-11-09 17:59	Sprint 7
FT1-92	Historia	Ingreso manual por DNI	FT1-7	Listo	Martín Zanandrea	2026-11-09 17:59	Sprint 7
FT1-93	Historia	Reestructura a backend/frontend	FT1-7	Listo	Martín Zanandrea	2026-11-09 17:59	Sprint 7
FT1-94	Historia	Contrato OpenAPI exportado (swagger.json)	FT1-7	Listo	Martín Zanandrea	2026-11-09 17:59	Sprint 7
FT1-95	Historia	Migraciones reproducibles con reversión	FT1-7	Listo	Máximo Retamoso	2026-11-09 17:59	Sprint 7
FT1-96	Historia	Datos de prueba con 25 sedes	FT1-7	Listo	Máximo Retamoso	2026-11-09 17:59	Sprint 7
FT1-97	Historia	Docker compose y almacenamiento local de archivos	FT1-7	Listo	Máximo Retamoso	2026-11-09 17:59	Sprint 7
FT1-98	Historia	Los diez ADR obligatorios en la plantilla del Anexo B	FT1-7	Listo	Máximo Retamoso	2026-11-09 17:59	Sprint 7
FT1-99	Historia	Diagramas C4, DER, despliegue y pipeline	FT1-7	Listo	Máximo Retamoso	2026-11-09 17:59	Sprint 7
FT1-100	Historia	Corrección de la carga de clases (429 con 25 sedes)	FT1-7	Listo	Máximo Retamoso	2026-11-09 17:59	Sprint 7
FT1-101	Historia	Plazos de cancelación iguales a los del servidor	FT1-7	Listo	Máximo Retamoso	2026-11-09 17:59	Sprint 7
FT1-102	Historia	Manuales de usuario, de instalación y técnico	FT1-7	Listo	Máximo Retamoso	2026-11-09 17:59	Sprint 7
FT1-103	Historia	README con los seis puntos de la cátedra	FT1-7	Listo	Máximo Retamoso	2026-11-09 17:59	Sprint 7
FT1-104	Historia	Integración con la reestructura	FT1-7	Listo	Máximo Retamoso	2026-11-09 17:59	Sprint 7
FT1-105	Subtask	Claves ES256 del QR y verificación dual (HS256/ES256)	FT1-91	Listo	Martín Zanandrea	2026-11-09 18:51	Sprint 7
FT1-106	Subtask	Migración 012: columnas de acceso offline y tabla de eventos	FT1-91	Listo	Martín Zanandrea	2026-11-09 18:51	Sprint 7
FT1-107	Subtask	Endpoints de clave pública y snapshot de la sede	FT1-91	Listo	Martín Zanandrea	2026-11-09 18:51	Sprint 7
FT1-108	Subtask	Endpoint de sincronización de ingresos offline	FT1-91	Listo	Martín Zanandrea	2026-11-09 18:51	Sprint 7
FT1-109	Subtask	Terminal: réplica local, verificación del QR y cola de ingresos	FT1-91	Listo	Martín Zanandrea	2026-11-09 18:51	Sprint 7
FT1-110	Subtask	Terminal: integración en la pantalla de control de acceso	FT1-91	Listo	Martín Zanandrea	2026-11-09 18:51	Sprint 7
FT1-111	Subtask	Documentación del modo offline (ADR-003, ADR-007, secuencia y manual)	FT1-91	Por hacer	Martín Zanandrea	2026-11-09 18:51	Sprint 7
FT1-112	Historia	Planificación del proyecto: exportar Jira y armar Planificacion.pdf	FT1-7	Listo	Martín Zanandrea	2026-11-09 18:51	Sprint 7
FT1-113	Historia	Bitácora individual de Martín Zanandrea	FT1-7	Por hacer	Martín Zanandrea	2026-11-09 18:51	Sprint 7

Clave	Tipo	Resumen	Padre	Estado	Responsable	Creada (hora de Argentina)	Sprint
FT1-114	Historia	Bitácora individual de Teo Ava	FT1-7	Listo	Teo Ava	2026-11-09 18:51	Sprint 7
FT1-115	Historia	Bitácora individual de Máximo Retamoso: completar y verificar hashes	FT1-7	Listo	Máximo Retamoso	2026-11-09 18:51	Sprint 7
FT1-116	Historia	Diagramas faltantes: clases, secuencia, estados y actividad	FT1-7	Por hacer	Máximo Retamoso	2026-11-09 18:51	Sprint 7
FT1-117	Historia	Evidencia de requisitos no funcionales en docs/rnf	FT1-7	Por hacer	Máximo Retamoso	2026-11-09 18:51	Sprint 7
FT1-118	Historia	Verificar docker compose up desde cero y agregar smoke test en el CI	FT1-7	Listo	Máximo Retamoso	2026-11-09 18:51	Sprint 7
FT1-119	Historia	Descartada: el equipo siempre fue de tres integrantes y la cátedra lo conoce	FT1-7	Por hacer	Martín Zanandrea	2026-11-09 18:51	Sprint 7
FT1-120	Historia	Cierre de entrega: checklist del Anexo G, tag v1.0-entrega y subida a Moodle	FT1-7	Por hacer	Martín Zanandrea	2026-11-09 18:51	Sprint 7
FT1-121	Historia	Restaurar Supabase, verificar el backend en Render y monitor de actividad	FT1-7	Por hacer	Martín Zanandrea	2026-11-09 18:51	Sprint 7
FT1-122	Historia	Auditoría (@Auditable) de acciones sensibles	FT1-7	Por hacer	Martín Zanandrea	2026-11-09 23:44	Sprint 7
FT1-123	Historia	Actualizar dependencias vulnerables del backend (npm audit)	FT1-7	Por hacer	Martín Zanandrea	2026-11-09 23:44	Sprint 7
FT1-124	Historia	Actualizar dependencias vulnerables del frontend (npm audit)	FT1-7	Por hacer	Teo Ava	2026-11-09 23:44	Sprint 7
FT1-125	Historia	CI: ejecutar las pruebas del frontend (Vitest)	FT1-7	Listo	Máximo Retamoso	2026-11-09 23:44	Sprint 7
FT1-126	Historia	Protección de main con checks obligatorios y colaboradora de solo lectura para la cátedra	FT1-7	Por hacer	Martín Zanandrea	2026-11-09 23:44	Sprint 7
FT1-127	Historia	README: roles, declaración de IA y bibliotecas, credenciales del seed	FT1-7	Por hacer	Máximo Retamoso	2026-11-09 23:44	Sprint 7
FT1-128	Historia	Actualizar documentación obsoleta (manual de instalación, informe histórico)	FT1-7	Listo	Máximo Retamoso	2026-11-09 23:44	Sprint 7
FT1-129	Historia	Segunda ronda de auditoría contra el Anexo G	FT1-7	Por hacer	Máximo Retamoso	2026-11-09 23:44	Sprint 7
FT1-130	Historia	Prueba de punta a punta del modo offline en producción y su evidencia	FT1-7	Por hacer	Martín Zanandrea	2026-11-09 23:44	Sprint 7
FT1-131	Historia	Estructura del frontend frente al punto 4: verificar y documentar	FT1-7	Por hacer	Teo Ava	2026-11-09 23:44	Sprint 7
FT1-132	Historia	Lint: errores del frontend y saltos de línea del backend	FT1-7	Por hacer	Teo Ava	2026-11-09 23:44	Sprint 7
FT1-133	Historia	Preparación de la defensa: demo, plan B y escenario de canchas de tenis	FT1-7	Por hacer	Martín Zanandrea	2026-11-09 23:44	Sin sprint

Nota. Hoja Tarjetas del archivo exportado; las fechas están en hora de Argentina.

Referencias

Brown, S. (s. f.). *The C4 model for visualizing software architecture*. https://c4model.com

Equipo 08. (2026). *Tablero FT1 FitZoneSports1* [Registro de proyecto]. Jira. https://fitzone-sports.atlassian.net

Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Meta Platforms, Inc. (s. f.). *React documentation*. https://react.dev

NestJS. (s. f.). *NestJS documentation*. https://docs.nestjs.com

Nygard, M. (2011, 15 de noviembre). *Documenting architecture decisions*. Cognitect. https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions

PostgreSQL Global Development Group. (s. f.). *PostgreSQL 17 documentation*. https://www.postgresql.org/docs/17/

TypeORM. (s. f.). *TypeORM documentation*. https://typeorm.io

Universidad Nacional de Entre Ríos, Facultad de Ciencias de la Administración. (2026a). *Trabajo integrador final: Caso FitZone Sports* [Enunciado]. Programación V.

Universidad Nacional de Entre Ríos, Facultad de Ciencias de la Administración. (2026b). *Lineamientos de entrega del TIF FitZone Sports, 2.º cuatrimestre 2026* [Documento de cátedra]. Programación V.

Vite. (s. f.). *Vite documentation*. https://vite.dev/guide/